

Scientific Computing



Teaching Assistens - TAs:

Shirin Gul Zaidi

Brian Damith Ratnasinghe

Maryna Mesiura

Teachers



Johan

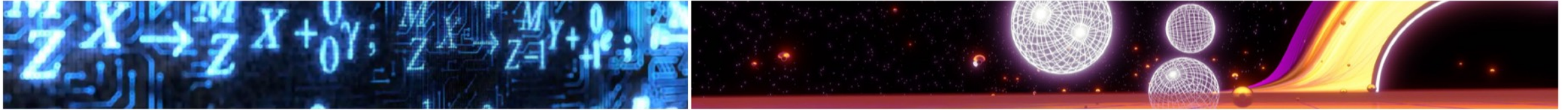


Gabriel



Kurt

Scientific Computing

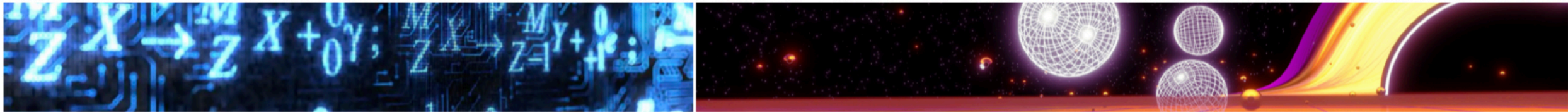


Johan Samsing
Faculty of Science
University of Copenhagen

Lecture W1 - L1

Practical Stuff!

Welcome to Scientific Computing 2026



On this page you can find links to all the basic information about course. Any important messages will appear under [Announcements](#). Please make sure to set your notifications on Absalon (click Accounts and then Notifications) so you get notifications when new announcements are posted. If this is your first visit, please read any existing announcements.

Course materials, plans for the course days can be found through the [Modules](#) in the left hand pane. Here you can also find slides, descriptions of assignments, readings and so forth.

(Go to website)

Topics of this lecture:

- What is Scientific Computing?
 - Examples from Astrophysics
 - Steps in a Scientific Computing Project
-
- Accuracy of Computational Simulations
 - Sources of Approximation
 - Definition of different types of errors
 - Condition number

What is Scientific Computing?

Design and analysis of algorithms for the solution of mathematical problems in science and engineering

Scientific Computing deals with continuous quantities:

Functions and equations of variables such as:
time, position, probability, energy, and so on...

Example:

The motion of 3 black holes: How and will they collide?

How do we solve this problem?

What forces do we need?



Example:

The motion of 3 black holes: How and will they collide?

How do we solve this problem?

$$\mathbf{a} = \mathbf{a}_0 + c^{-2}\mathbf{a}_2 + c^{-4}\mathbf{a}_4 + c^{-5}\mathbf{a}_5 + \mathcal{O}(c^{-6})$$

$$\mathbf{a}_0 = -\frac{Gm_2}{r_{12}^2}\hat{\mathbf{r}}_{12}$$

$$\begin{aligned} \mathbf{a}_5 = & \frac{4}{5} \frac{G^2 m_1 m_2}{r_{12}^3} \left[\left(\frac{2Gm_1}{r_{12}} - \frac{8Gm_2}{r_{12}} - v_{12}^2 \right) \mathbf{v}_{12} \right. \\ & \left. + (\hat{\mathbf{r}}_{12} \cdot \mathbf{v}_{12}) \left(\frac{52Gm_2}{3r_{12}} - \frac{6Gm_1}{r_{12}} + 3v_{12}^2 \right) \hat{\mathbf{r}}_{12} \right] \end{aligned}$$

(Mathematical approximation)



Example:

The motion of 3 black holes: How and will they collide?

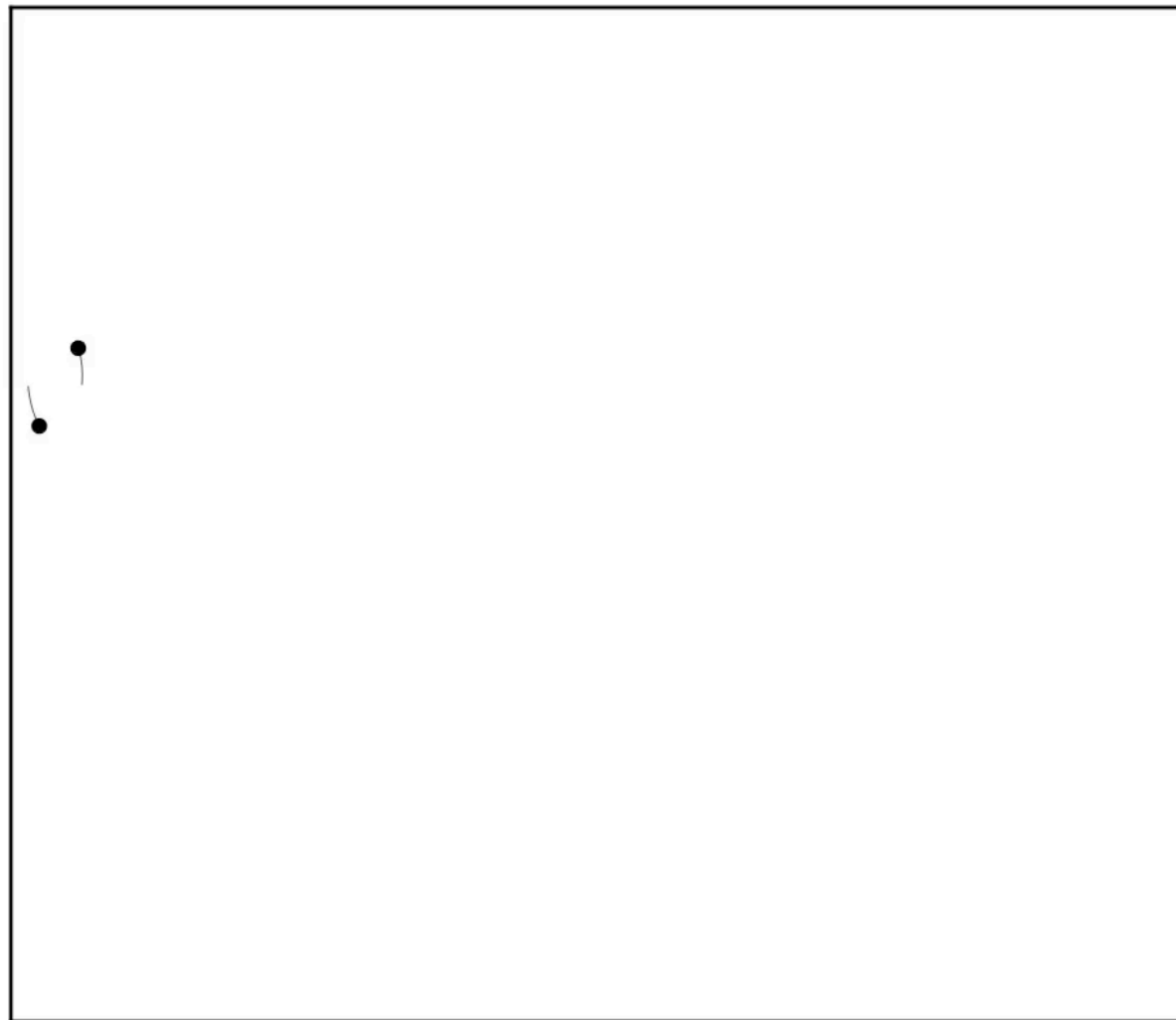
How do we solve this problem?

$$\mathbf{a} = \mathbf{a}_0 + c^{-2}\mathbf{a}_2 + c^{-4}\mathbf{a}_4 + c^{-5}\mathbf{a}_5 + \mathcal{O}(c^{-6})$$

$$\mathbf{a}_0 = -\frac{Gm_2}{r_{12}^2}\hat{\mathbf{r}}_{12}$$

$$\begin{aligned} \mathbf{a}_5 = & \frac{4}{5} \frac{G^2 m_1 m_2}{r_{12}^3} \left[\left(\frac{2Gm_1}{r_{12}} - \frac{8Gm_2}{r_{12}} - v_{12}^2 \right) \mathbf{v}_{12} \right. \\ & \left. + (\hat{\mathbf{r}}_{12} \cdot \mathbf{v}_{12}) \left(\frac{52Gm_2}{3r_{12}} - \frac{6Gm_1}{r_{12}} + 3v_{12}^2 \right) \hat{\mathbf{r}}_{12} \right] \end{aligned}$$

What have we not taken into account?



Example:

The motion of 3 black holes: How and will they collide?

How do we solve this problem?

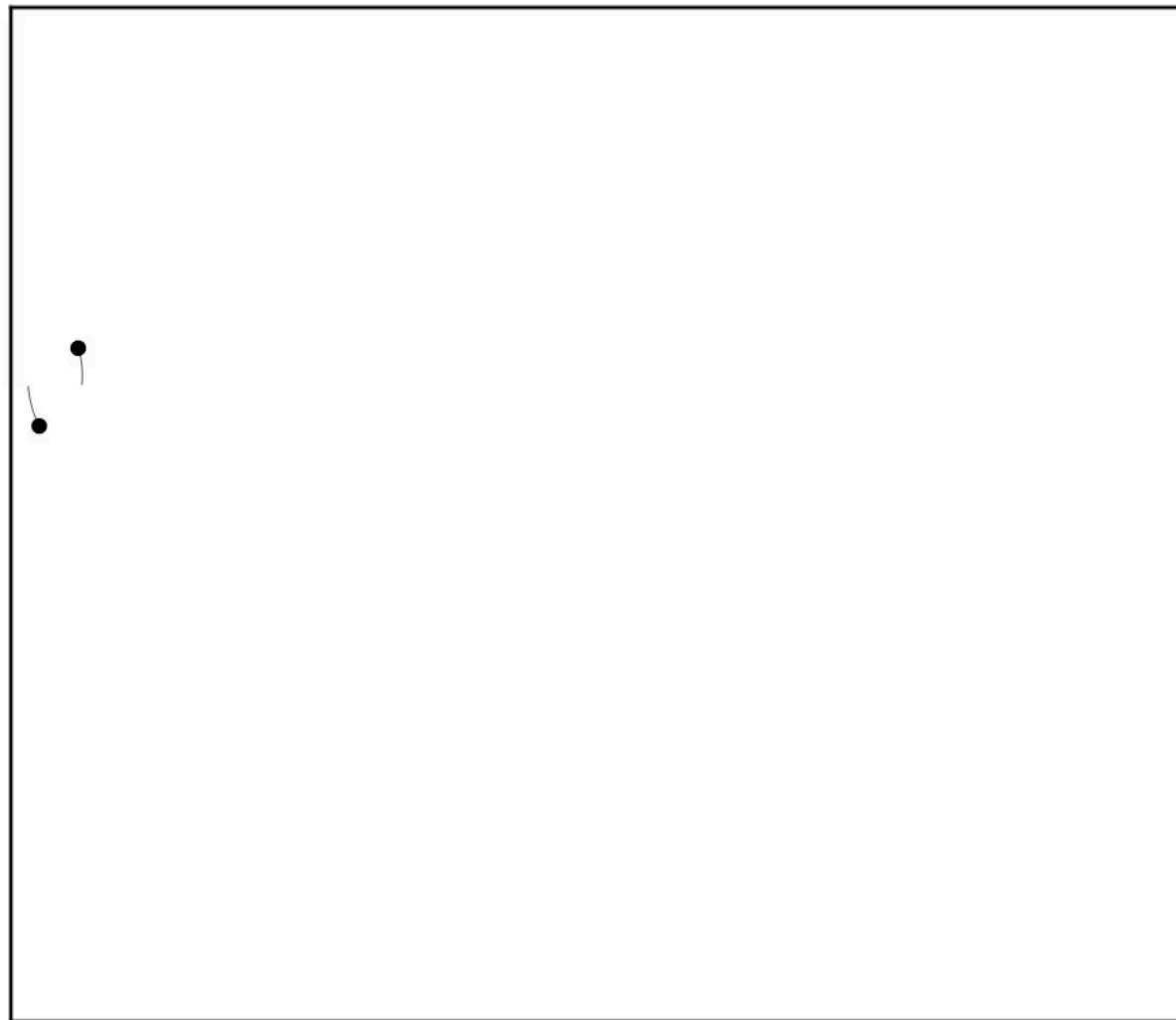
$$\mathbf{a} = \mathbf{a}_0 + c^{-2}\mathbf{a}_2 + c^{-4}\mathbf{a}_4 + c^{-5}\mathbf{a}_5 + \mathcal{O}(c^{-6})$$

$$\mathbf{a}_0 = -\frac{Gm_2}{r_{12}^2}\hat{\mathbf{r}}_{12}$$

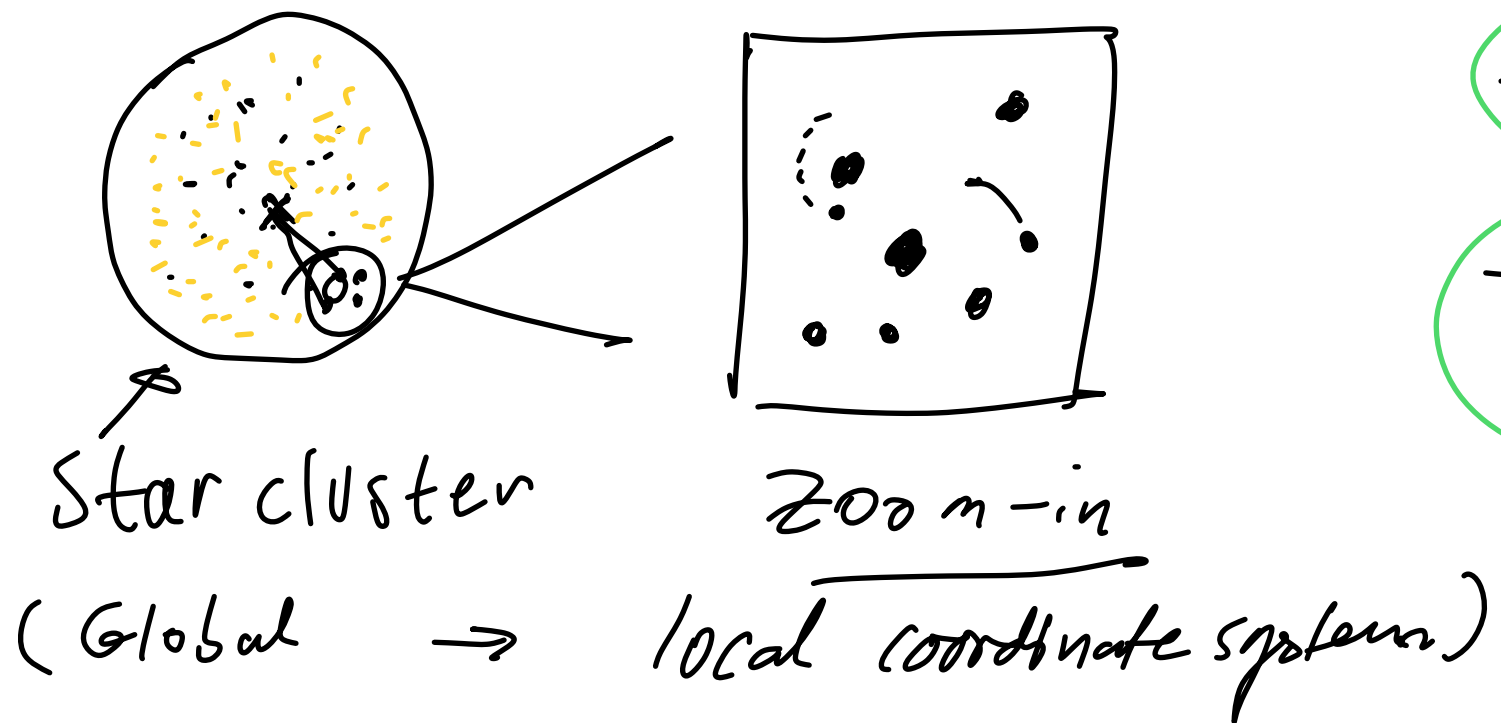
$$\mathbf{a}_5 = \frac{4}{5}\frac{G^2m_1m_2}{r_{12}^3}\left[\left(\frac{2Gm_1}{r_{12}} - \frac{8Gm_2}{r_{12}} - v_{12}^2\right)\mathbf{v}_{12} + (\hat{\mathbf{r}}_{12} \cdot \mathbf{v}_{12})\left(\frac{52Gm_2}{3r_{12}} - \frac{6Gm_1}{r_{12}} + 3v_{12}^2\right)\hat{\mathbf{r}}_{12}\right]$$

How do we optimise the precision?

How do we quantify accumulated errors?



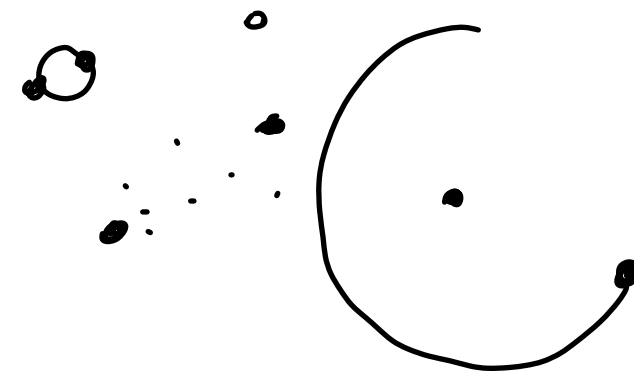
Evolution of positions in time: $x(t)$



– Positions (roundoff errors)

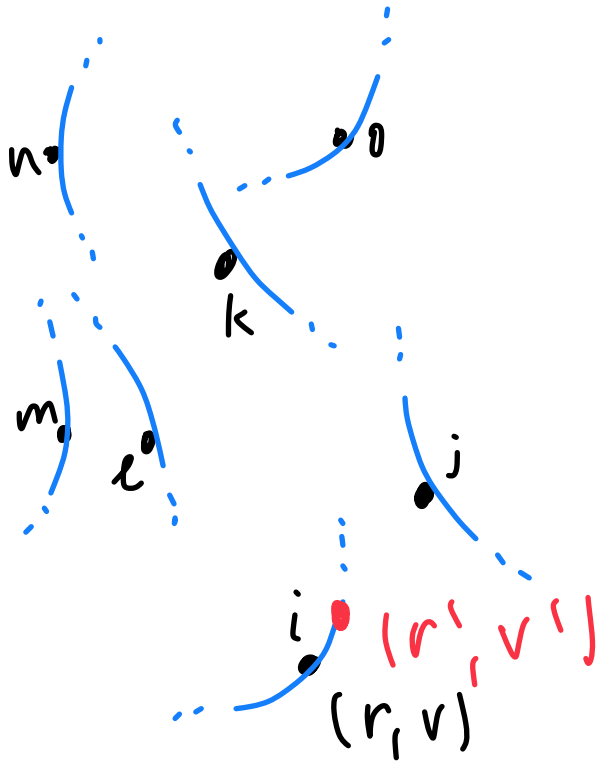
– time (same or individual?)

Different length and timescales:



Computational Cost:

Euler Integrator:



$$r_i' = r_i + v_i dt$$

$$v_i' = v_i + a_i dt$$

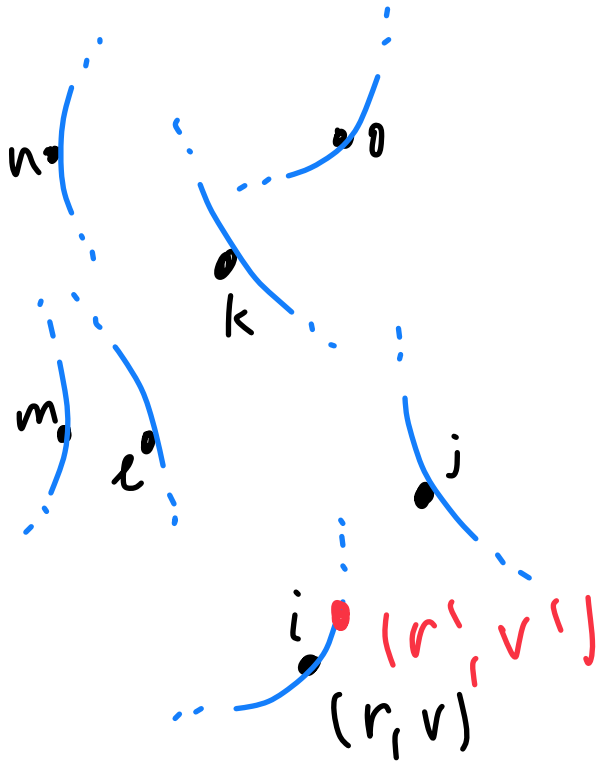
$$a_i \sim \frac{1}{m_i} \sum_j F_{ij}$$

(real motion)

(calc motion after time dt)

Computational Cost:

Euler Integrator:



(real motion)

(calc motion after time dt)

$$r_i' = r_i + v_i dt$$

$$v_i' = v_i + a_i dt$$

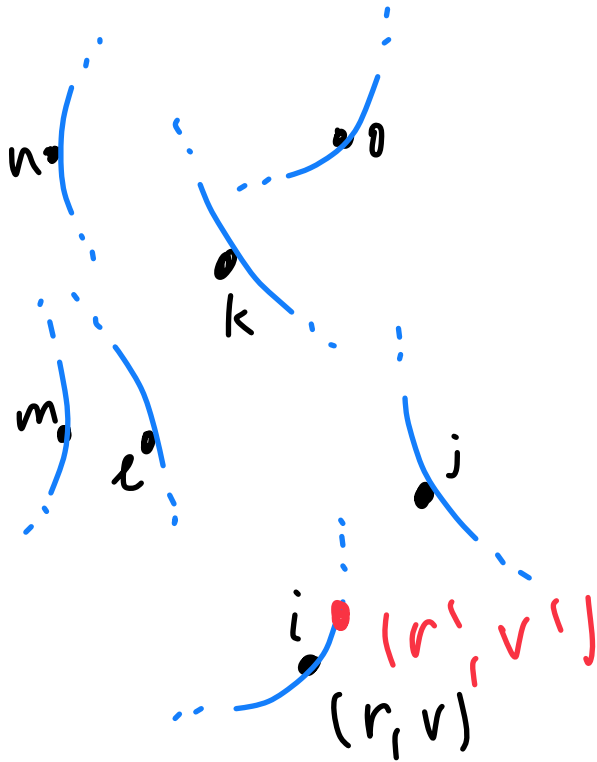
$$a_i \sim \frac{1}{m_i} \sum_j F_{ij}$$

$$F_{ij} = \begin{bmatrix} F_{11} & F_{12} & F_{13} & F_{14} & \dots \\ F_{21} & F_{22} & F_{23} & F_{24} & \\ F_{31} & F_{32} & F_{33} & F_{34} & \\ F_{41} & F_{42} & F_{43} & F_{44} & \\ \vdots & & & & \ddots \\ & & & & & F_{NN} \end{bmatrix}$$

For N -body problem.

Computational Cost:

Euler Integrator:



(real motion)

(calc motion after time dt)

$$r_i' = r_i + v_i dt$$

$$v_i' = v_i + a_i dt$$

$$a_i \sim \frac{1}{m_i} \sum_j F_{ij}$$

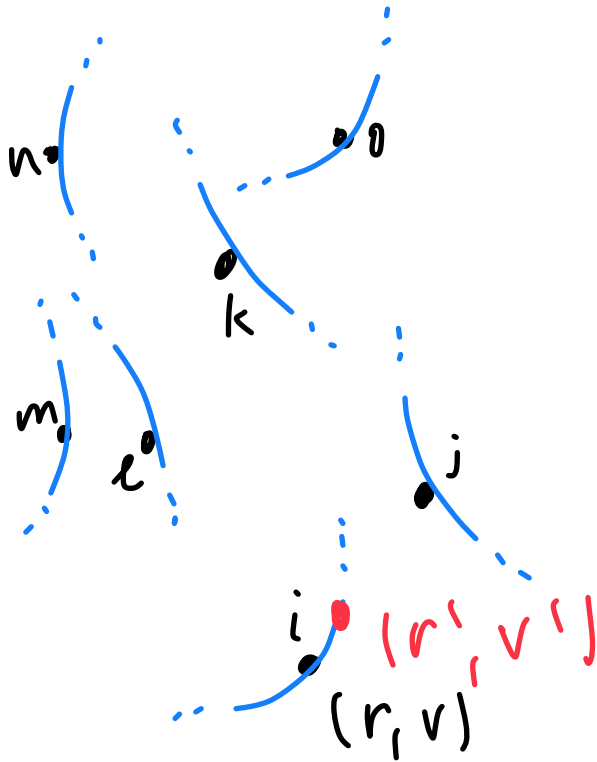
$$F_{ij} = \begin{bmatrix} F_{11} & F_{12} & F_{13} & F_{14} & \dots \\ F_{21} & F_{22} & F_{23} & F_{24} & \\ F_{31} & F_{32} & F_{33} & F_{34} & \\ F_{41} & F_{42} & F_{43} & F_{44} & \\ \vdots & & & & \ddots \\ & & & & & F_{NN} \end{bmatrix}$$

For N -body problem.

How many computations
are needed pr. time?

Computational Cost:

Euler Integrator:



(real motion)

(calc motion after time dt)

$$r'_i = r_i + v_i dt$$

$$v'_i = v_i + a_i dt$$

$$a_i \sim \frac{1}{m_i} \sum_j F_{ij}$$

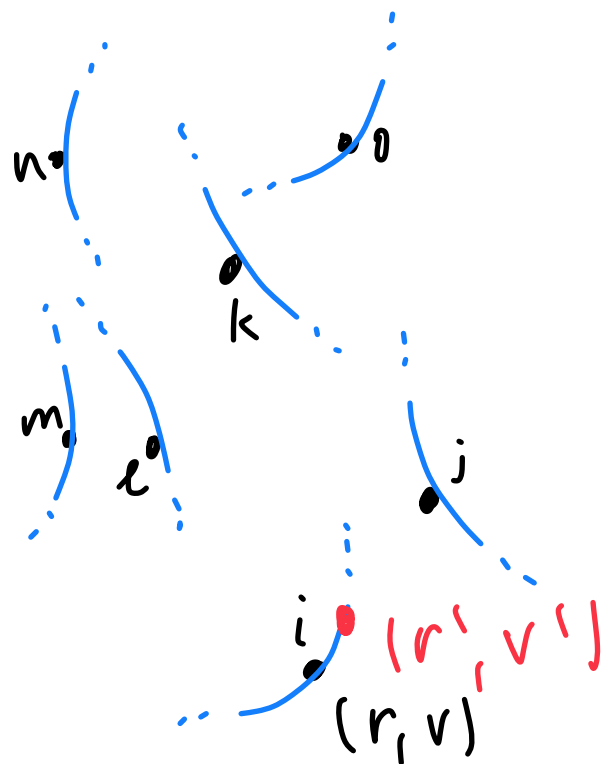
$$F_{ij} = \begin{bmatrix} F_{11} & F_{12} & F_{13} & F_{14} & \dots \\ F_{21} & F_{22} & F_{23} & F_{24} & \\ F_{31} & F_{32} & F_{33} & F_{34} & \\ F_{41} & F_{42} & F_{43} & F_{44} & \\ \vdots & & & & \ddots \end{bmatrix}$$

For N -body problem.

How many computations
are needed pr. time?

Computational Cost:

Euler Integrator:



(real motion)

(calc motion after time dt)

$$r'_i = r_i + v_i dt$$

$$v'_i = v_i + a_i dt$$

$$a_i \sim \frac{1}{m_i} \sum_j F_{ij}$$

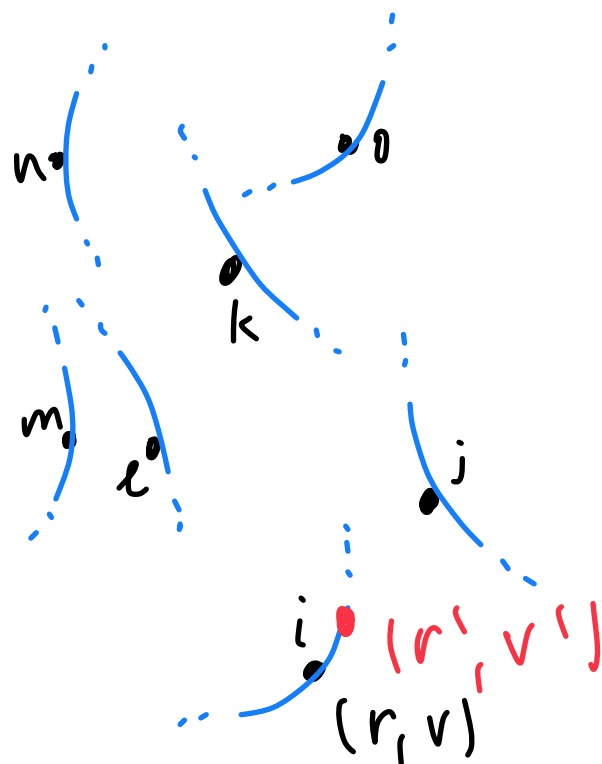
$$F_{ij} = \begin{bmatrix} F_{11} & F_{12} & F_{13} & F_{14} & \dots \\ F_{21} & F_{22} & F_{23} & F_{24} & \\ F_{31} & F_{32} & F_{33} & F_{34} & \\ F_{41} & F_{42} & F_{43} & F_{44} & \\ \vdots & \vdots & \vdots & \vdots & \ddots \end{bmatrix}$$

For N -body problem.

How many computations are needed pr. time?

Computational Cost:

Euler Integrator:



(real motion)

(calc motion after time dt)

$$r'_i = r_i + v_i dt$$

$$v'_i = v_i + a_i dt$$

$$a_i \sim \frac{1}{m_i} \sum_j F_{ij}$$

$$F_{ij} = \begin{bmatrix} F_{11} & F_{12} & F_{13} & F_{14} & \dots \\ F_{21} & F_{22} & F_{23} & F_{24} & \\ F_{31} & F_{32} & F_{33} & F_{34} & \\ F_{41} & F_{42} & F_{43} & F_{44} & \\ \vdots & \vdots & \vdots & \vdots & \ddots \end{bmatrix}$$

For N -body problem.

$$N \sim \frac{N^2 - N}{2}, \text{ for } N \gg 1.$$

$$\propto N^2 //$$

What is Scientific Computing?

Scientific Computing deals with problems/equations which cannot be solved exactly in a finite number of steps. For example as we have seen: Newtons equations of motion.

Other scenarios:

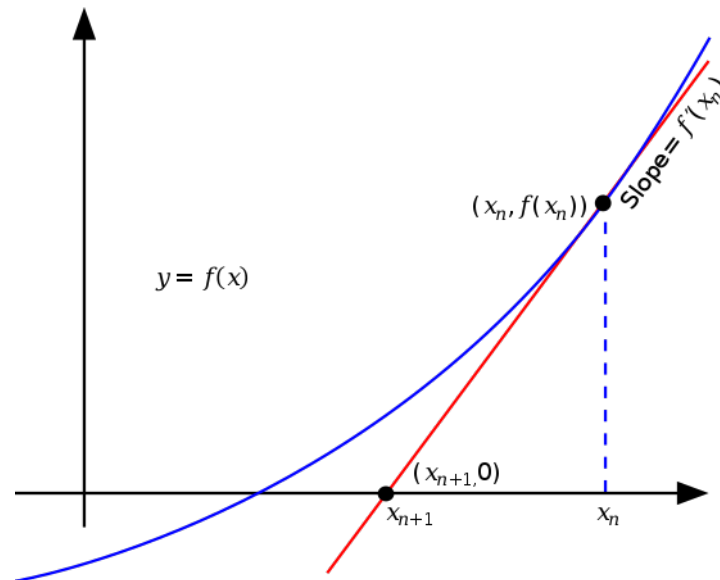
- Iterative Algorithms. For example: Finding 0'th points, or local/global maxima.

What is Scientific Computing?

Scientific Computing deals with problems/equations which cannot be solved exactly in a finite number of steps. For example as we have seen: Newtons equations of motion.

Other scenarios:

- Iterative Algorithms. For example: Finding 0'th points, or local/global maxima.

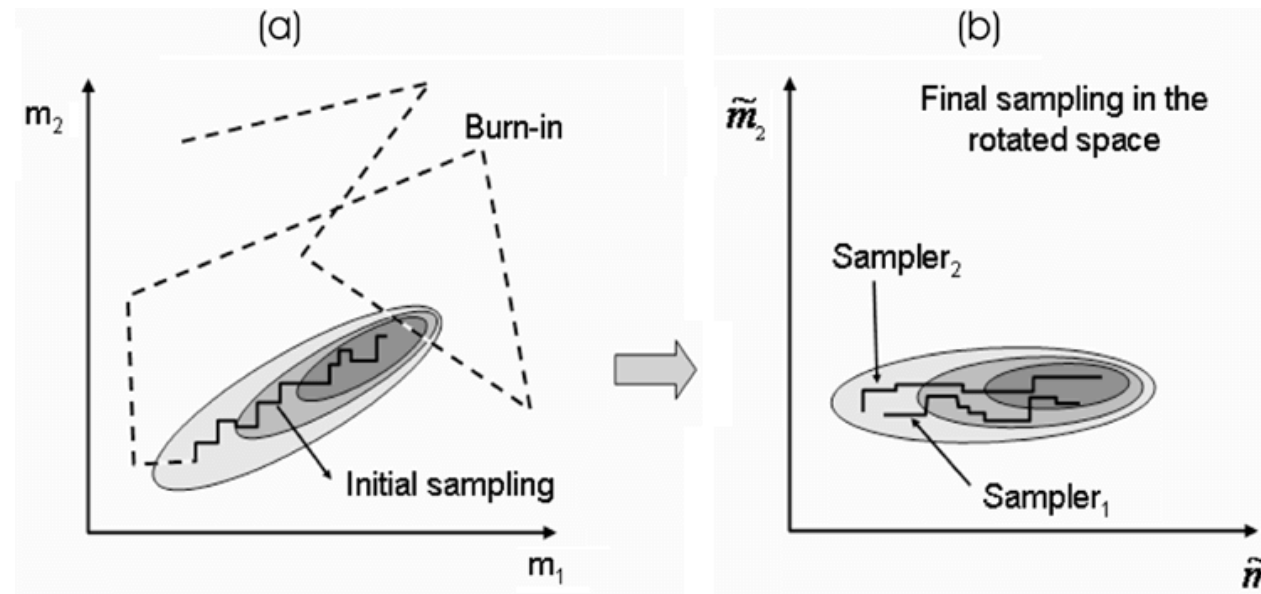


What is Scientific Computing?

Scientific Computing deals with problems/equations which cannot be solved exactly in a finite number of steps. For example as we have seen: Newtons equations of motion.

Other scenarios:

- Iterative Algorithms. For example: Finding 0'th points, or local/global maxima.



What is Scientific Computing?

Scientific Computing deals with problems/equations which cannot be solved exactly in a finite number of steps. For example as we have seen: Newtons equations of motion.

Other scenarios:

- Iterative Algorithms. For example: Finding 0'th points, or local/global maxima.

NOTE: Most real numbers cannot be represented exactly in a finite computer!

Examples for Scientific Computer Problems:

Design of materials and devices.

- Rapid or virtual prototyping by computer simulations.

Understanding and prediction of natural phenomena:

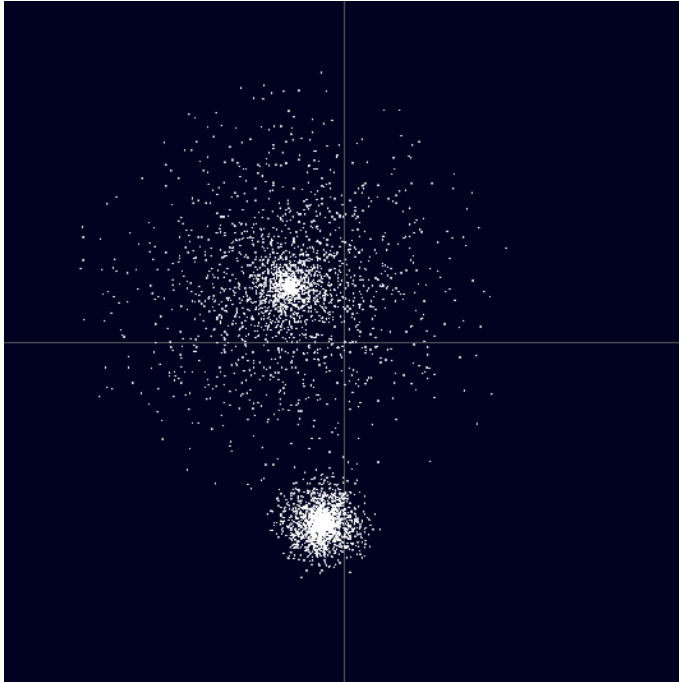
- Computational simulations of systems
- Study of situations which are difficult or impossible to study experimentally:

Example!



$t = 0.1 \text{ Gyr}$

The N-body problem:

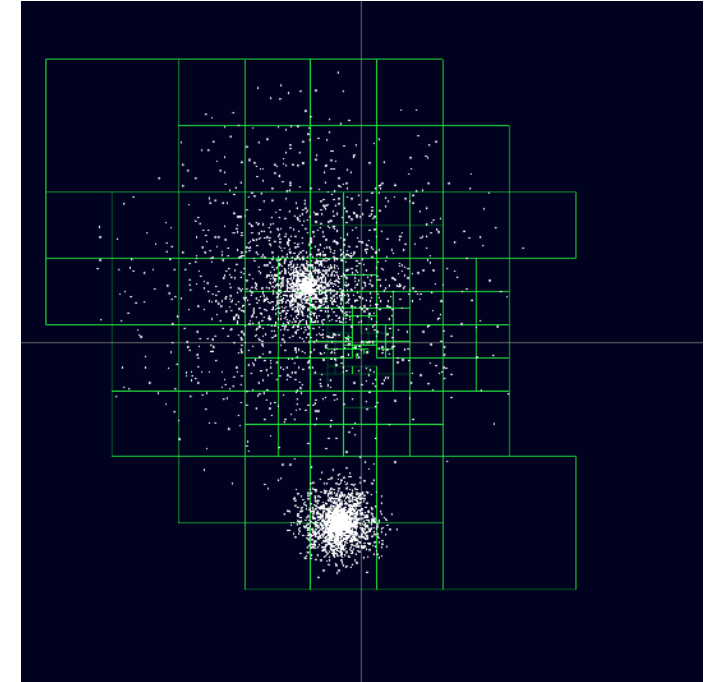
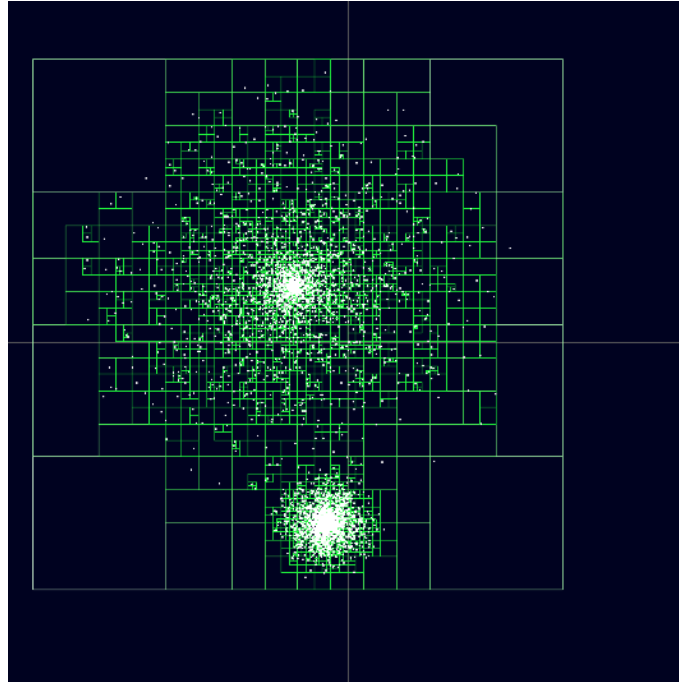
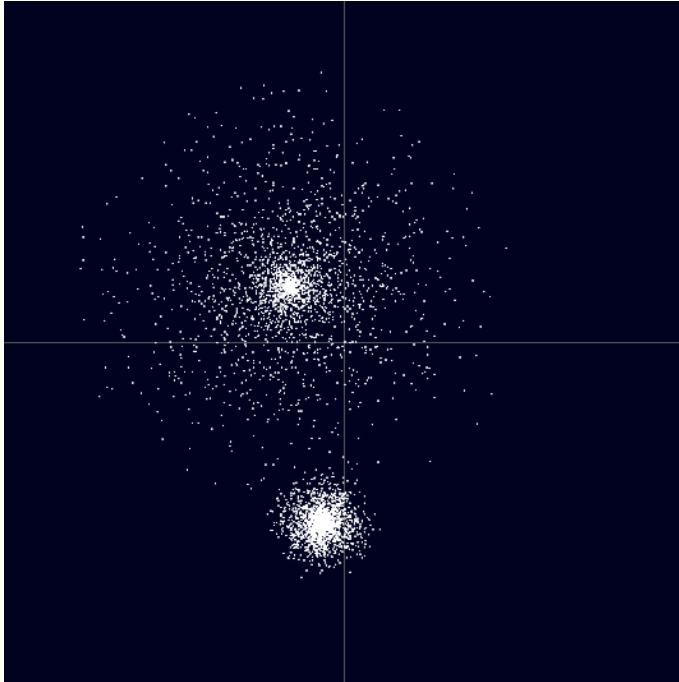


$$\mathcal{N} \propto N^2$$

(N = number of objects)

total the number of computations: $\mathcal{N}^2$

Optimise the algorithm: The Barnes-Hut Algorithm



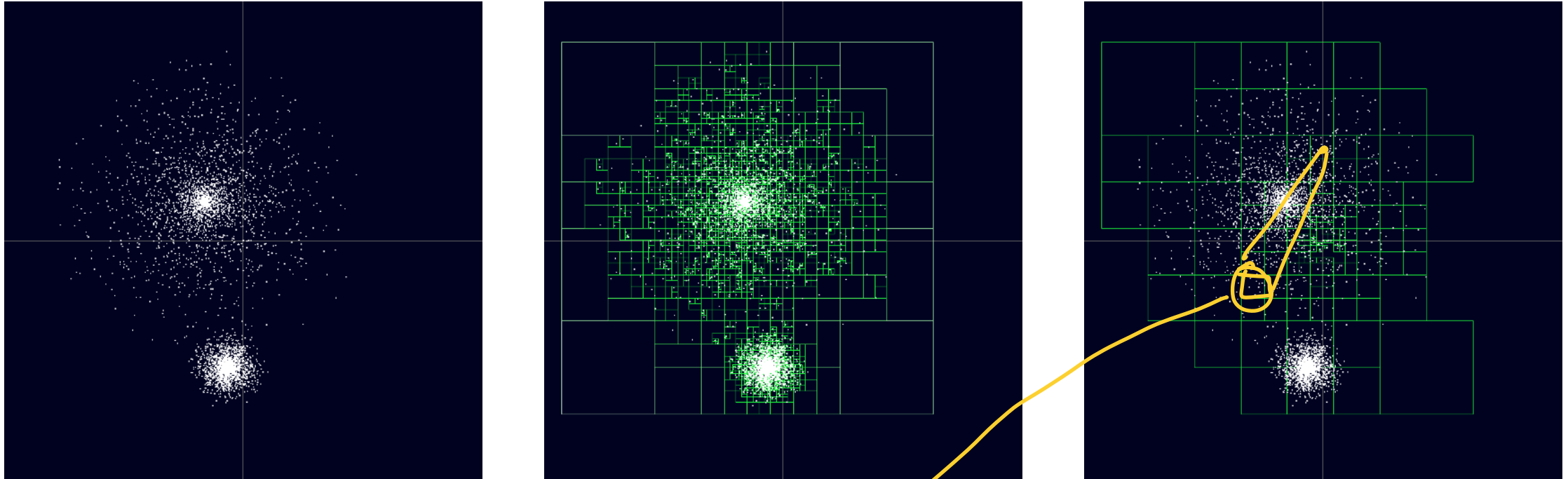
Reduces the number of computations:

$$N \propto N \log N$$

(Often associated with sorting algorithms)

Approximations and accuracy in this algorithm:

Optimise the algorithm: The Barnes-Hut Algorithm



Reduces the number of computations:

$$N \propto N \log N \quad (\text{Often associated with sorting algorithms})$$

Approximations and accuracy in this algorithm:



$$\theta \sim s/d. \quad (\theta: N \propto N^2)$$

Steps in a Scientific Computing Project

1. Formulation of a mathematical model for the system: one or more equations
2. Design of an algorithm to solve the equations numerically
3. Implementation of the algorithm
4. Test and debug of the implementation
5. Evaluation / validation of the mathematic model via simulations of know properties of the system
6. Application of the model and program for study new properties/processes

Steps 1,5,6 require expertise in the field of the applications

Steps 2,3,4 require expertise in scientific computing

Accuracy of Computational Simulations

Simulations replace/complement measurements:

- What are the error bars of the results of simulations?

Conditioning of a mathematical model:

- Well-posed mathematical problem: The problem has a unique solution which depends continuously on the problem data.
- Sensitivity of the mathematical problem: Continuous, but strong dependence on input data.

Numerical solution of the problem on a computer:

- Stable algorithm: Does not increase the sensitivity of the problem.

Optimisation and Stability

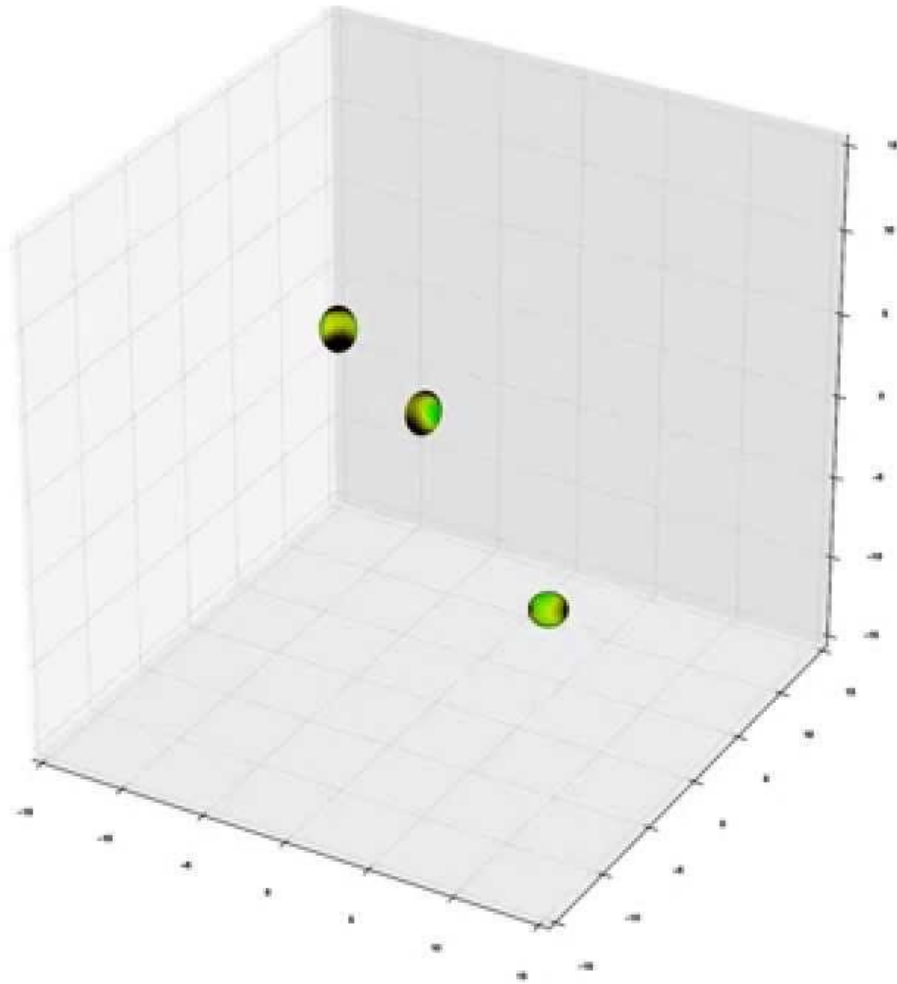
Conditioning of a mathematical model:

Well-posed mathematical problem: The problem has a unique solution which depends continuously on the problem data.

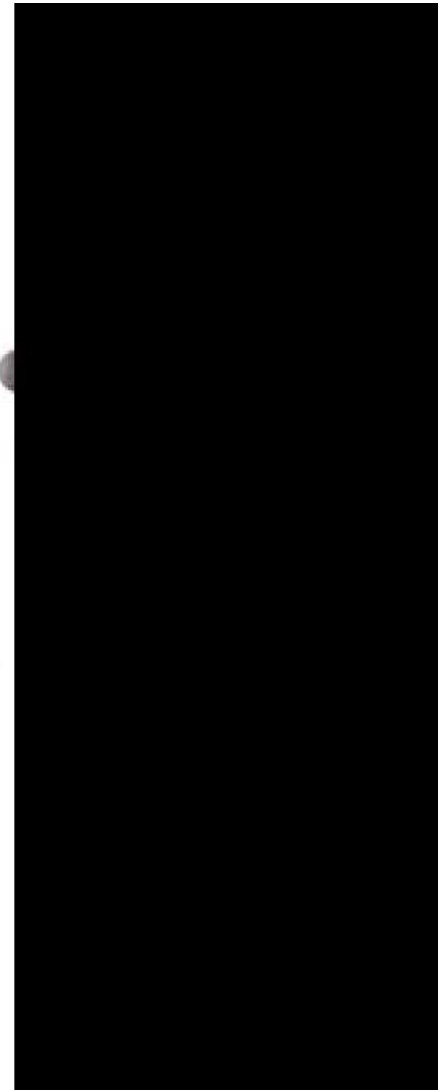
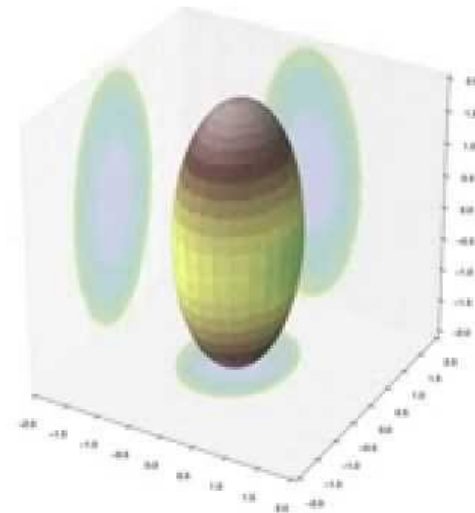
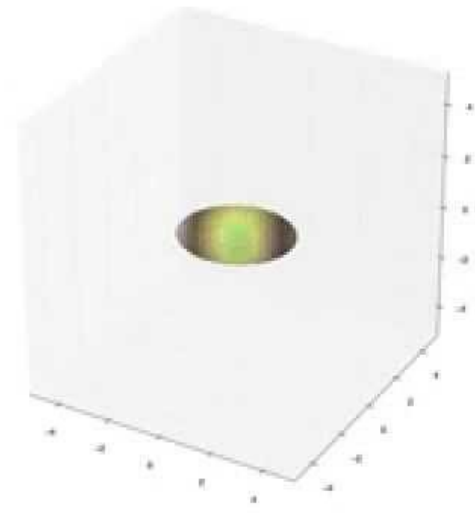
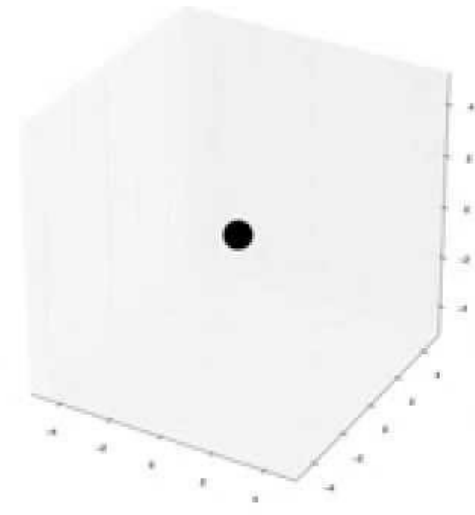
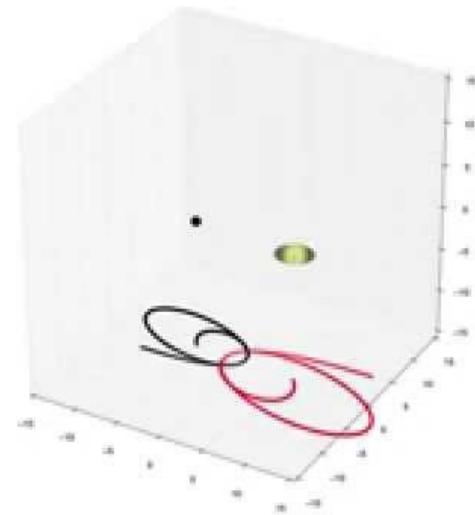
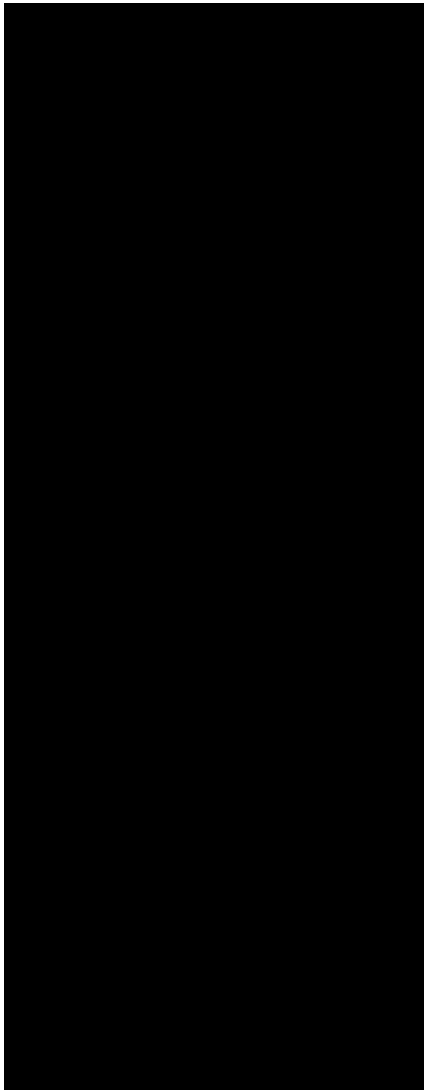
Sensitivity of the mathematical problem: Continuous, but strong dependence on input data.

EXAMPLE

Optimisation and Stability

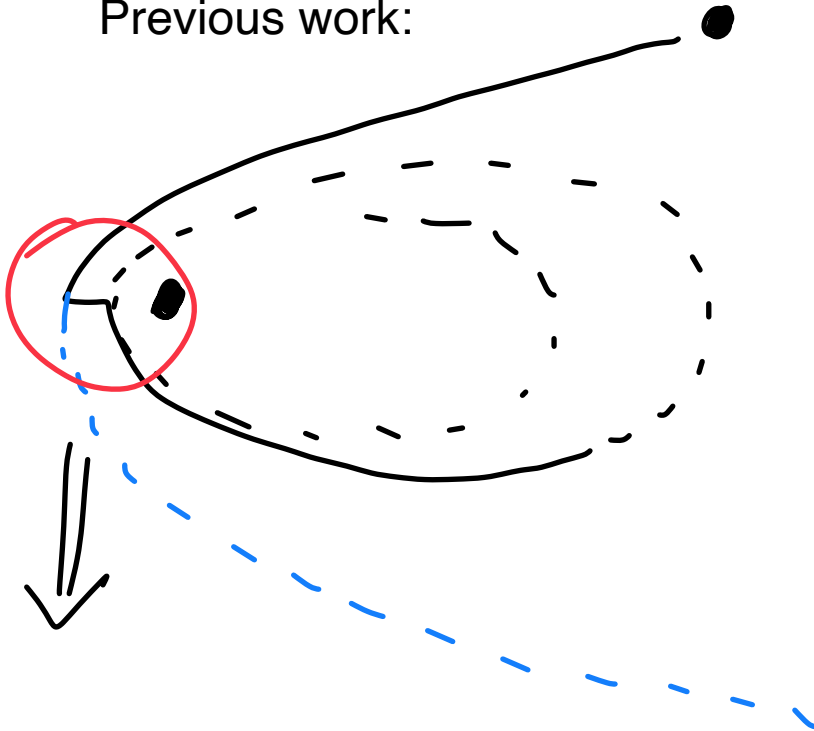


Optimisation and Stability

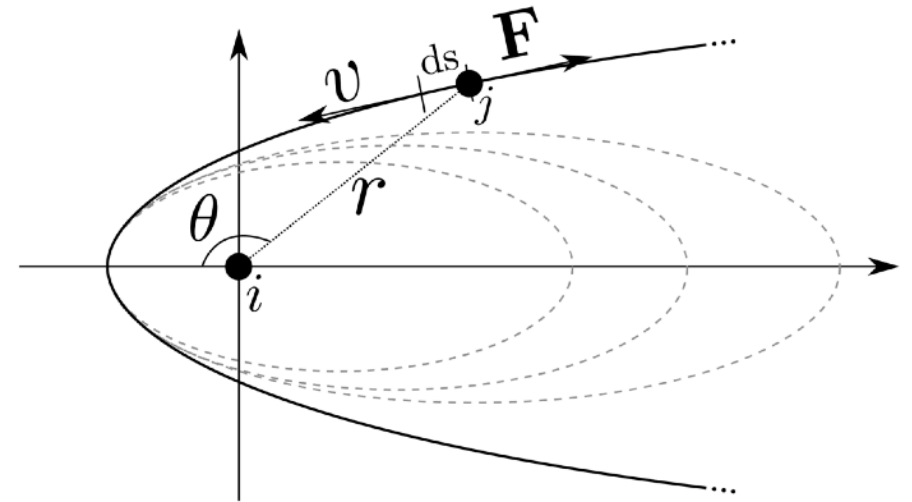


Optimisation and Stability

Previous work:

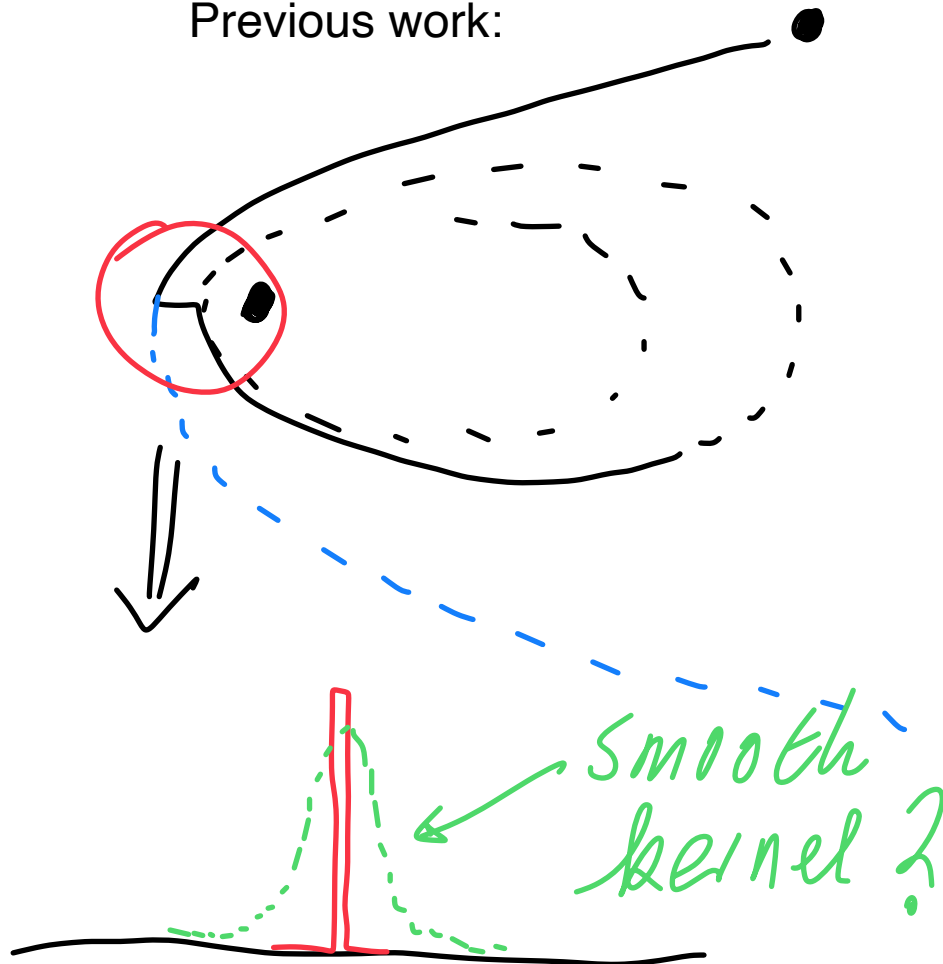


New idea:

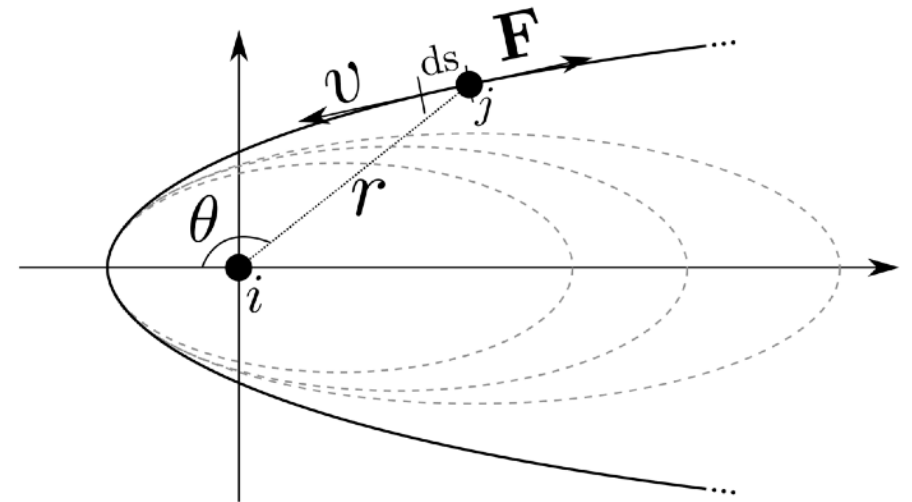


Optimisation and Stability

Previous work:



New idea:



Optimisation and Stability

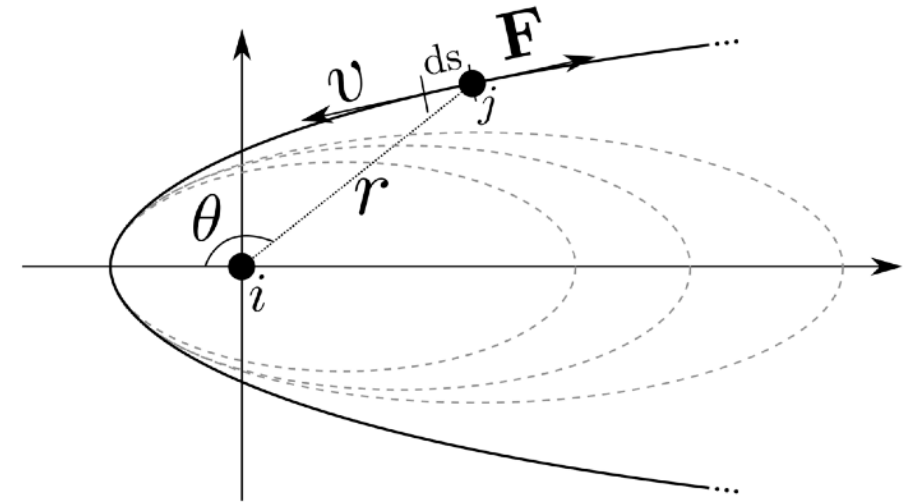
$$\Delta E \approx \int_{-\theta_0}^{+\theta_0} F \frac{ds}{dt} \frac{dt}{d\theta} d\theta, \quad (1)$$

where dt is the differential change in time, θ is the true anomaly, and $\theta_0 = \pi$ for a bound orbit and $= \cos^{-1}(-1/e)$ for an unbound orbit (see Figure 1). The term ds/dt is simply the relative velocity between the two interacting objects, denoted by v , which can be written as,

$$\frac{ds}{dt} = v = \left[2GM \left(\frac{1}{r} - \frac{1}{2a} \right) \right]^{1/2}, \quad (2)$$

where $M = m_i + m_j$ is the total mass of the two interacting objects, referred to as i and j , and r is their relative distance. The term $dt/d\theta$ can be derived from Kepler's relation $r = a(1 - e^2)/(1 + e \cos \theta)$, from which it follows,

$$\frac{dt}{d\theta} = \frac{[a(1 - e^2)]^{3/2}}{(GM)^{1/2}} \frac{1}{(1 + e \cos \theta)^2}. \quad (3)$$



Optimisation and Stability

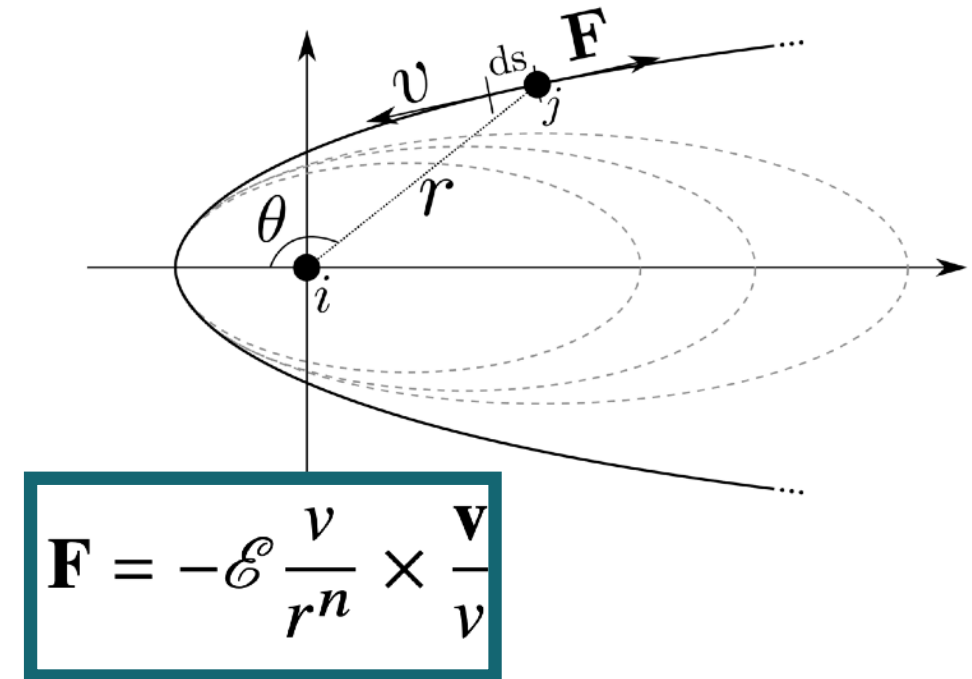
$$\Delta E \approx \int_{-\theta_0}^{+\theta_0} F \frac{ds}{dt} \frac{dt}{d\theta} d\theta, \quad (1)$$

where dt is the differential change in time, θ is the true anomaly, and $\theta_0 = \pi$ for a bound orbit and $= \cos^{-1}(-1/e)$ for an unbound orbit (see Figure 1). The term ds/dt is simply the relative velocity between the two interacting objects, denoted by v , which can be written as,

$$\frac{ds}{dt} = v = \left[2GM \left(\frac{1}{r} - \frac{1}{2a} \right) \right]^{1/2}, \quad (2)$$

where $M = m_i + m_j$ is the total mass of the two interacting objects, referred to as i and j , and r is their relative distance. The term $dt/d\theta$ can be derived from Kepler's relation $r = a(1 - e^2)/(1 + e \cos \theta)$, from which it follows,

$$\frac{dt}{d\theta} = \frac{[a(1 - e^2)]^{3/2}}{(GM)^{1/2}} \frac{1}{(1 + e \cos \theta)^2}. \quad (3)$$



Optimisation and Stability

$$\mathbf{F} = -\mathcal{E} \frac{v}{r^n} \times \frac{\mathbf{v}}{v}$$

$$\Delta E \approx \int_{-\theta_0}^{+\theta_0} F \frac{ds}{dt} \frac{dt}{d\theta} d\theta, \quad (1)$$

where dt is the differential change in time, θ is the true anomaly, and $\theta_0 = \pi$ for a bound orbit and $= \cos^{-1}(-1/e)$ for an unbound orbit (see Figure 1). The term ds/dt is simply the relative velocity between the two interacting objects, denoted by v , which can be written as,

$$\frac{ds}{dt} = v = \left[2GM \left(\frac{1}{r} - \frac{1}{2a} \right) \right]^{1/2}, \quad (2)$$

where $M = m_i + m_j$ is the total mass of the two interacting objects, referred to as i and j , and r is their relative distance. The term $dt/d\theta$ can be derived from Kepler's relation $r = a(1 - e^2)/(1 + e \cos \theta)$, from which it follows,

$$\frac{dt}{d\theta} = \frac{[a(1 - e^2)]^{3/2}}{(GM)^{1/2}} \frac{1}{(1 + e \cos \theta)^2}. \quad (3)$$

$$\mathcal{E} = \Delta E \times \frac{1}{2} \frac{[a(1 - e^2)]^{n-1/2}}{(GM)^{1/2} \mathcal{J}(e, n)}, \quad (5)$$

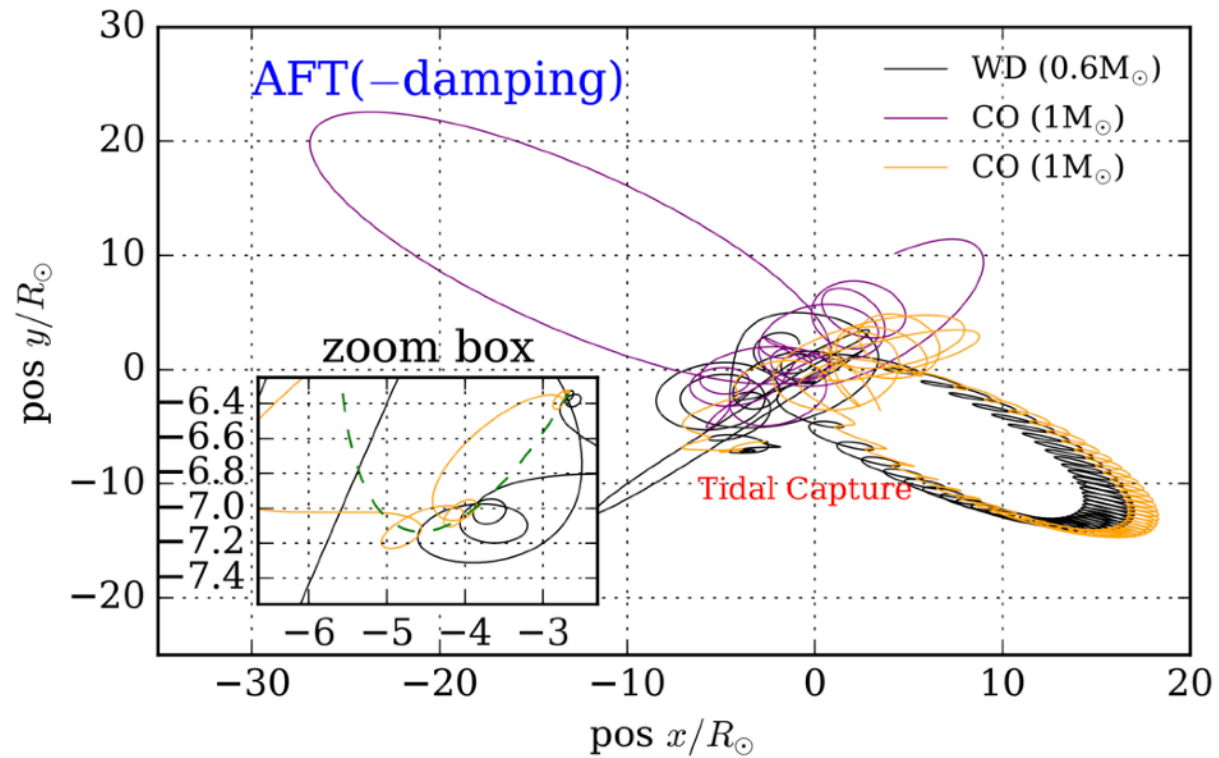
where $\mathcal{J}(e, n)$ is the solution to the following integral

$$\mathcal{J}(e, n) = \int_{-\theta_0}^{+\theta_0} \frac{(1 + e \cos \theta) - (1 - e^2)/2}{(1 + e \cos \theta)^{2-n}} d\theta. \quad (6)$$

This factor $\mathcal{J}(e, n)$ can be written out in closed form for any value of n . In this paper we will study the performance for two different values of n , namely for $n = 4$ and $n = 10$. For these two cases $\mathcal{J}$ evaluates to,

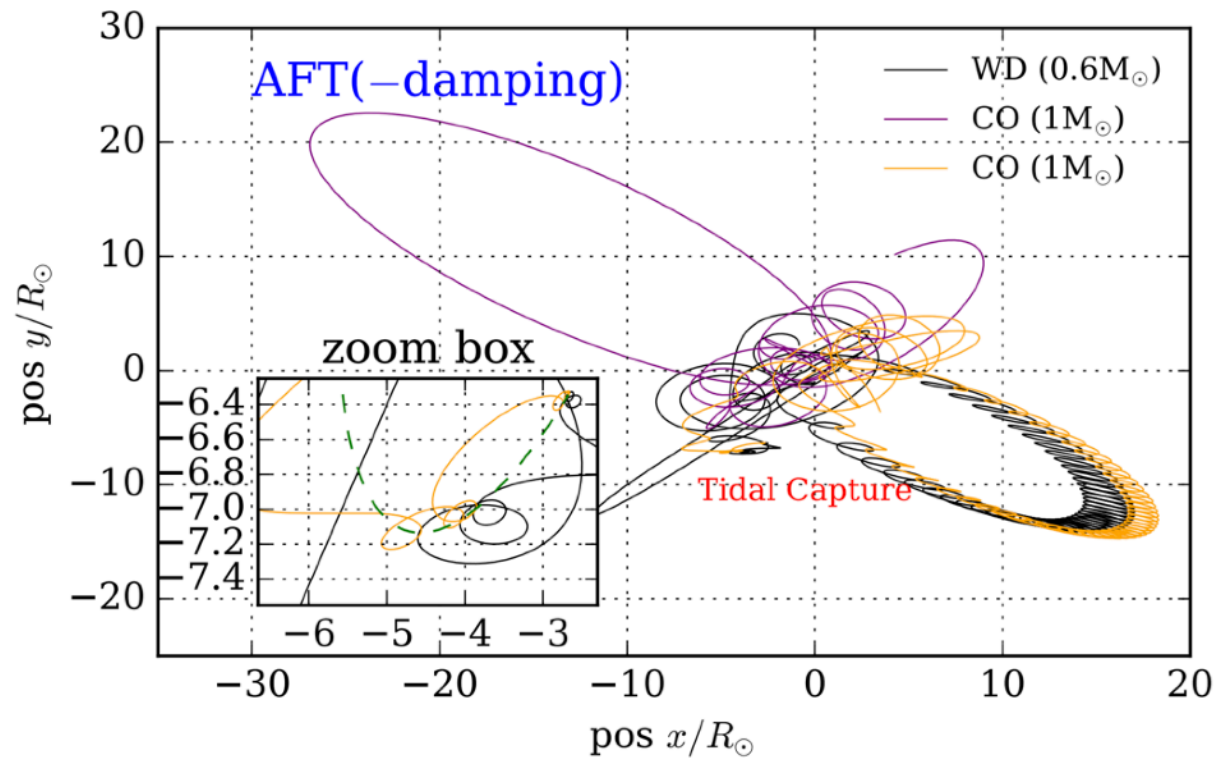
$$\mathcal{J}(e, n = 4, 10) = \begin{cases} \frac{\pi}{2} (2 + 7e^2 + e^4), & \text{for } n = 4 \\ \frac{\pi}{128} (128 + 2944e^2 + 10528e^4 + 8960e^6 + 1715e^8 + 35e^{10}), & \text{for } n = 10, \end{cases} \quad (7)$$

Optimisation and Stability

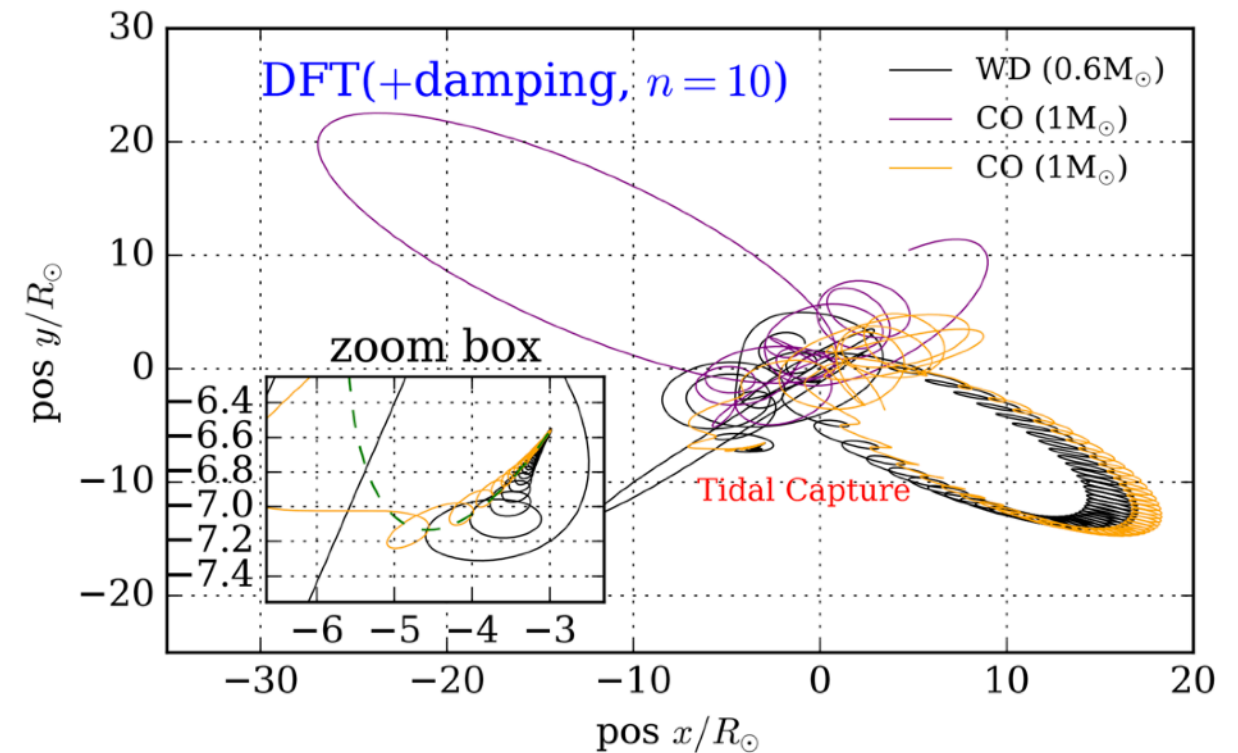


“Affine model”: very computationally heavy (from video).”

Optimisation and Stability



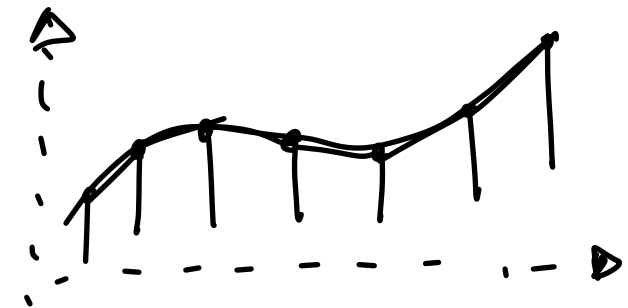
“Affine model”: very computationally heavy (from video).”



- $>10^3$ times faster
- More stable
- Continues evolution
- Computational effective in N-body code

Sources of Approximations

- **In the mathematical model** (we just saw an example of this)
- **In the input data to the simulations:**
 - *(beyond our control)*
- **Truncation and discretisation of the mathematical model:**
 - *Replacing integrals or infinite sums by finite sums*
 - *Replacing derivatives by finite differences*
- **Rounding:**
 - *The representation of real numbers is limited to a finite number of digits*
 - *Arithmetic on real numbers is inherently inexact*



Study of these effects: *Error Analysis!*

Definition of Errors - 1

Absolute and relative errors:

absolute error = approximate value - true value

relative error = absolute error / true value

Approximate value = (true value)(1+relative error)*

Precision:

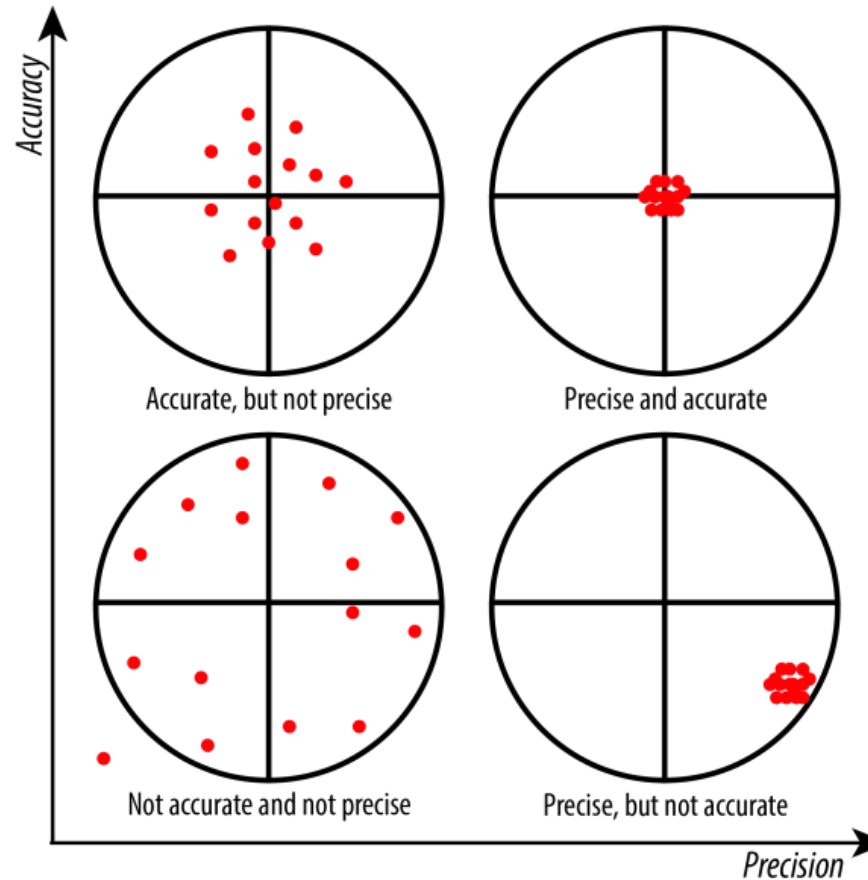
Significant digits: The leading and all following non-zero digits.

Definition of Errors - 1

Precision and accuracy:

is 3.25260378204729746 precise and/or accurate value for π ?

Definition of Errors - 1



Precision and accuracy:

is 3.25260378204729746 precise and/or accurate value for π ?

Definition of Errors - 2

Data and computational errors:

For example in one dimension:

The computation of the algorithm can be considered as a function, which maps input data on output data.

But: we have only inexact input data, and compute only an approximation.

We make a total error:

Note here that the propagated data error is independent of the algorithm!

$$f: x \rightarrow f(x)$$

$$\text{inexact data: } \hat{x} \\ \text{compute only app: } \hat{f}$$

$$= \hat{f}(\hat{x}) - f(x)$$

$$= (\hat{f}(\hat{x}) - f(\hat{x})) + (f(\hat{x}) - f(x))$$

computational

propagated data

Definition of Errors - 3

An example:

Inexact input data

Approximation to the computation

Final approximate result

Total absolute error

Exact result of inexact input data

Computational error

Propagated data error

$$f(x) = \sin\left(\frac{\pi}{8}\right) \approx 0.3827$$

$$\pi \approx 3$$

$$\hat{f}(x) = \sin(x) \approx x$$

$$\hat{f}(\hat{x}) = \sin\left(\frac{\pi}{8}\right) \approx \frac{\pi}{8} \approx \frac{3}{8} \approx 0.3750$$

$$\hat{f}(\hat{x}) - f(x) = 0.3750 - 0.3827 = -0.0077$$

$$f(\hat{x}) = \sin\left(\frac{3}{8}\right) \approx 0.3663$$

$$(\hat{f}(\hat{x}) - f(\hat{x})) \approx 0.3750 - 0.3663 = 0.0087$$

$$(f(\hat{x}) - f(x)) \approx 0.3663 - 0.3827 = -0.0164$$

Definition of Errors - 4

Truncation and rounding errors: - a further division of the computational error.

Truncation error:

- Error due to an approximate algorithm but using infinite precision exact arithmetic.
- E.g. finite differences instead of derivatives, or finite sums instead of infinite series.

Rounding error:

- Difference between the result of an algorithm assuming exact arithmetic and the actual result using finite-precision rounded arithmetic.

Example

$$f'(x) \approx \frac{f(x+h) - f(x)}{h}$$

$$f(x+h) = f(x) + f'(x)h + f''(\theta)h^2/2$$

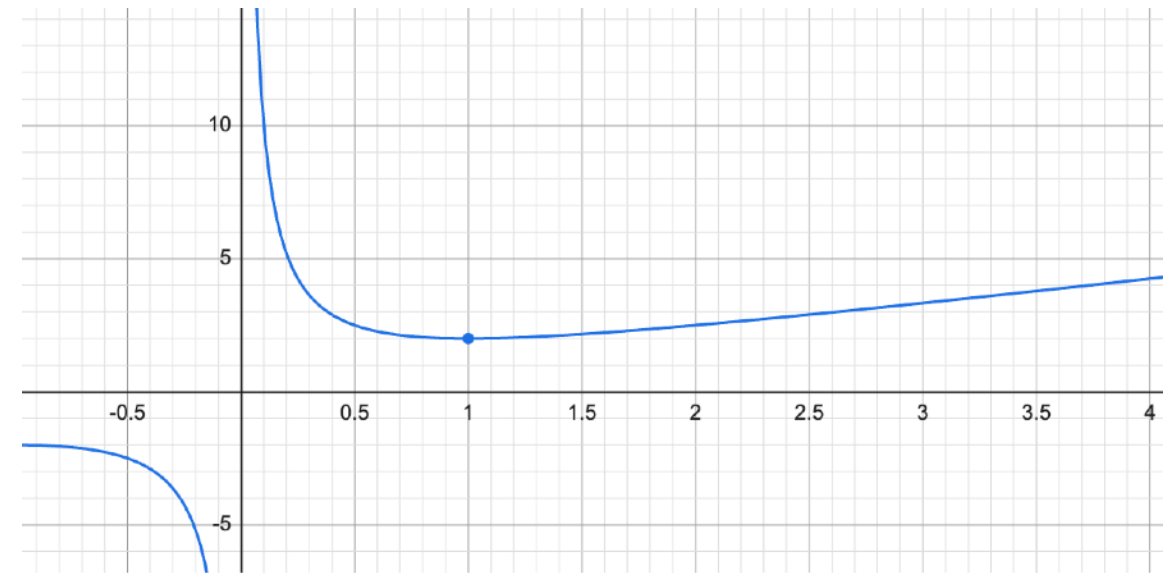
Total (computational) error

$$\frac{Mh}{2} + \frac{2\epsilon}{h}$$

Minimised for

$$h = 2\sqrt{\epsilon/M}$$

Total error



h

Example

$$f'(x) \approx \frac{f(x+h) - f(x)}{h}$$

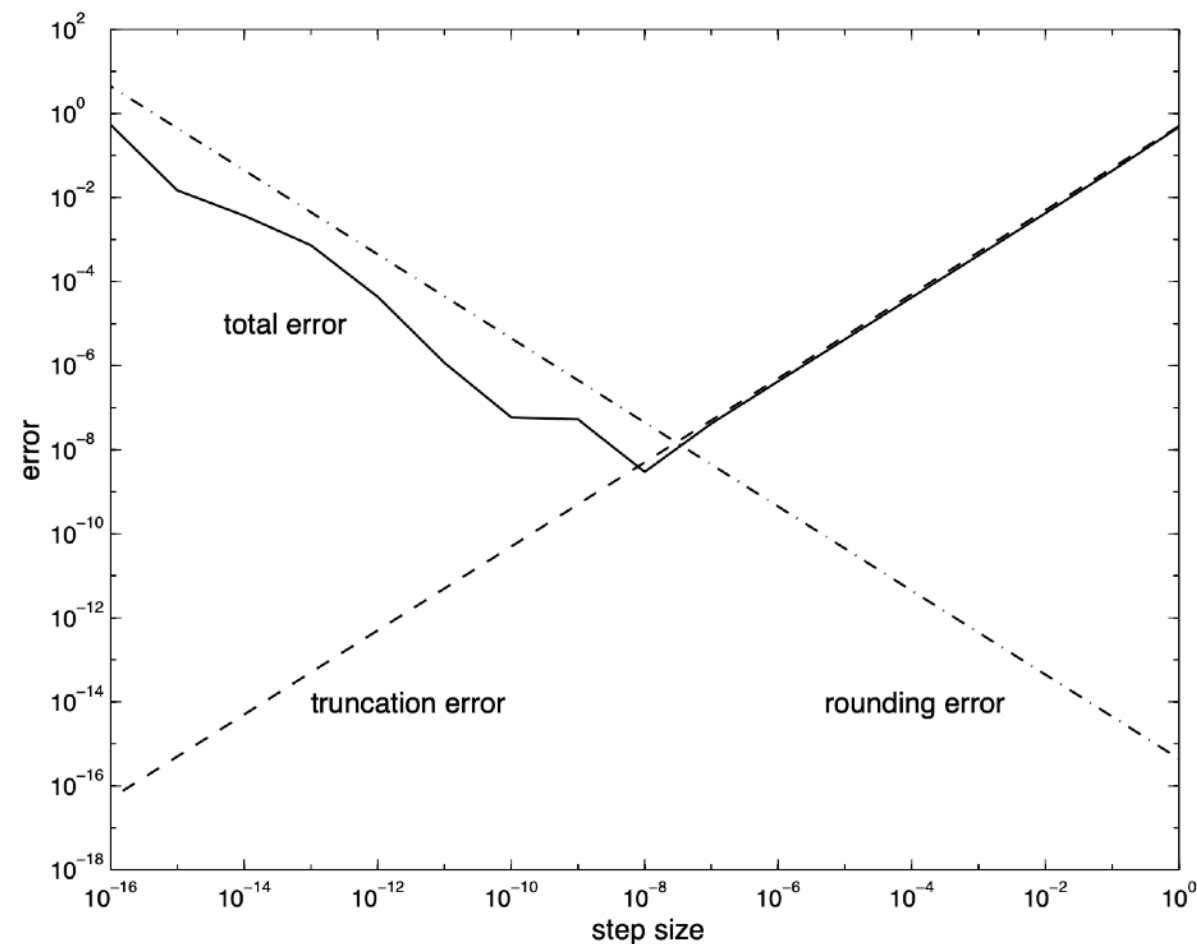
$$f(x+h) = f(x) + f'(x)h + f''(\theta)h^2/2$$

Total (computational) error

$$\frac{Mh}{2} + \frac{2\epsilon}{h}$$

Minimised for

$$h = 2\sqrt{\epsilon/M}$$



Definition of Errors - 5

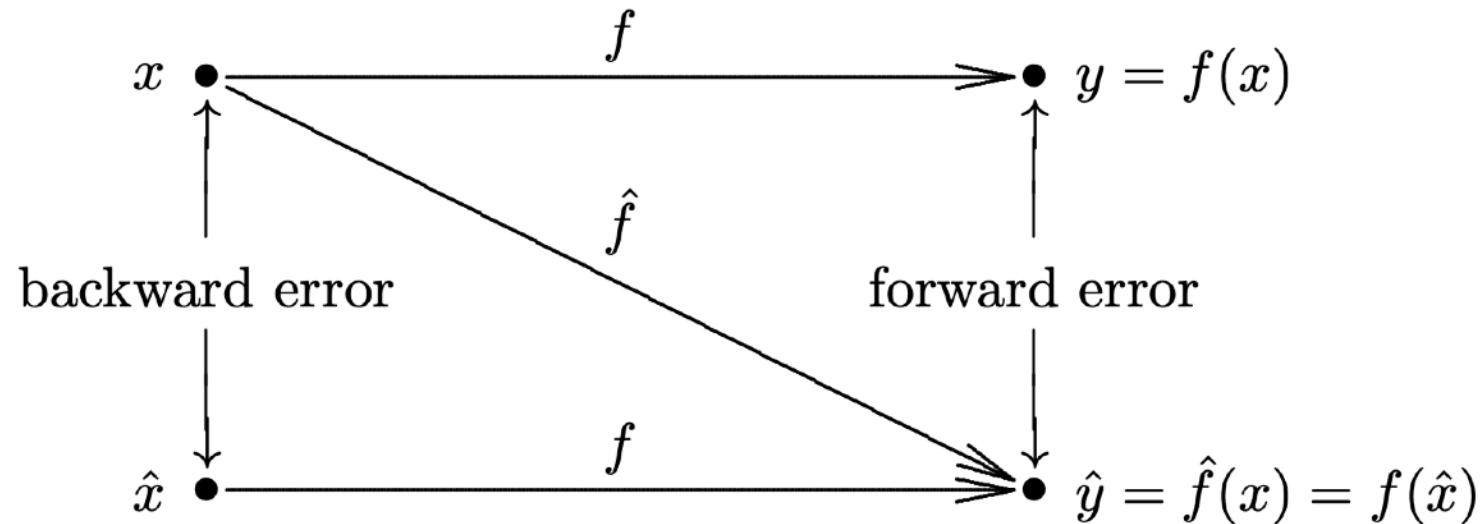
Forward and backward errors:

- Forward error: difference between computed value and true value.
- Backward error: which input data would in an exact calculation give the result of the approximation computation

Definition of Errors - 5

Forward and backward errors:

- Forward error: difference between computed value and true value.
- Backward error: which input data would in an exact calculation give the result of the approximation computation



Definition of Errors - 5

Forward and backward errors:

- Forward error: difference between computed value and true value.

$$\Delta y = \hat{f}(x) - f(x)$$

- Backward error: which input data would in an exact calculation give the result of the approximation computation

$$\Delta x = \hat{x} - x \quad : \quad \text{for which } \hat{x} \text{ is } \hat{f}(x) = f(\hat{x})?$$

Example:

$$f(x) = \sqrt{2}, \quad \hat{f}(x) = 1.4$$

Forward error: $|\Delta y| = |\hat{f}(x) - f(x)| = |1.4 - 1.4142| \approx \underline{\underline{0.0142}}$

Backward error: $(f(\hat{x}) = 1.4 = \sqrt{1.96} = f(\hat{x})), \quad |\Delta x| = |\hat{x} - x| = |1.96 - 2| = \underline{\underline{0.04}}$

Sensitivity and Conditioning - 1

A sensitiv or ill-conditioned mathematical problem:

If the relative change in the solution is possibly much larger than the relative change in the input data

Condition number:

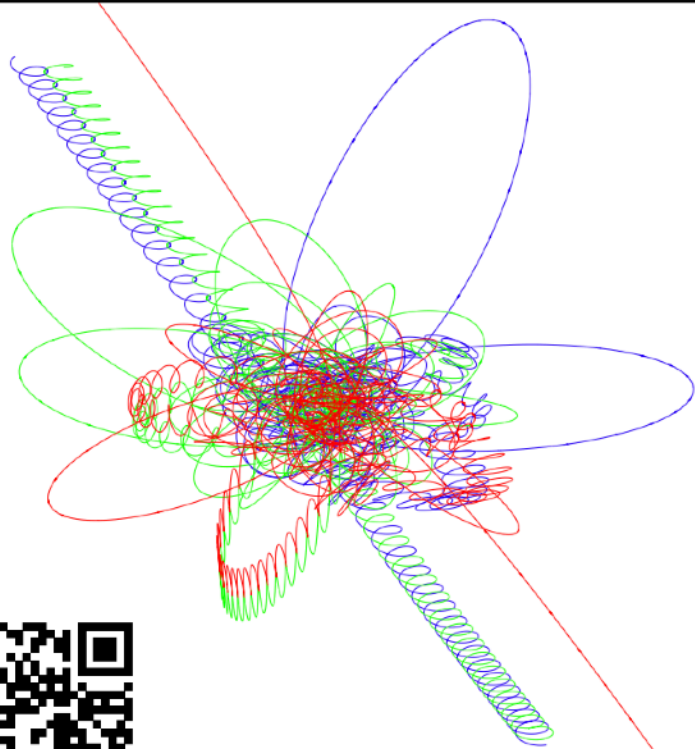
$$\frac{\left| \frac{\hat{f}(x) - f(x)}{f(x)} \right|}{\left| \frac{\hat{x} - x}{x} \right|} \approx \frac{\left| \frac{\Delta y}{y} \right|}{\left| \frac{\Delta x}{x} \right|}$$

The **Condition Number** of a problem is the ratio of the relative change in the solution to the relative change in the input.

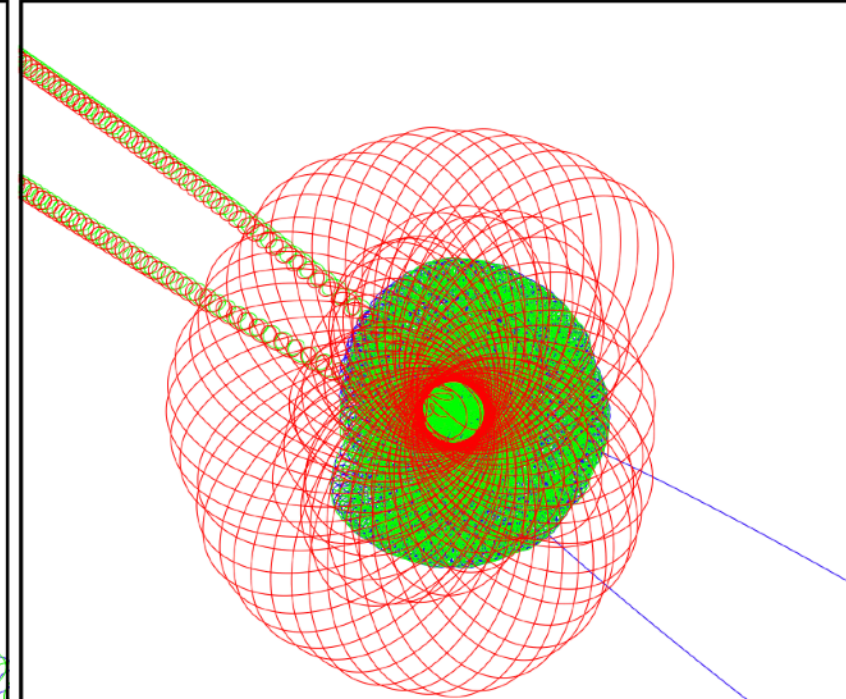
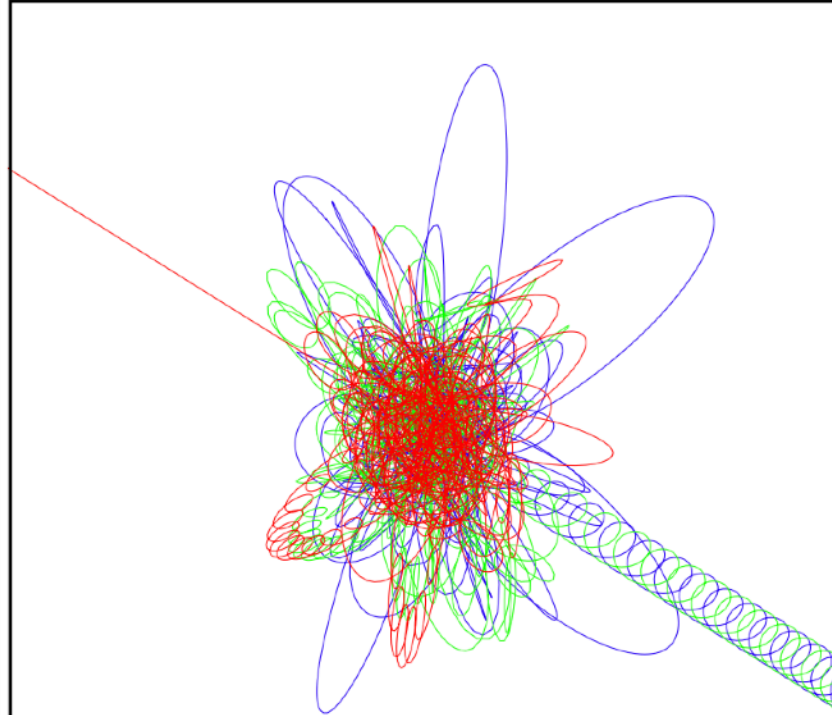
- | Relative forward error | = condition number x | Relative backward error |
- **Ill-conditioned** if condition number is **much larger than 1**.
- Concerns the propagated data error.
- Depends on the input data.
- *Only rough estimate or upper bound possible*
- *Condition number can be used to bound the forward error.*

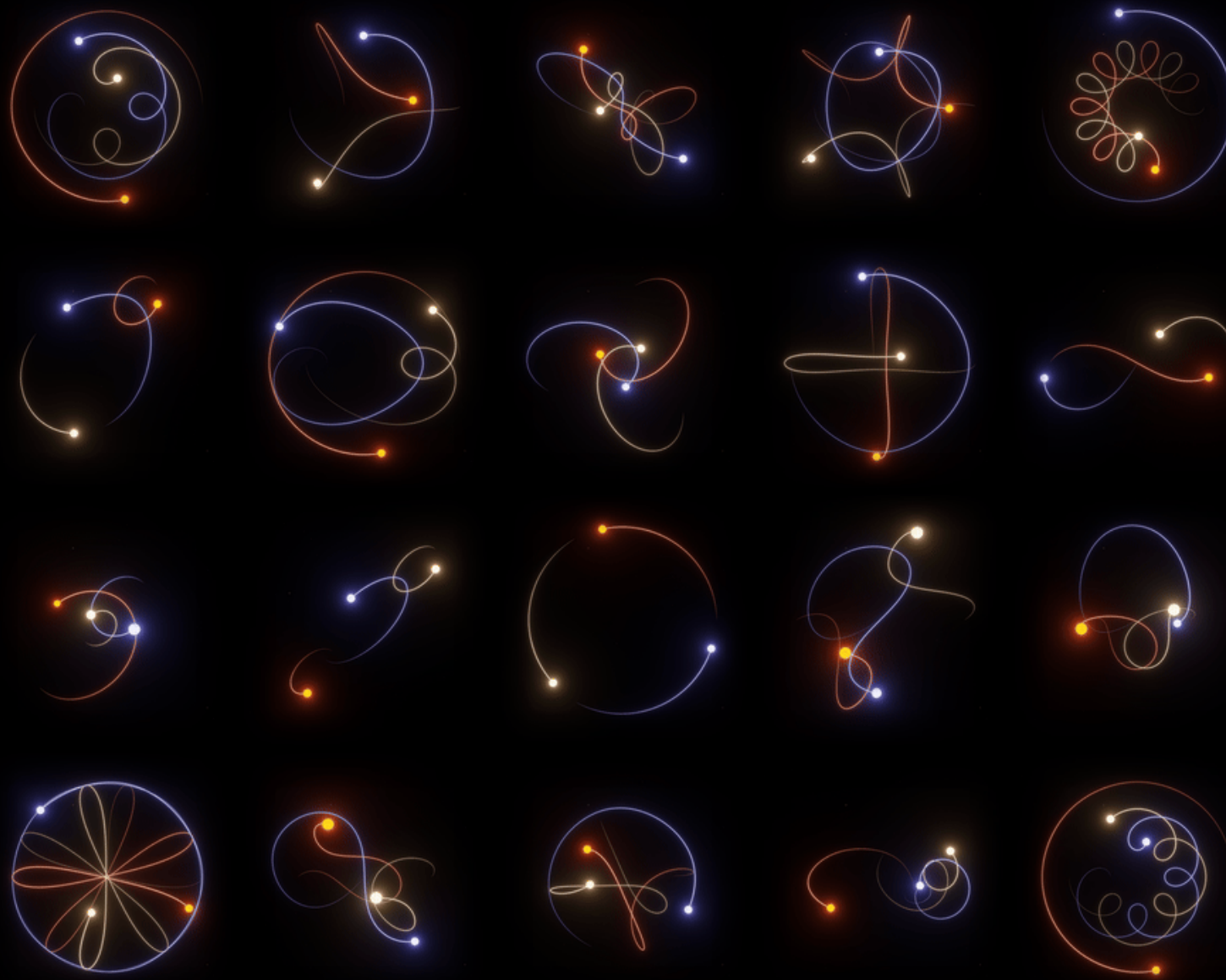


Chaos and Physical Constraints



Scan for animation





Sensitivity and Conditioning - 1

A sensitiv or ill-conditioned mathematical problem:

If the relative change in the solution is possibly much larger than the relative change in the input data

Condition number:

$$\frac{\left| \frac{\hat{f}(x) - f(x)}{f(x)} \right|}{\left| \frac{\hat{x} - x}{x} \right|} \approx \frac{\left| \frac{\Delta y}{y} \right|}{\left| \frac{\Delta x}{x} \right|}$$

The **Condition Number** of a problem is the ratio of the relative change in the solution to the relative change in the input.

- | Relative forward error | = condition number x | Relative backward error |
- **Ill-conditioned** if condition number is **much larger than 1**.
- Concerns the propagated data error.
- Depends on the input data.
- *Only rough estimate or upper bound possible*
- *Condition number can be used to bound the forward error.*



Sensitivity and Conditioning - 2

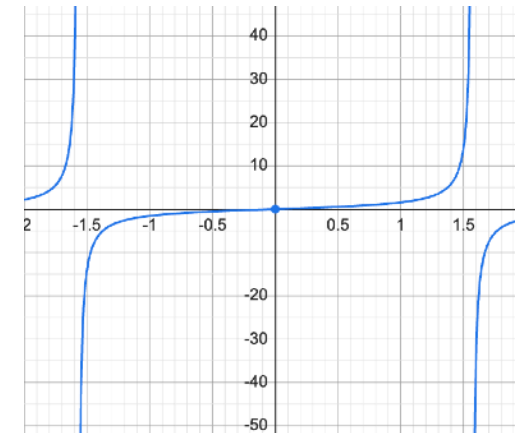
Condition number for a differentiable function:

- Absolute forward error
- Relative forward error
- Condition number

$$= f(x + \Delta x) - f(x) \approx f'(x) \Delta x$$

$$= \frac{f(x + \Delta x) - f(x)}{\Delta x} \approx \frac{f'(x) \Delta x}{\Delta x} = f'(x)$$

$$= \left| \frac{x f'(x)}{f(x)} \right|$$



Example: $f(x) = \tanh(x)$

Condition number $\left| \frac{x f'(x)}{f(x)} \right| = \left| \frac{x (1 + \tanh^2(x))}{\tanh(x)} \right| = \left| x \left(\frac{1}{\tanh(x)} + \tanh(x) \right) \right|$

for $x = 1.57079$, $CN \approx 2.48 \cdot 10^5$

Stability and Accuracy of an Algorithm

Stability of an algorithm:

- Refers to the effects of computational error.
- An algorithm is stable, if the result it produces is the exact solution to a nearby problem (it has a small backward error)

Accuracy of a solution:

- Refers to the closeness of the computed to the true solution (a small forward error)
- Stability is necessary, but not sufficient, for accuracy (the solution to a nearby problem can be far away from the the true solution for an ill-conditioned problem.)

Computer Arithmetic

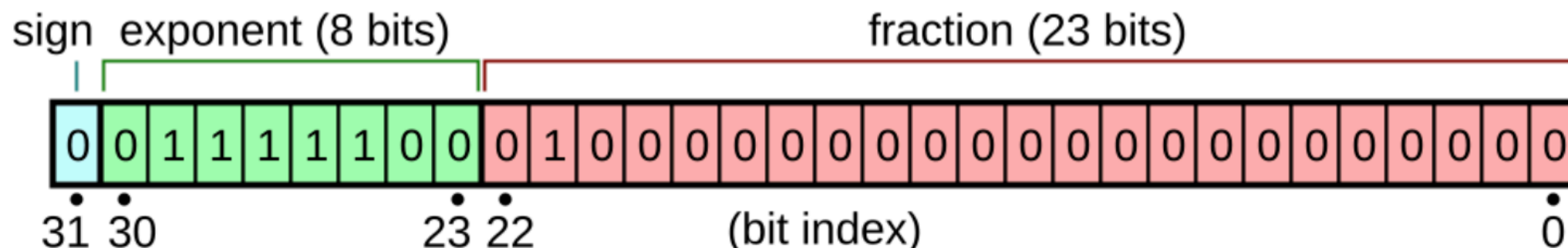
$$x = \pm \left(d_0 + \frac{d_1}{\beta} + \frac{d_2}{\beta^2} + \cdots + \frac{d_{p-1}}{\beta^{p-1}} \right) \beta^E$$

β Base or radix

p Precision

$[L, U]$ Exponent range

$$0 \leq d_i \leq \beta - 1, \quad i = 0, \dots, p - 1, \quad L \leq E \leq U$$



Computer Arithmetic

System	β	p	L	U
IEEE SP	2	24	-126	127
IEEE DP	2	53	-1,022	1,023
Cray	2	48	-16,383	16,384
HP calculator	10	12	-499	499
IBM mainframe	16	6	-64	63

Table 1.1: Parameters for typical floating-point systems

Computer Arithmetic

A floating-point number system is finite and discrete. The number of normalized floating-point numbers in a given floating-point system is

$$2(\beta - 1)\beta^{p-1}(U - L + 1) + 1$$

$$\epsilon_{\text{mach}} = \beta^{1-p}$$

Computer Arithmetic

Example 1.9 Floating-Point System. An example floating-point system is illustrated in Fig. 1.3, where the tick marks indicate all of the 25 floating-point numbers in a system having $\beta = 2$, $p = 3$, $L = -1$, and $U = 1$. For this system, the largest number is $\text{OFL} = (1.11)_2 \times 2^1 = (3.5)_{10}$, and the smallest positive normalized number is $\text{UFL} = (1.00)_2 \times 2^{-1} = (0.5)_{10}$. This is a very tiny, toy system for illustrative purposes only, but it is in fact characteristic of floating-point systems in general: at a sufficiently high level of magnification, every normalized floating-point system looks essentially like this one—grainy and unequally spaced.

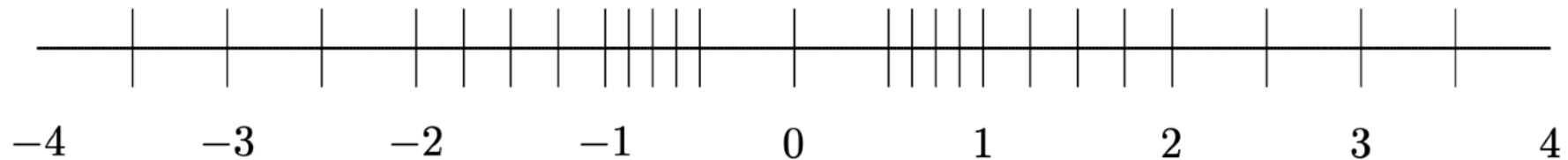


Figure 1.3: Example floating-point number system.

Computer Arithmetic

Example 1.11 Floating-Point Arithmetic. Consider a floating-point system with $\beta = 10$ and $p = 6$. If $x = 1.92403 \times 10^2$ and $y = 6.35782 \times 10^{-1}$, then floating-point addition gives the result $x + y = 1.93039 \times 10^2$, assuming rounding to nearest. Note that the last two digits of y have no effect on the result. With an even smaller exponent, y could have had no effect at all on the result. Similarly, floating-point multiplication gives the result $x * y = 1.22326 \times 10^2$, which discards half of the digits of the true product.

Questions and Discussions

1.1. True or false: A problem is ill-conditioned if its solution is highly sensitive to small changes in the problem data.

Questions and Discussions

1.1. True or false: A problem is ill-conditioned if its solution is highly sensitive to small changes in the problem data.

1.2. True or false: Using higher-precision arithmetic will make an ill-conditioned problem better conditioned.

Questions and Discussions

1.1. True or false: A problem is ill-conditioned if its solution is highly sensitive to small changes in the problem data.

1.2. True or false: Using higher-precision arithmetic will make an ill-conditioned problem better conditioned.

1.3. True or false: The conditioning of a problem depends on the algorithm used to solve it.

Questions and Discussions

1.1. True or false: A problem is ill-conditioned if its solution is highly sensitive to small changes in the problem data.

1.2. True or false: Using higher-precision arithmetic will make an ill-conditioned problem better conditioned.

1.3. True or false: The conditioning of a problem depends on the algorithm used to solve it.

1.4. True or false: A good algorithm will produce an accurate solution regardless of the condition of the problem being solved.

Questions and Discussions

1.1. True or false: A problem is ill-conditioned if its solution is highly sensitive to small changes in the problem data.

1.2. True or false: Using higher-precision arithmetic will make an ill-conditioned problem better conditioned.

1.3. True or false: The conditioning of a problem depends on the algorithm used to solve it.

1.4. True or false: A good algorithm will produce an accurate solution regardless of the condition of the problem being solved.

1.5. True or false: The choice of algorithm for solving a problem has no effect on the propagated data error.

Questions and Discussions

1.1. True or false: A problem is ill-conditioned if its solution is highly sensitive to small changes in the problem data.

1.2. True or false: Using higher-precision arithmetic will make an ill-conditioned problem better conditioned.

1.3. True or false: The conditioning of a problem depends on the algorithm used to solve it.

1.4. True or false: A good algorithm will produce an accurate solution regardless of the condition of the problem being solved.

1.5. True or false: The choice of algorithm for solving a problem has no effect on the propagated data error.

1.6. True or false: A stable algorithm applied to a well-conditioned problem necessarily produces an accurate solution.

Questions and Discussions

1.1. True or false: A problem is ill-conditioned if its solution is highly sensitive to small changes in the problem data.

1.2. True or false: Using higher-precision arithmetic will make an ill-conditioned problem better conditioned.

1.3. True or false: The conditioning of a problem depends on the algorithm used to solve it.

1.4. True or false: A good algorithm will produce an accurate solution regardless of the condition of the problem being solved.

1.5. True or false: The choice of algorithm for solving a problem has no effect on the propagated data error.

1.6. True or false: A stable algorithm applied to a well-conditioned problem necessarily produces an accurate solution.

1.7. True or false: If two real numbers are exactly representable as floating-point numbers, then the result of a real arithmetic operation on them will also be representable as a floating-point number.

Questions and Discussions

1.1. True or false: A problem is ill-conditioned if its solution is highly sensitive to small changes in the problem data.

1.2. True or false: Using higher-precision arithmetic will make an ill-conditioned problem better conditioned.

1.3. True or false: The conditioning of a problem depends on the algorithm used to solve it.

1.4. True or false: A good algorithm will produce an accurate solution regardless of the condition of the problem being solved.

1.5. True or false: The choice of algorithm for solving a problem has no effect on the propagated data error.

1.6. True or false: A stable algorithm applied to a well-conditioned problem necessarily produces an accurate solution.

1.7. True or false: If two real numbers are exactly representable as floating-point numbers, then the result of a real arithmetic operation on them will also be representable as a floating-point number.

1.8. True or false: Floating-point numbers are distributed uniformly throughout their range.

Questions and Discussions

1.1. True or false: A problem is ill-conditioned if its solution is highly sensitive to small changes in the problem data.

1.2. True or false: Using higher-precision arithmetic will make an ill-conditioned problem better conditioned.

1.3. True or false: The conditioning of a problem depends on the algorithm used to solve it.

1.4. True or false: A good algorithm will produce an accurate solution regardless of the condition of the problem being solved.

1.5. True or false: The choice of algorithm for solving a problem has no effect on the propagated data error.

1.6. True or false: A stable algorithm applied to a well-conditioned problem necessarily produces an accurate solution.

1.7. True or false: If two real numbers are exactly representable as floating-point numbers, then the result of a real arithmetic operation on them will also be representable as a floating-point number.

1.8. True or false: Floating-point numbers are distributed uniformly throughout their range.

1.9. True or false: Floating-point addition is associative but not commutative.

Scientific Computing



Johan Samsing
Faculty of Science
University of Copenhagen

Lecture W1 - L2

Linear Systems

We know of many systems in Nature that are of linear character. But what does linear means?

...meaning that effects are proportional to their causes.

We know of many systems that can be described by a linear approximation.

Lets take a look at a few examples!

Linear Systems

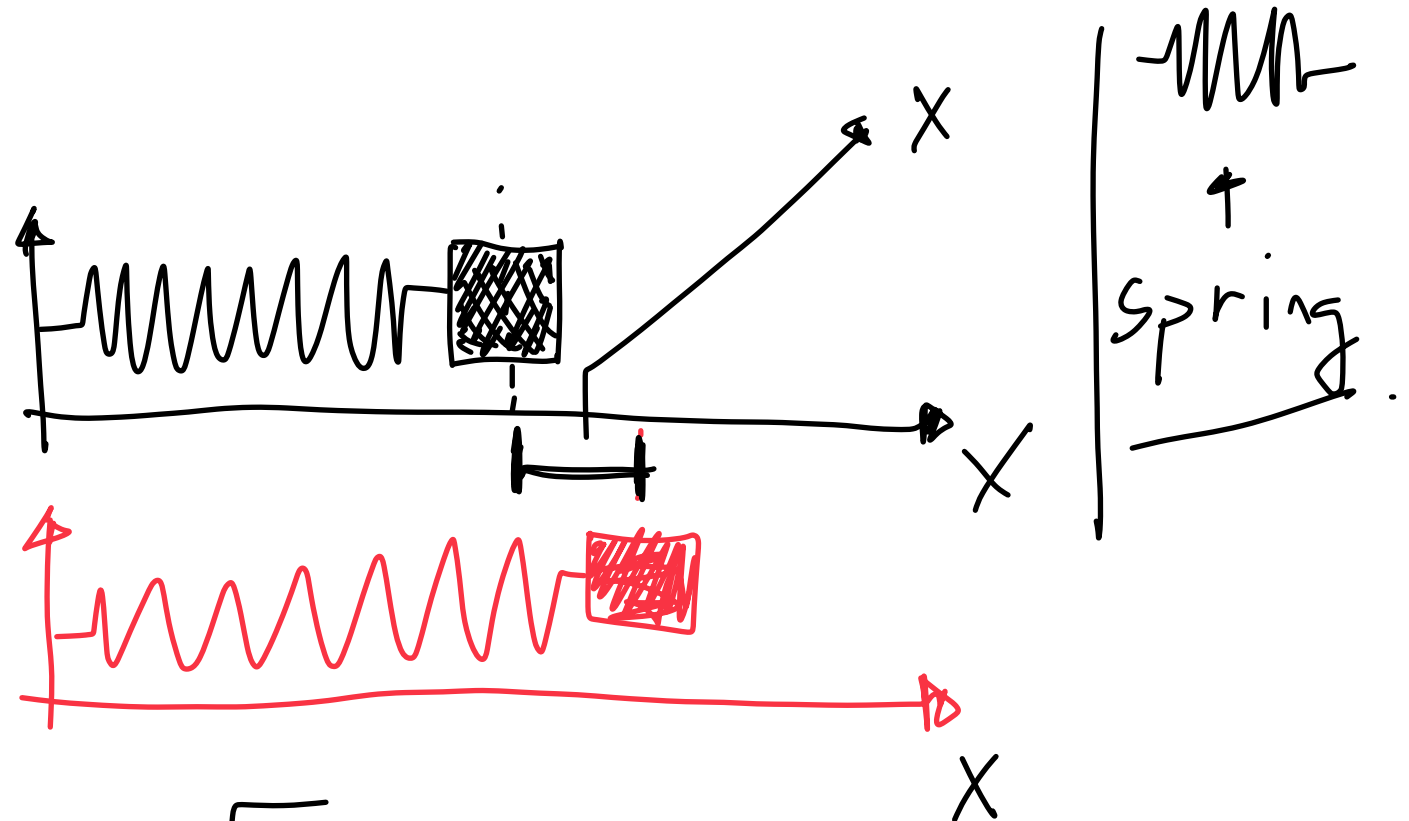
Example: **Harmonic Oscillator**

$$F = -k \cdot x$$

Solution:

$$\left[F = m \cdot a = m \frac{d^2 x}{dt^2} = -k \cdot x \right]$$

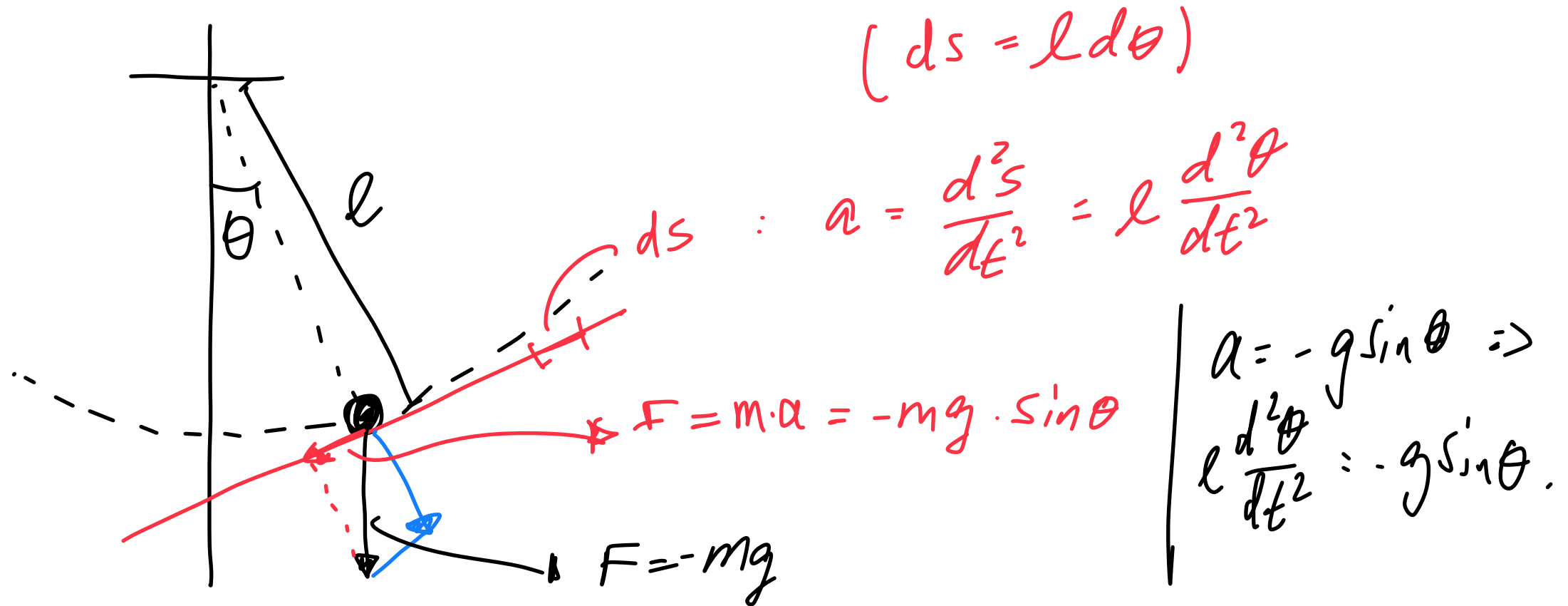
$$x(t) = A \cos(\omega t + \varphi), \quad \omega = \sqrt{\frac{k}{m}}$$



Also see the black board!

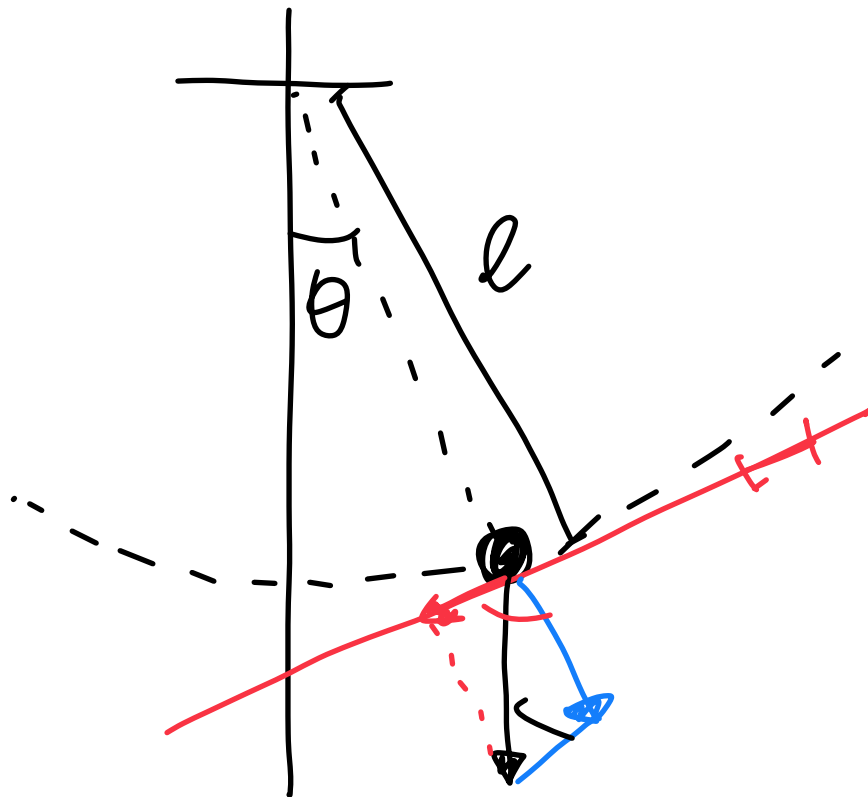
Linear Systems

From non-linear to linear systems: **The pendulum**



Linear Systems

From non-linear to linear systems: **The pendulum**



$$l \frac{d^2\theta}{dt^2} = -g \sin \theta : \text{non-linear!}$$

$$\sin \theta \approx \theta :$$

$$l \frac{d^2\theta}{dt^2} = -g\theta \Rightarrow \boxed{\frac{d^2\theta}{dt^2} \approx -\frac{g}{l} \theta}$$

$$\boxed{\theta(t) \approx A \cos(\sqrt{\frac{g}{l}} t + \varphi)}$$



Linear Systems

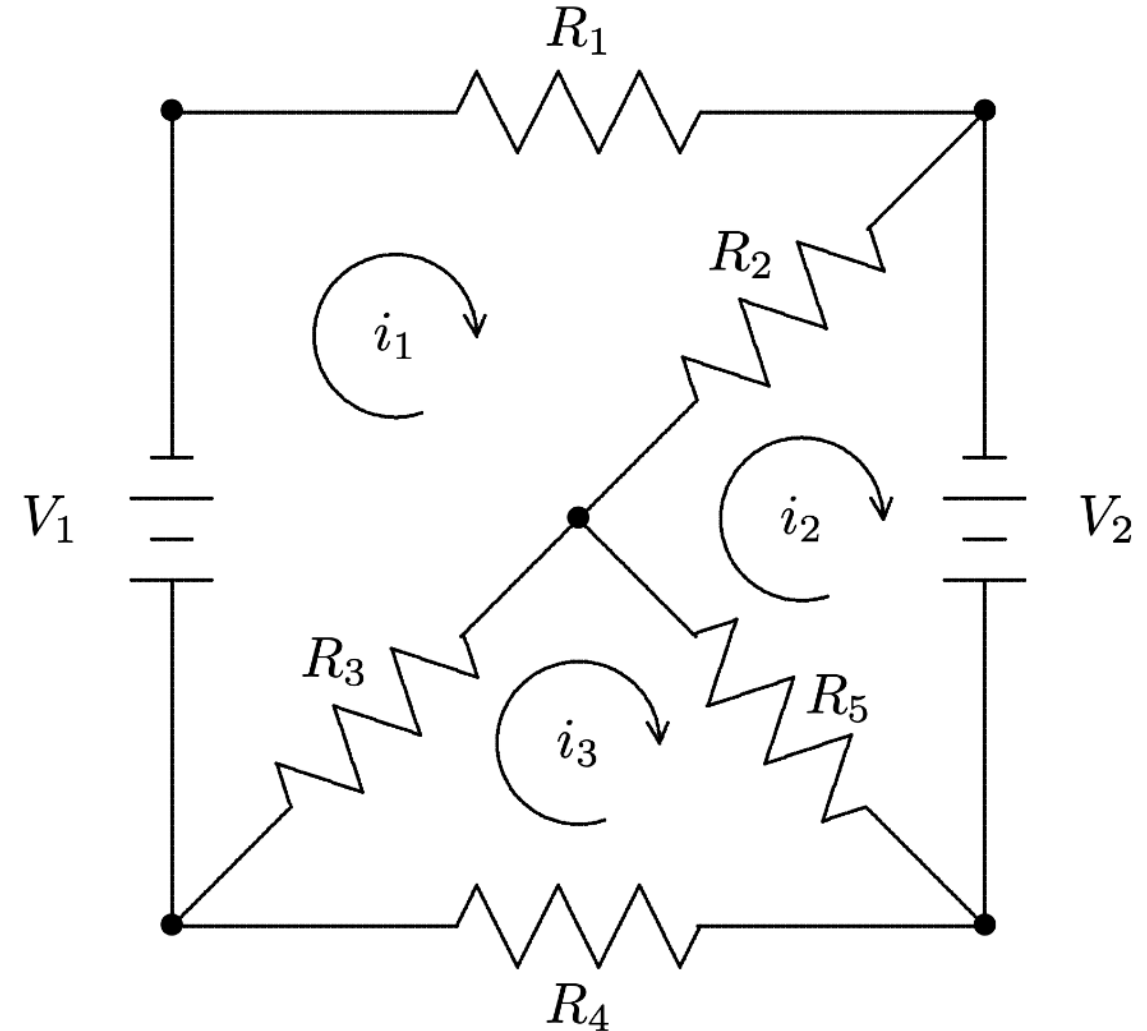
Example: **Electric Circuit**

In short, $V = iR$

Set system of equations up from physical laws:
(Ohm and Kirchhoff's laws...)

Ohm's Law: The voltage drop across a resistance R in the direction of a current i is given by iR .

Kirchhoff's Law: The net voltage drop in a closed loop is zero.



(V = voltage, R = resistance, $i = (V/R/t)$)

Linear Systems

Example: **Electric Circuit**

In short, $V = iR$

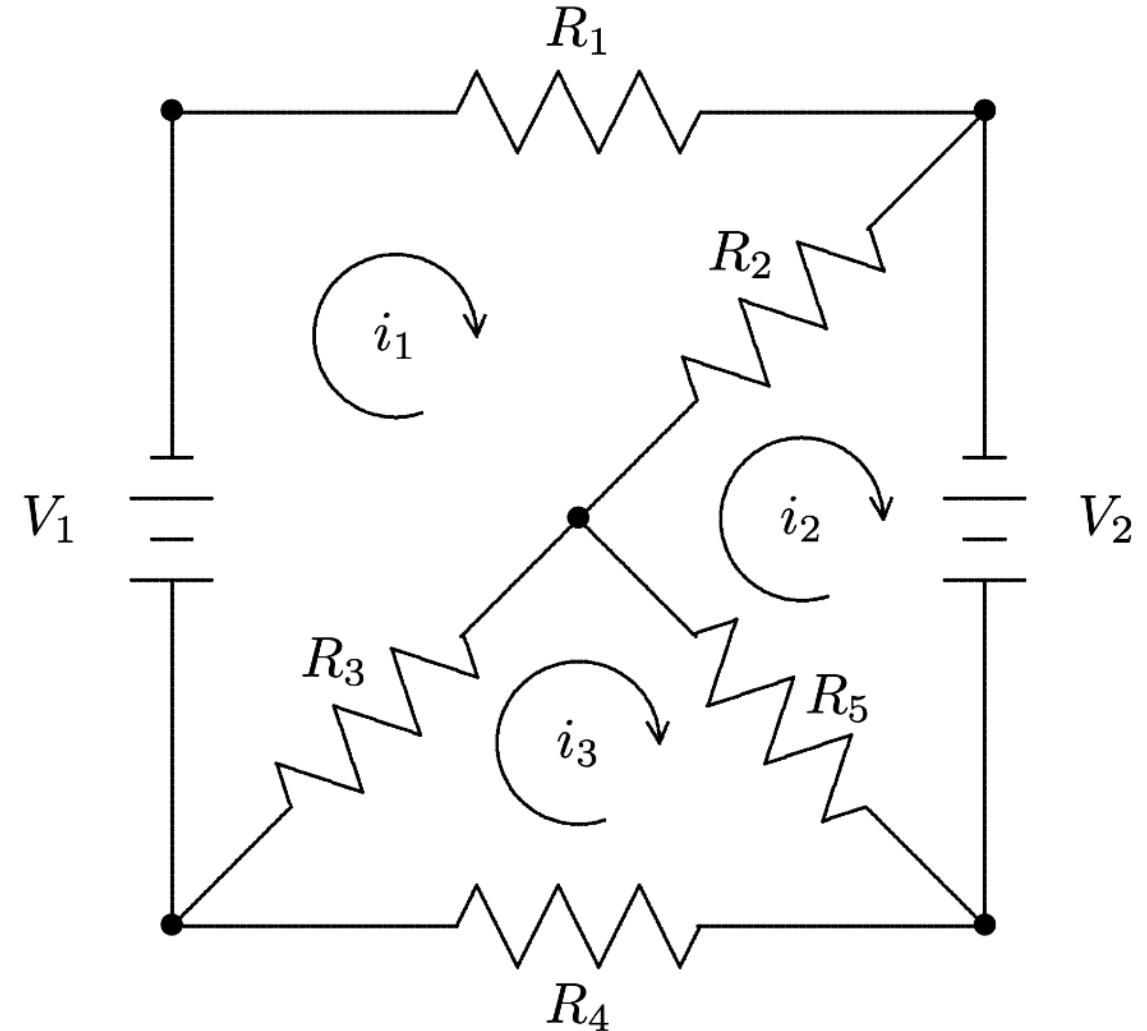
Set system of equations up from physical laws:
(Ohm and Kirchhoff's laws...)

Linear equations:

$$i_1 R_1 + (i_1 - i_2) R_2 + (i_1 - i_3) R_3 + V_1 = 0$$

$$(i_2 - i_1) R_2 + (i_2 - i_3) R_5 - V_2 = 0$$

$$(i_3 - i_1) R_3 + i_3 R_4 + (i_3 - i_2) R_5 = 0$$



(V = voltage, R = resistance, $i = (V/R/t)$)

Linear Systems

- *In matrix form*

$$\mathbf{Ax} = \mathbf{b}$$

Matrix Theory

- Vectors and Matrices: Examples and definitions

$m \times n$:

$$\begin{matrix}
 & 1 & \dots & n \\
 \begin{matrix} 1 \\ \vdots \\ m \end{matrix} & \begin{bmatrix} a_{11} & a_{12} & \dots \\ a_{21} & a_{22} & \dots \\ \vdots & \vdots & \ddots \\ a_{m1} & a_{m2} & \dots \end{bmatrix}
 \end{matrix}$$

(clarify on Blackboard)

Row Vector:
 $[1 \ 4 \ 3]$

Column Vector:
 $\begin{bmatrix} 1 \\ 4 \\ 3 \end{bmatrix}$

Matrix Theory

- Vectors and Matrices: Examples and definitions
- Basic Matrix operations: Addition, scalar multiplications, Transpositions, etc.

$$(\bar{A} + \bar{B})_{i,j} = \bar{A}_{i,j} + \bar{B}_{i,j} \quad (\text{add each entry})$$

$$(c \cdot \bar{A})_{i,j} = c \cdot \bar{A}_{i,j} \quad (\text{scalar factors out})$$

$$(\bar{A}^T)_{i,j} = \bar{A}_{j,i} \quad (\text{rows} \leftrightarrow \text{columns})$$

Matrix Theory

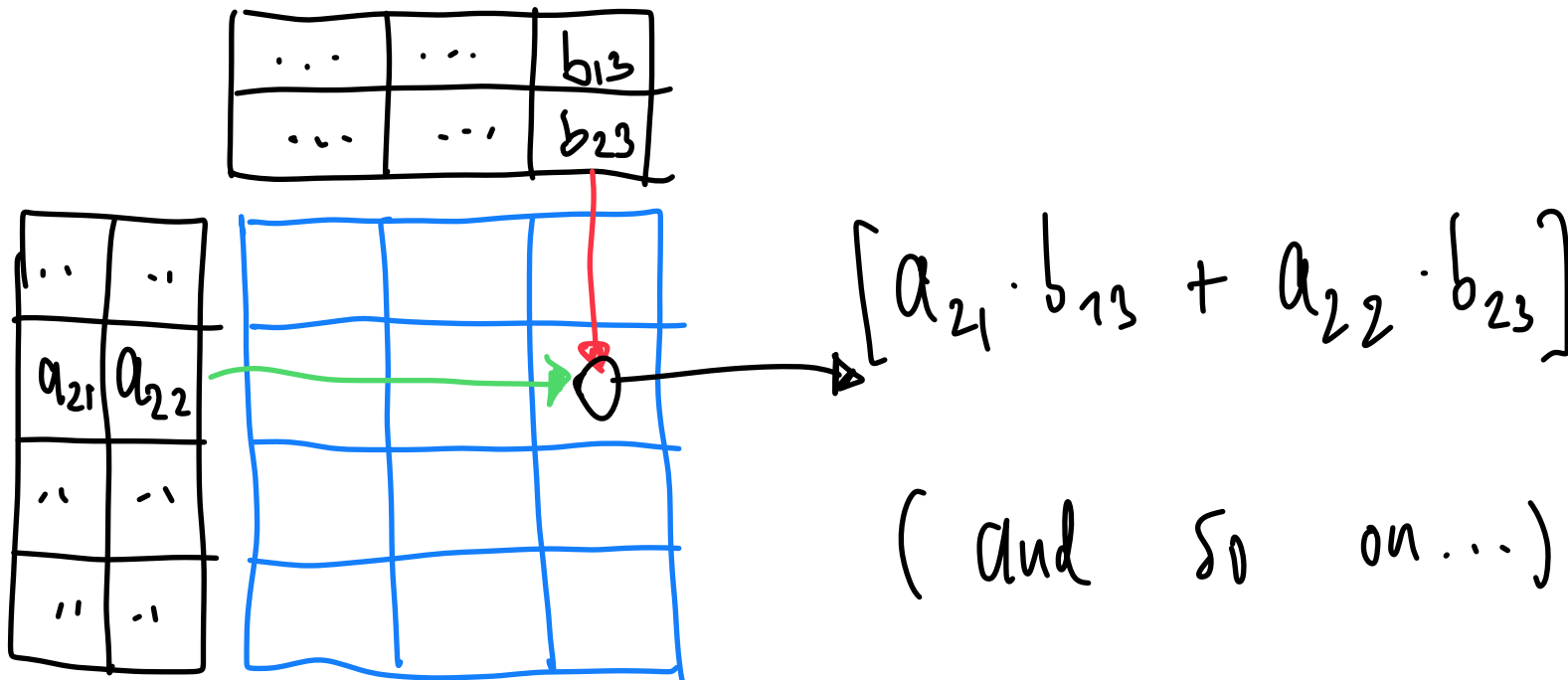
- Vectors and Matrices: Examples and definitions
- Basic Matrix operations: Addition, scalar multiplications, Transpositions, etc.

$$\bar{A} + \bar{B} = \bar{B} + \bar{A} , \quad (c \cdot \bar{A})^T = c (\bar{A}^T) ,$$

$$(\bar{A} + \bar{B})^T = \bar{A}^T + \bar{B}^T \quad (\bar{A}^T)^T = \bar{A}$$

Matrix Theory

- Vectors and Matrices: Examples and definitions
- Basic Matrix operations: Addition, scalar multiplications, Transpositions, etc.

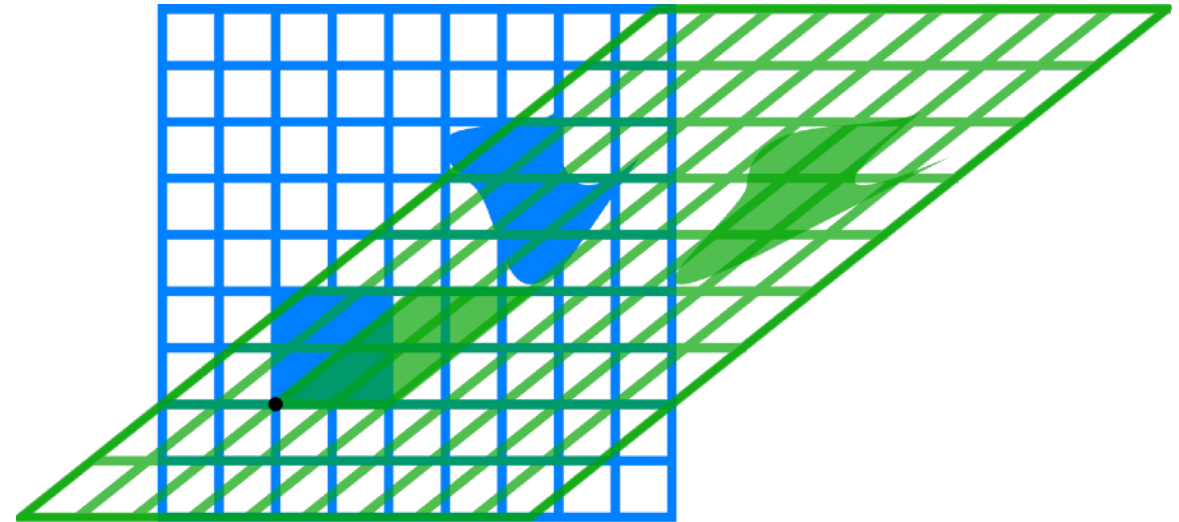


Matrix Theory

- Vectors and Matrices: Examples and definitions
 - Basic Matrix operations: Addition, scalar multiplications, Transpositions, etc.
 - Matrix Multiplications: Rules and results.
 - Examples of Matrix transformations: Scalings, Rotations, etc.
-

Horizontal Shear:

$$\begin{bmatrix} 1 & 1.25 \\ 0 & 1 \end{bmatrix}$$

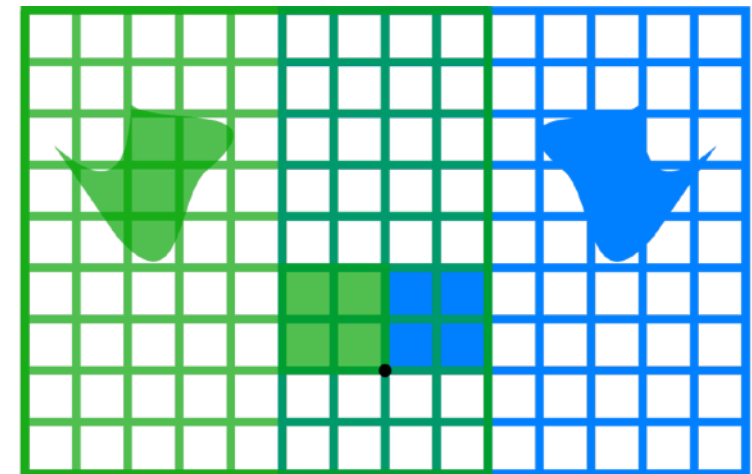


Matrix Theory

- Vectors and Matrices: Examples and definitions
 - Basic Matrix operations: Addition, scalar multiplications, Transpositions, etc.
 - Matrix Multiplications: Rules and results.
 - Examples of Matrix transformations: Scalings, Rotations, etc.
-

Reflection through vertical axis:

$$\begin{bmatrix} -1 & 0 \\ 0 & 1 \end{bmatrix}$$

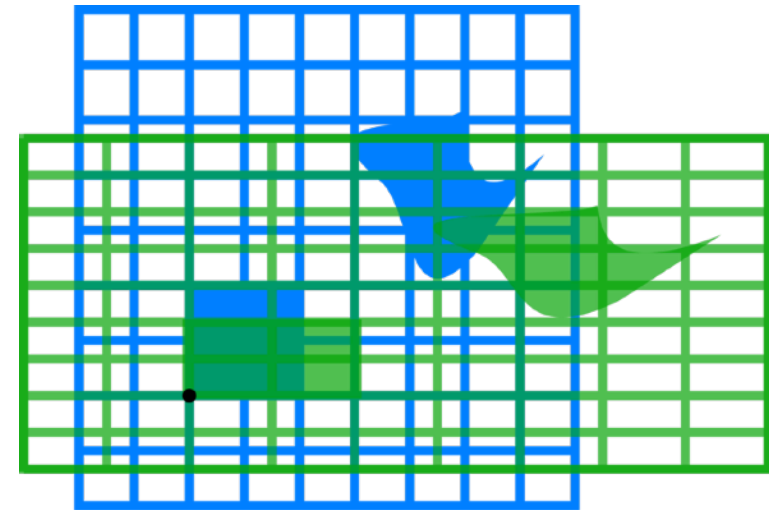


Matrix Theory

- Vectors and Matrices: Examples and definitions
- Basic Matrix operations: Addition, scalar multiplications, Transpositions, etc.
- Matrix Multiplications: Rules and results.
- Examples of Matrix transformations: Scalings, Rotations, etc.

Squeeze mapping:

$$\begin{bmatrix} \frac{3}{2} & 0 \\ 0 & \frac{2}{3} \end{bmatrix}$$

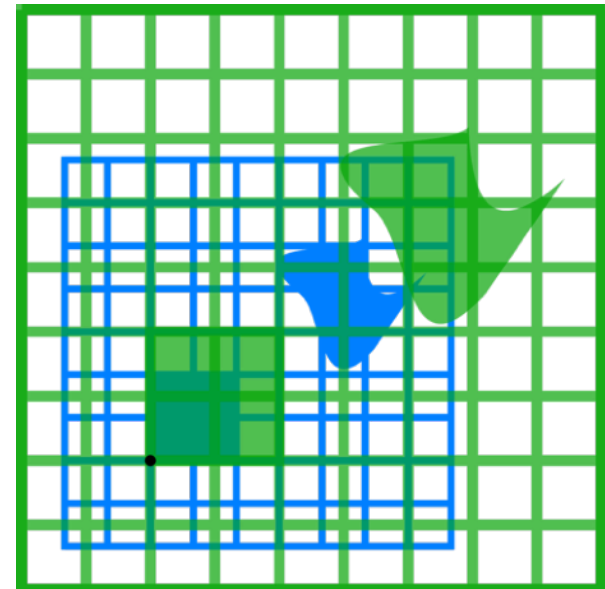


Matrix Theory

- Vectors and Matrices: Examples and definitions
 - Basic Matrix operations: Addition, scalar multiplications, Transpositions, etc.
 - Matrix Multiplications: Rules and results.
 - Examples of Matrix transformations: Scalings, Rotations, etc.
-

Scaling:

$$\begin{bmatrix} \frac{3}{2} & 0 \\ 0 & \frac{3}{2} \end{bmatrix}$$

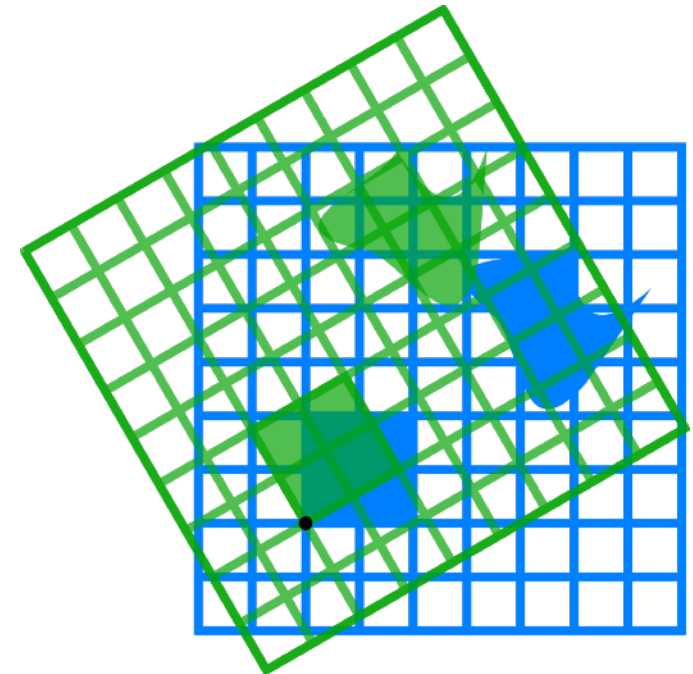


Matrix Theory

- Vectors and Matrices: Examples and definitions
 - Basic Matrix operations: Addition, scalar multiplications, Transpositions, etc.
 - Matrix Multiplications: Rules and results.
 - Examples of Matrix transformations: Scalings, Rotations, etc.
-

Rotations:

$$\begin{bmatrix} \cos\left(\frac{\pi}{6}\right) & -\sin\left(\frac{\pi}{6}\right) \\ \sin\left(\frac{\pi}{6}\right) & \cos\left(\frac{\pi}{6}\right) \end{bmatrix}$$



Linear Systems

- In matrix form

$$\begin{aligned} 2x_1 + 3x_2 &= b_1 \\ 5x_1 + 4x_2 &= b_2 \end{aligned}$$

$\Rightarrow$

$$\bar{A} \bar{x} = \bar{b}$$

$$\bar{A} \bar{x} = \begin{bmatrix} 2 & 3 \\ 5 & 4 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \end{bmatrix} = \bar{b}$$

$$\mathbf{Ax} = \mathbf{b}$$

Go through example on black board

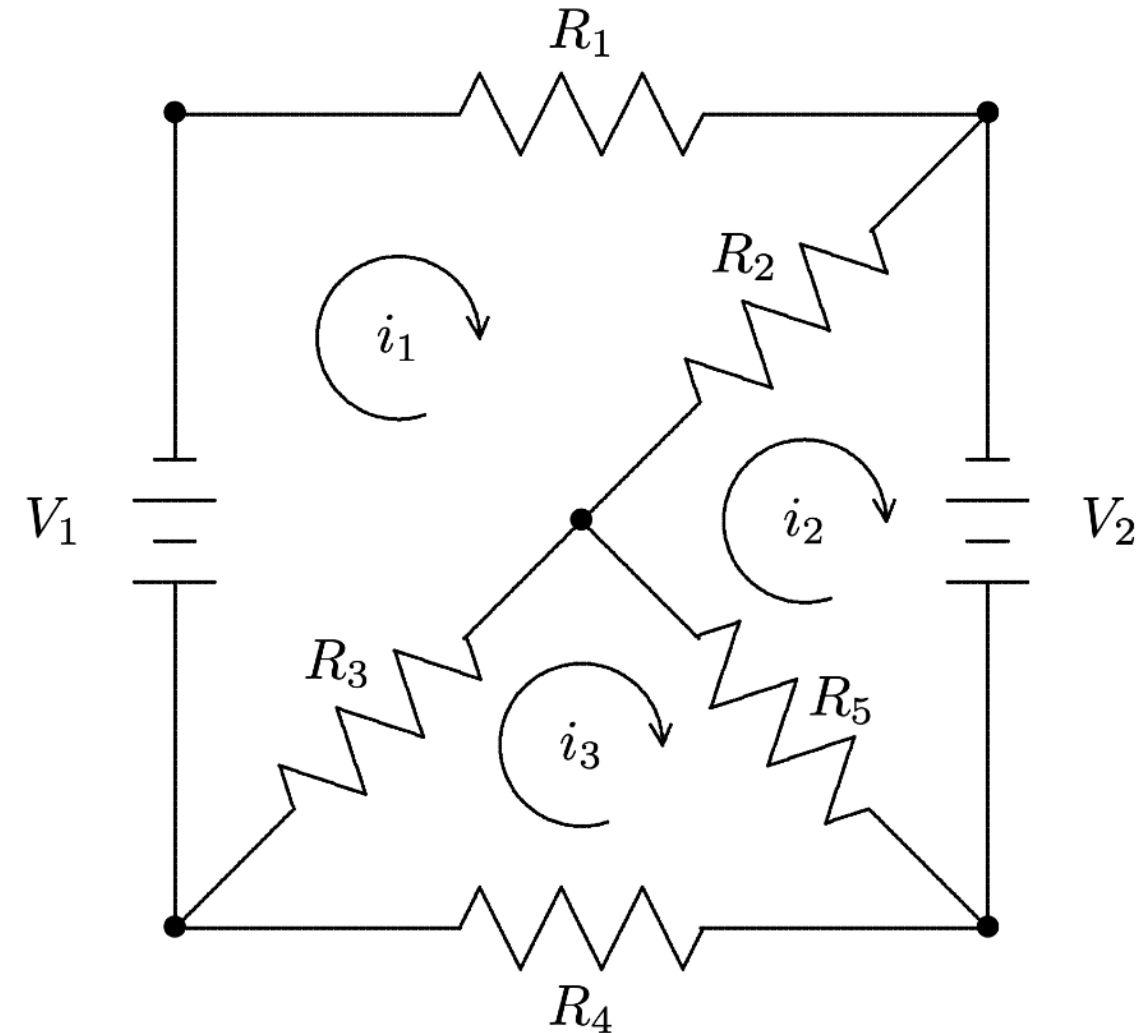
Linear Systems

In equations form:

$$i_1 R_1 + (i_1 - i_2) R_2 + (i_1 - i_3) R_3 + V_1 = 0$$

$$(i_2 - i_1) R_2 + (i_2 - i_3) R_5 - V_2 = 0$$

$$(i_3 - i_1) R_3 + i_3 R_4 + (i_3 - i_2) R_5 = 0$$

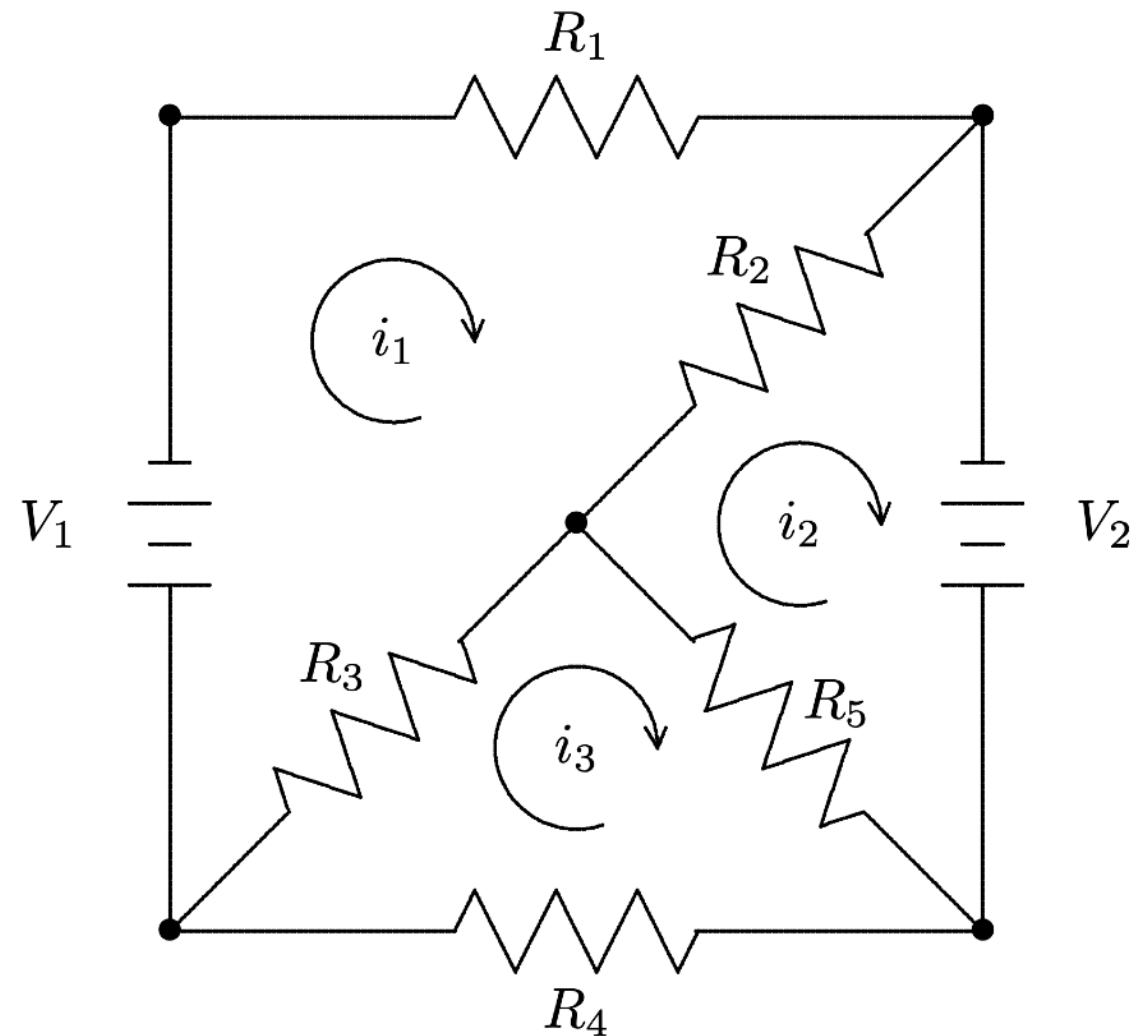


Linear Systems

In matrix form:

$$\begin{bmatrix} R_1 + R_2 + R_3 & -R_2 & -R_3 \\ -R_2 & R_2 + R_5 & -R_5 \\ -R_3 & -R_5 & R_3 + R_4 + R_5 \end{bmatrix} \begin{bmatrix} i_1 \\ i_2 \\ i_3 \end{bmatrix} = \begin{bmatrix} -V_1 \\ V_2 \\ 0 \end{bmatrix}$$

We will develop techniques to solve this types of equations



Existence and Uniqueness

Consider again $\mathbf{Ax} = \mathbf{b}$ system. Regarding a possible solution to this relation depends partly on the following properties of $\mathbf{A}$:

$$\bar{\mathbf{A}} \bar{\mathbf{x}} = \bar{\mathbf{b}} \quad : \quad \bar{\mathbf{A}}^{-1} \bar{\mathbf{A}} \bar{\mathbf{x}} = \bar{\mathbf{A}}^{-1} \bar{\mathbf{b}} \quad \Rightarrow$$

$$\boxed{\mathbf{x} = \mathbf{A}^{-1} \mathbf{b}}$$

Does $\mathbf{A}$ inverse exist?

Existence and Uniqueness

Consider again $\mathbf{Ax} = \mathbf{b}$ system. Regarding a possible solution to this relation depends partly on the following properties of $\mathbf{A}$:

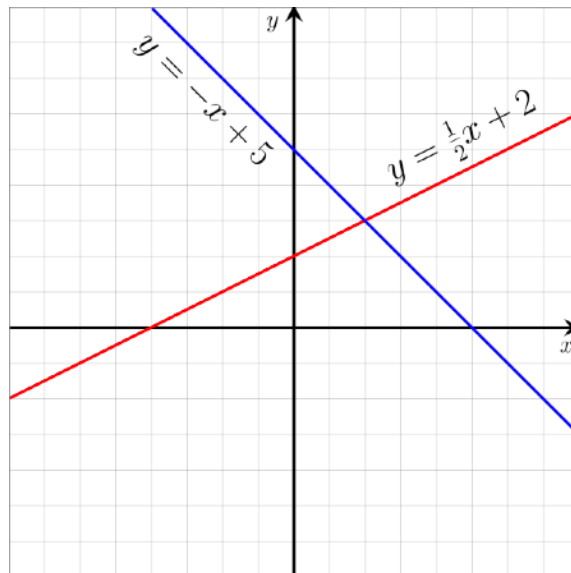
An $n \times n$ matrix $\mathbf{A}$ is said to be nonsingular if it satisfies any one of the following equivalent conditions:

1. $\mathbf{A}$ has an inverse (i.e., there is an $n \times n$ matrix, denoted by $\mathbf{A}^{-1}$, such that $\mathbf{A} \mathbf{A}^{-1} = \mathbf{A}^{-1} \mathbf{A} = \mathbf{I}$, the identity matrix).
2. $\det(\mathbf{A}) \neq 0$ (i.e., the determinant of $\mathbf{A}$ is nonzero).

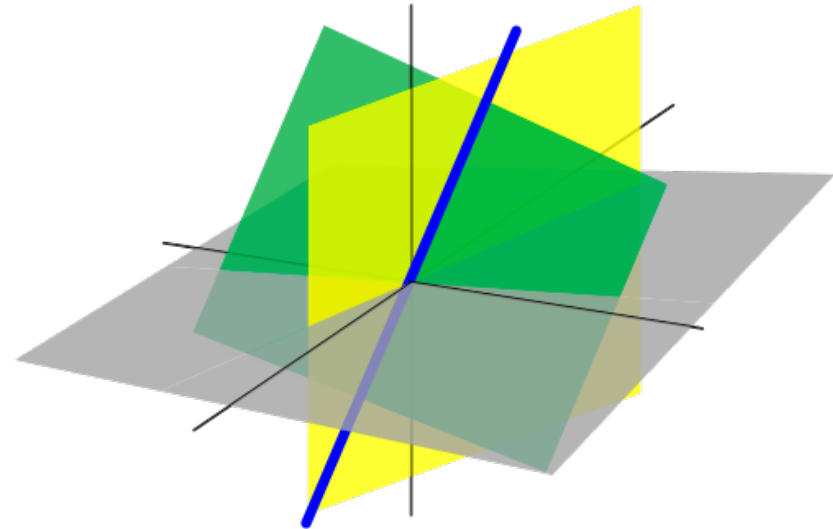
Existence and Uniqueness

Solutions to matrix and linear systems

Intersections of “lines”:



Intersections of “planes”:



Existence and Uniqueness

Example

$$2x_1 + 3x_2 = b_1$$

$$5x_1 + 4x_2 = b_2$$

$$\mathbf{A}\mathbf{x} = \begin{bmatrix} 2 & 3 \\ 5 & 4 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \end{bmatrix} = \mathbf{b}$$

What's is $\mathbf{A}^{-1}$ in the two cases? What does that mean?

Consider instead the matrix equation:

$$\mathbf{A}\mathbf{x} = \begin{bmatrix} 2 & 3 \\ 4 & 6 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \end{bmatrix} = \mathbf{b}$$

Sensitivity and Conditioning:

We have until now stated criteria for the existence and uniqueness of a solution to a linear system $\mathbf{Ax} = \mathbf{b}$. We now study “sensitivity”: How sensitive is the solution $\mathbf{x}$ to perturbations in the input data, which in our setup is $\mathbf{A}$ and $\mathbf{b}$.

For determining the “*size of perturbations*” in both the solution and the input, we need ways to define “*sizes*”, which we in our more general case with matrices refer to as “norms”

Sensitivity and Conditioning:

Vector norms:

Sensitivity and Conditioning:

Vector norms:

The “p-norm”

$$\|\mathbf{x}\|_p = \left(\sum_{i=1}^n |x_i|^p \right)^{1/p}$$

Sensitivity and Conditioning:

Vector norms:

The “p-norm”

$$\|\mathbf{x}\|_p = \left(\sum_{i=1}^n |x_i|^p \right)^{1/p}$$

The “1-norm”

$$\|\mathbf{x}\|_1 = \sum_{i=1}^n |x_i|$$

Sensitivity and Conditioning:

Vector norms:

The “p-norm”

$$\|\mathbf{x}\|_p = \left(\sum_{i=1}^n |x_i|^p \right)^{1/p}$$

The “1-norm”

$$\|\mathbf{x}\|_1 = \sum_{i=1}^n |x_i|$$

The “2-norm”

$$\|\mathbf{x}\|_2 = \left(\sum_{i=1}^n |x_i|^2 \right)^{1/2}$$

Sensitivity and Conditioning:

Vector norms:

The “p-norm”

$$\|\mathbf{x}\|_p = \left(\sum_{i=1}^n |x_i|^p \right)^{1/p}$$

The “1-norm”

$$\|\mathbf{x}\|_1 = \sum_{i=1}^n |x_i|$$

The “2-norm”

$$\|\mathbf{x}\|_2 = \left(\sum_{i=1}^n |x_i|^2 \right)^{1/2}$$

The “infty-norm”

$$\|\mathbf{x}\|_\infty = \max_{1 \leq i \leq n} |x_i|$$

Sensitivity and Conditioning:

Vector norms:

The “p-norm”

$$\|\mathbf{x}\|_p = \left(\sum_{i=1}^n |x_i|^p \right)^{1/p}$$

The “1-norm”

$$\|\mathbf{x}\|_1 = \sum_{i=1}^n |x_i|$$

The “2-norm”

$$\|\mathbf{x}\|_2 = \left(\sum_{i=1}^n |x_i|^2 \right)^{1/2}$$

The “infty-norm”

$$\|\mathbf{x}\|_\infty = \max_{1 \leq i \leq n} |x_i|$$

All of these norms give similar results qualitatively, but in certain circumstances a particular norm may be easiest to work with analytically or computationally. Either the 1-norm or the ∞ -norm is usually used in analyzing the sensitivity of solutions to linear systems

Sensitivity and Conditioning:

Vector norms:

Example:

$$\mathbf{x} = [-1.6, 1.2]^T$$

$$\|\mathbf{x}\|_1 = \sum_{i=1}^n |x_i|$$

$$\|\mathbf{x}\|_2 = \left(\sum_{i=1}^n |x_i|^2 \right)^{1/2}$$

$$\|\mathbf{x}\|_\infty = \max_{1 \leq i \leq n} |x_i|$$

Sensitivity and Conditioning:

Vector norms:

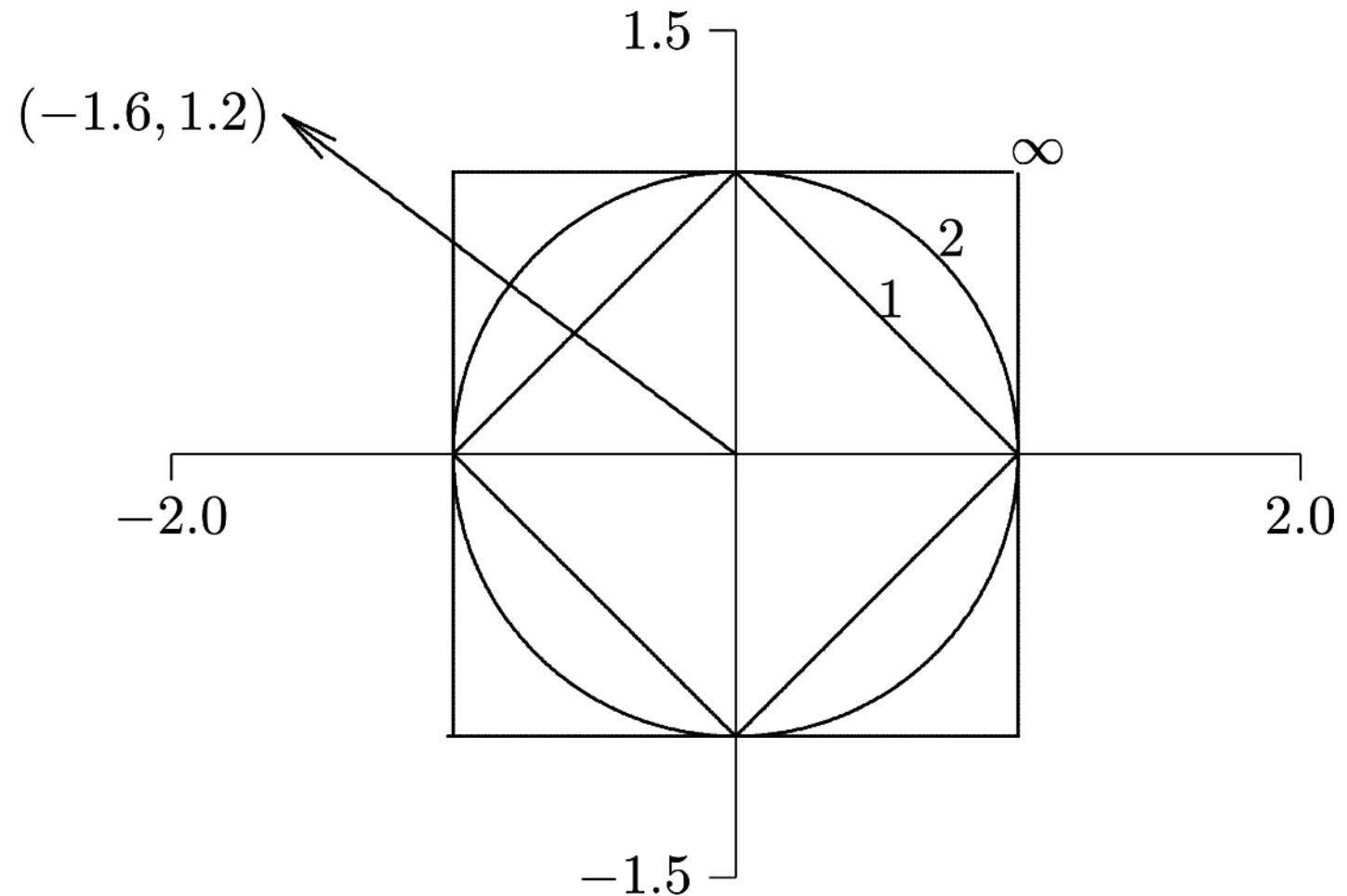
Example:

$$\mathbf{x} = [-1.6, 1.2]^T$$

$$\|\mathbf{x}\|_1 = 2.8, \quad \|\mathbf{x}\|_2 = 2.0, \quad \|\mathbf{x}\|_\infty = 1.6.$$

Sensitivity and Conditioning:

Geometrical explanation:



Example from book

Sensitivity and Conditioning:

Vector norms:

For any n -vector $\mathbf{x}$:

$$\|\mathbf{x}\|_1 \geq \|\mathbf{x}\|_2 \geq \|\mathbf{x}\|_\infty$$

“Norm Rules”

$$\|\mathbf{x}\|_1 \leq \sqrt{n} \|\mathbf{x}\|_2, \quad \|\mathbf{x}\|_2 \leq \sqrt{n} \|\mathbf{x}\|_\infty, \quad \text{and} \quad \|\mathbf{x}\|_1 \leq n \|\mathbf{x}\|_\infty$$

Sensitivity and Conditioning:

Vector norms:

More general definition of “norm-function”:

1. $\|\mathbf{x}\| > 0$ if $\mathbf{x} \neq \mathbf{0}$.
2. $\|\gamma\mathbf{x}\| = |\gamma| \cdot \|\mathbf{x}\|$ for any scalar γ .
3. $\|\mathbf{x} + \mathbf{y}\| \leq \|\mathbf{x}\| + \|\mathbf{y}\|$ (triangle inequality).

Sensitivity and Conditioning:

Matrix norms:

The norm of a matrix measures the maximum stretching the matrix does to any vector.

$$\|A\| = \max_{x \neq 0} \frac{\|Ax\|}{\|x\|}$$

Sensitivity and Conditioning:

Matrix norms:

$$\|\mathbf{A}\|_1 = \max_j \sum_{i=1}^m |a_{ij}|$$

$$\|\mathbf{A}\|_\infty = \max_i \sum_{j=1}^n |a_{ij}|$$

Sensitivity and Conditioning:

Matrix norms:

Example:

$$\mathbf{A} = \begin{bmatrix} 2 & -1 & 1 \\ 1 & 0 & 1 \\ 3 & -1 & 4 \end{bmatrix}$$

$$\|\mathbf{A}\|_1 = 6 \quad \text{and} \quad \|\mathbf{A}\|_\infty = 8$$

Sensitivity and Conditioning:

Matrix norms: More general definition

1. $\|\mathbf{A}\| > 0$ if $\mathbf{A} \neq \mathbf{O}$.
2. $\|\gamma\mathbf{A}\| = |\gamma| \cdot \|\mathbf{A}\|$ for any scalar γ .
3. $\|\mathbf{A} + \mathbf{B}\| \leq \|\mathbf{A}\| + \|\mathbf{B}\|$.
4. $\|\mathbf{AB}\| \leq \|\mathbf{A}\| \cdot \|\mathbf{B}\|$.
5. $\|\mathbf{Ax}\| \leq \|\mathbf{A}\| \cdot \|\mathbf{x}\|$ for any vector $\mathbf{x}$.

Sensitivity and Conditioning:

Matrix Condition Number

$$\text{cond}(\mathbf{A}) = \|\mathbf{A}\| \cdot \|\mathbf{A}^{-1}\|$$

Remember:

$$|\text{Relative forward error}| = \text{condition number} \times |\text{relative backward error}|.$$

Thus, the condition number can be interpreted as an “amplification factor”

Sensitivity and Conditioning:

Matrix Condition Number

$$\mathbf{A} = \begin{bmatrix} 2 & -1 & 1 \\ 1 & 0 & 1 \\ 3 & -1 & 4 \end{bmatrix} \quad \mathbf{A}^{-1} = \begin{bmatrix} 0.5 & 1.5 & -0.5 \\ -0.5 & 2.5 & -0.5 \\ -0.5 & -0.5 & 0.5 \end{bmatrix}$$

Example:

$$\|\mathbf{A}^{-1}\|_1 = 4.5 \quad \text{and} \quad \|\mathbf{A}^{-1}\|_\infty = 3.5$$

$$\|\mathbf{A}\|_1 = 6 \quad \text{and} \quad \|\mathbf{A}\|_\infty = 8$$

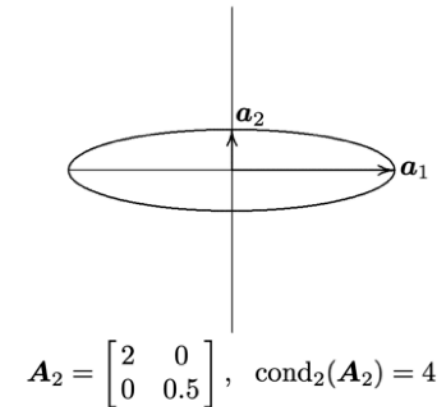
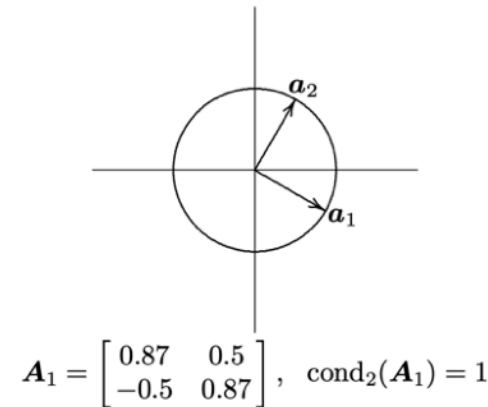
$$\text{cond}_1(\mathbf{A}) = \|\mathbf{A}\|_1 \cdot \|\mathbf{A}^{-1}\|_1 = 6 \cdot 4.5 = 27$$

$$\text{cond}_\infty(\mathbf{A}) = \|\mathbf{A}\|_\infty \cdot \|\mathbf{A}^{-1}\|_\infty = 8 \cdot 3.5 = 28$$

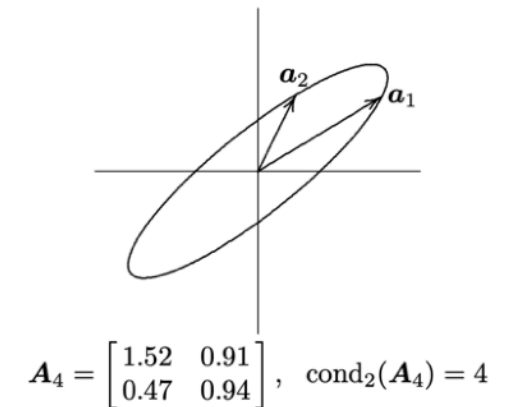
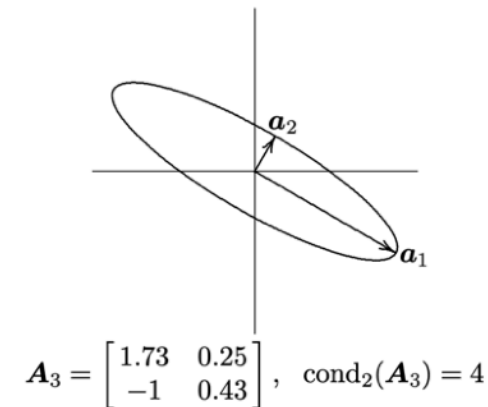
Sensitivity and Conditioning:

Matrix Condition Number

The condition matrix condition number measures the ratio between the maximum to the minimum stretching the matrix does to any non-zero vectors:



The condition number is a measure of how close a matrix is to being singular



Sensitivity and Conditioning:

Matrix Condition Number

Usefulness of the Condition Number:

The **condition number** is a measure of how close a matrix is to being **singular**: a matrix with a large condition number is nearly singular, whereas a matrix with a condition number close to 1 is far from being singular.

The usefulness of the condition number is in assessing the accuracy of solutions to linear systems.

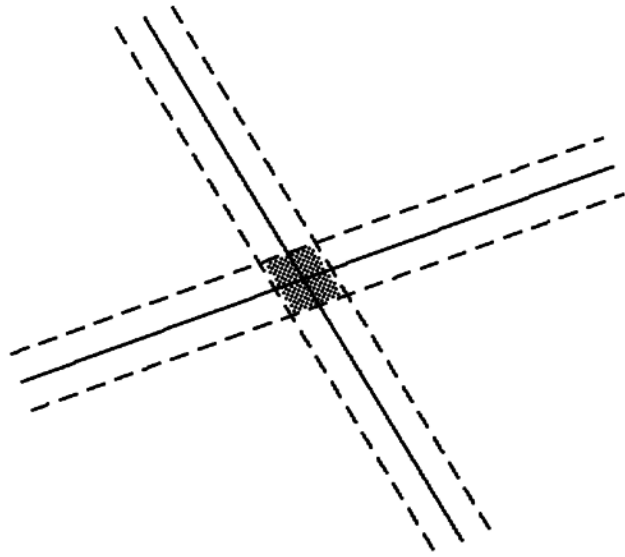
In addition to being a reliable indicator of near singularity, the condition number also provides a **quantitative bound for the error in the computed solution** to a linear system:

$$\frac{\|\Delta \mathbf{x}\|}{\|\mathbf{x}\|} \leq \text{cond}(\mathbf{A}) \frac{\|\Delta \mathbf{b}\|}{\|\mathbf{b}\|}$$

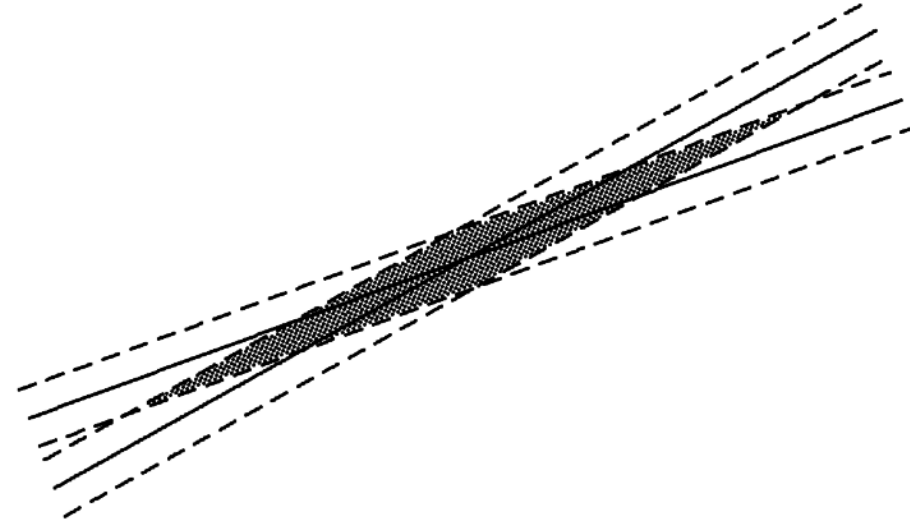
$$\frac{\|\Delta \mathbf{x}\|}{\|\hat{\mathbf{x}}\|} \leq \text{cond}(\mathbf{A}) \frac{\|\mathbf{E}\|}{\|\mathbf{A}\|}$$

$$\frac{\|\hat{\mathbf{x}} - \mathbf{x}\|}{\|\mathbf{x}\|} \lesssim \text{cond}(\mathbf{A}) \epsilon_{\text{mach}}$$

Therefore: a computed solution can be expected to lose about $\log(\text{cond}(\mathbf{A}))$ decimal digits!



well-conditioned



ill-conditioned

“Amplification” of errors as the system becomes closer to “singular”

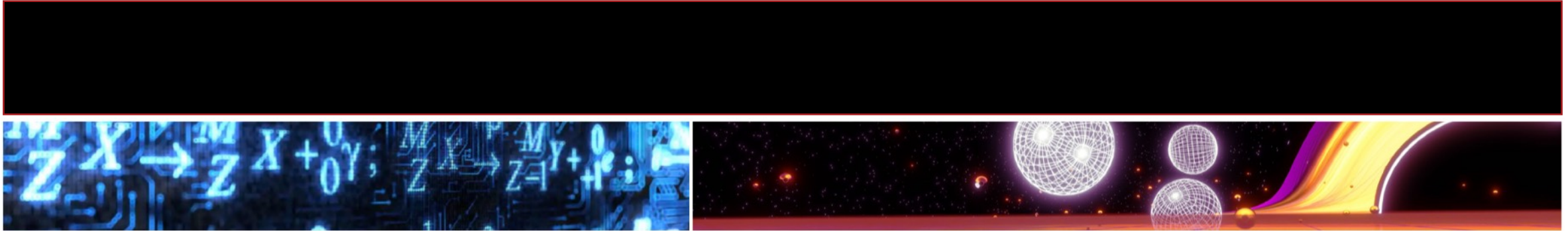
Solving Linear Systems

- Replacing infinite-dimensional spaces with finite-dimensional spaces
- Replacing infinite processes with finite processes, such as replacing integrals or infinite series with finite sums, or derivatives with finite differences
- Replacing differential equations with algebraic equations
- Replacing nonlinear problems with linear problems
- Replacing high-order systems with low-order systems
- Replacing complicated functions with simple functions, such as polynomials
- Replacing general matrices with matrices having a simpler form

Solving Linear Systems

Algorithm 2.1 Forward-Substitution for Lower Triangular System

```
for  $j = 1$  to  $n$                                 { loop over columns }
  if  $\ell_{jj} = 0$  then stop                       { stop if matrix is singular }
   $x_j = b_j / \ell_{jj}$                            { compute solution component }
  for  $i = j + 1$  to  $n$ 
     $b_i = b_i - \ell_{ij}x_j$                        { update right-hand side }
  end
end
end
```



Johan Samsing
Faculty of Science
University of Copenhagen

Lecture W2 - L1

Linear Least Squares Problems

It is common for a variety of theoretical and practical reasons to seek a lower-dimensional approximation to some higher-dimensional object.

Linear Least Squares Problems

It is common for a variety of theoretical and practical reasons to seek a lower-dimensional approximation to some higher-dimensional object.

1. A square linear system is exactly determined: with the same number of equations and unknowns, a unique solution always exists, provided the matrix is nonsingular.

Linear Least Squares Problems

It is common for a variety of theoretical and practical reasons to seek a lower-dimensional approximation to some higher-dimensional object.

1. A square linear system is exactly determined: with the same number of equations and unknowns, a unique solution always exists, provided the matrix is nonsingular.
2. In the current setting, however, we presume that the given data are noisy or contain irrelevant detail, and hence there is no particular virtue in fitting them exactly.

Example 3.1 Overdetermined System. A surveyor is to determine the heights of three hills above some reference point. Sighting first from the reference point, the surveyor measures their respective heights to be $x_1 = 1237$ ft., $x_2 = 1941$ ft., and $x_3 = 2417$ ft. To confirm these initial measurements, the surveyor climbs to the top of the first hill and measures the height of the second hill above the first to be $x_2 - x_1 = 711$ ft., and the third above the first to be $x_3 - x_1 = 1177$ ft. Finally, the surveyor climbs to the top of the second hill and measures the height of the third hill above the second to be $x_3 - x_2 = 475$ ft. Noting the inconsistency among these measurements, the surveyor uses methods we will soon demonstrate to compute the least squares solution to the overdetermined linear system

EXAMPLE:

Overdetermined System.

$$\mathbf{Ax} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \\ -1 & 1 & 0 \\ -1 & 0 & 1 \\ 0 & -1 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} \approx \begin{bmatrix} 1237 \\ 1941 \\ 2417 \\ 711 \\ 1177 \\ 475 \end{bmatrix} = \mathbf{b}$$

EXAMPLE:

Overdetermined System.

$$\mathbf{Ax} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \\ -1 & 1 & 0 \\ -1 & 0 & 1 \\ 0 & -1 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} \approx \begin{bmatrix} 1237 \\ 1941 \\ 2417 \\ 711 \\ 1177 \\ 475 \end{bmatrix} = \mathbf{b}$$

$$\mathbf{x} = [1236, 1943, 2416]^T$$

Are these number consistent?

Linear Least Squares Problems

An overdetermined system with more equations than unknowns. Writing the linear system in matrix- vector notation, we have

$$\mathbf{Ax} = \mathbf{b}$$

where $\mathbf{A}$ is an $m \times n$ matrix with $m > n$, $\mathbf{b}$ is an m -vector, and $\mathbf{x}$ an n -vector.

Remember for an $n \times n$: has solution if A is non-singular:

$$\mathbf{Ax} = \begin{bmatrix} 2 & 3 \\ 5 & 4 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \end{bmatrix} = \mathbf{b}$$

Linear Least Squares Problems

An overdetermined system with more equations than unknowns. Writing the linear system in matrix- vector notation, we have

$$\mathbf{Ax} = \mathbf{b}$$

where $\mathbf{A}$ is an $m \times n$ matrix with $m > n$, $\mathbf{b}$ is an m -vector, and $\mathbf{x}$ an n -vector.

BUT for an $m \times n$: There is no general solution!

For an overdetermined system there is usually no solution in the usual sense. Instead, we minimize the distance between the left and right sides, i.e., we minimize some norm of the residual vector $\mathbf{r} = \mathbf{b} - \mathbf{Ax}$ as a function of $\mathbf{x}$.

Data Fitting:

Given data points (t_i, y_i) , $i = 1, \dots, m$, we wish to find the n -vector $\mathbf{x}$ of parameters that gives the “best fit” to the data by the model function $f(t, \mathbf{x})$.

$$\min_{\mathbf{x}} \sum_{i=1}^m (y_i - f(t_i, \mathbf{x}))^2$$

Data Fitting:

A data-fitting problem is linear if the function f is linear in the components of the parameter vector $\mathbf{x}$, which means that f is a linear combination:

$$f(t, \mathbf{x}) = x_1 \phi_1(t) + x_2 \phi_2(t) + \cdots + x_n \phi_n(t)$$

Data Fitting:

A data-fitting problem is linear if the function f is linear in the components of the parameter vector $\mathbf{x}$, which means that f is a linear combination:

$$f(t, \mathbf{x}) = x_1 \phi_1(t) + x_2 \phi_2(t) + \cdots + x_n \phi_n(t)$$

For example:

$$f(t, \mathbf{x}) = x_1 + x_2 t + x_3 t^2 + \cdots + x_n t^{n-1}$$

Data Fitting:

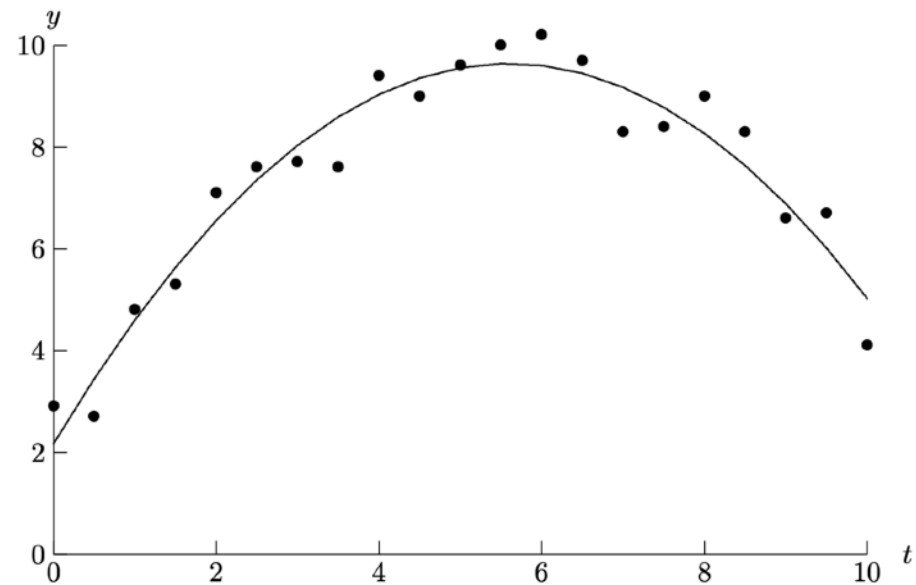
For example, in fitting a quadratic polynomial, which has three parameters, to five data points $(t_1, y_1), \dots, (t_5, y_5)$, the matrix A is 5×3 , and the problem has the form:

$$\mathbf{Ax} = \begin{bmatrix} 1 & t_1 & t_1^2 \\ 1 & t_2 & t_2^2 \\ 1 & t_3 & t_3^2 \\ 1 & t_4 & t_4^2 \\ 1 & t_5 & t_5^2 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} \cong \begin{bmatrix} y_1 \\ y_2 \\ y_3 \\ y_4 \\ y_5 \end{bmatrix} = \mathbf{b}$$

EXAMPLE:

Dataset:

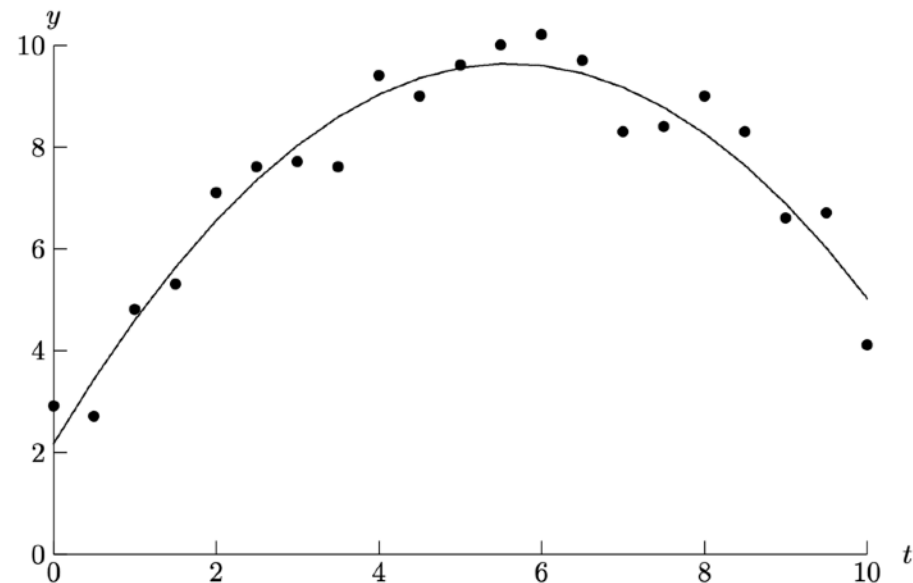
t	0.0	0.5	1.0	1.5	2.0	2.5	3.0	3.5	4.0	4.5	5.0
y	2.9	2.7	4.8	5.3	7.1	7.6	7.7	7.6	9.4	9.0	9.6
t	5.5	6.0	6.5	7.0	7.5	8.0	8.5	9.0	9.5	10.0	
y	10.0	10.2	9.7	8.3	8.4	9.0	8.3	6.6	6.7	4.1	



EXAMPLE:

Dataset:

t	0.0	0.5	1.0	1.5	2.0	2.5	3.0	3.5	4.0	4.5	5.0
y	2.9	2.7	4.8	5.3	7.1	7.6	7.7	7.6	9.4	9.0	9.6
t	5.5	6.0	6.5	7.0	7.5	8.0	8.5	9.0	9.5	10.0	
y	10.0	10.2	9.7	8.3	8.4	9.0	8.3	6.6	6.7	4.1	



Data fitting:

$$Ax = \begin{bmatrix} 1 & 0.0 & 0.0 \\ 1 & 0.5 & 0.25 \\ 1 & 1.0 & 1.0 \\ \vdots & \vdots & \vdots \\ 1 & 10.0 & 100.0 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} \cong \begin{bmatrix} 2.9 \\ 2.7 \\ 4.8 \\ \vdots \\ 4.1 \end{bmatrix} = b.$$

$$p(t) = 2.18 + 2.67t - 0.238t^2$$

Existence and Uniqueness

Consider

$$\mathbf{Ax} = \mathbf{b}$$

For a general $m \times n$ linear system:

Can $\mathbf{b}$ be expressed as a linear combination of the columns of $\mathbf{A}$?

Existence and Uniqueness

Consider

$$\mathbf{Ax} = \mathbf{b}$$

For a general $m \times n$ linear system:

Can $\mathbf{b}$ be expressed as a linear combination of the columns of $\mathbf{A}$?

For square systems ($m = n$), the answer is always “yes” for nonsingular $\mathbf{A}$.

For *overdetermined* systems ($m > n$), on the other hand, the answer is usually “no” unless $\mathbf{b}$ happens to lie in $\text{span}(\mathbf{A})$, which is highly unlikely in most applications(!)

Existence and Uniqueness

Consider

$$\mathbf{Ax} = \mathbf{b}$$

For a general $m \times n$ linear system:

Can $\mathbf{b}$ be expressed as a linear combination of the columns of $\mathbf{A}$?

For square systems ($m = n$), the answer is always “yes” for nonsingular $\mathbf{A}$.

For *overdetermined* systems ($m > n$), on the other hand, the answer is usually “no” unless $\mathbf{b}$ happens to lie in $\text{span}(\mathbf{A})$, which is highly unlikely in most applications(!)

- *The solution to a ($m > n$) system is unique if, and only if, $\mathbf{A}$ has full column rank, i.e., $\text{rank}(\mathbf{A}) = n$*

A possible Solution

We wish to minimize the squared Euclidean norm of the residual vector $\mathbf{r} = \mathbf{b} - \mathbf{Ax}$.

A possible Solution

We wish to minimize the squared Euclidean norm of the residual vector $\mathbf{r} = \mathbf{b} - \mathbf{Ax}$.

$$\phi(\mathbf{x}) = \|\mathbf{r}\|_2^2 = \mathbf{r}^T \mathbf{r} = (\mathbf{b} - \mathbf{Ax})^T (\mathbf{b} - \mathbf{Ax})$$

A possible Solution

We wish to minimize the squared Euclidean norm of the residual vector $\mathbf{r} = \mathbf{b} - \mathbf{A}\mathbf{x}$.

$$\begin{aligned}\phi(\mathbf{x}) &= \|\mathbf{r}\|_2^2 = \mathbf{r}^T \mathbf{r} = (\mathbf{b} - \mathbf{A}\mathbf{x})^T (\mathbf{b} - \mathbf{A}\mathbf{x}) \\ &= \mathbf{b}^T \mathbf{b} - 2\mathbf{x}^T \mathbf{A}^T \mathbf{b} + \mathbf{x}^T \mathbf{A}^T \mathbf{A} \mathbf{x}\end{aligned}$$

A possible Solution

We wish to minimize the squared Euclidean norm of the residual vector $\mathbf{r} = \mathbf{b} - \mathbf{A}\mathbf{x}$.

$$\begin{aligned}\phi(\mathbf{x}) &= \|\mathbf{r}\|_2^2 = \mathbf{r}^T \mathbf{r} = (\mathbf{b} - \mathbf{A}\mathbf{x})^T (\mathbf{b} - \mathbf{A}\mathbf{x}) \\ &= \mathbf{b}^T \mathbf{b} - 2\mathbf{x}^T \mathbf{A}^T \mathbf{b} + \mathbf{x}^T \mathbf{A}^T \mathbf{A} \mathbf{x}\end{aligned}$$

$$\mathbf{0} = \nabla \phi(\mathbf{x}) = 2\mathbf{A}^T \mathbf{A} \mathbf{x} - 2\mathbf{A}^T \mathbf{b},$$

A possible Solution

We wish to minimize the squared Euclidean norm of the residual vector $\mathbf{r} = \mathbf{b} - \mathbf{A}\mathbf{x}$.

$$\begin{aligned}\phi(\mathbf{x}) &= \|\mathbf{r}\|_2^2 = \mathbf{r}^T \mathbf{r} = (\mathbf{b} - \mathbf{A}\mathbf{x})^T (\mathbf{b} - \mathbf{A}\mathbf{x}) \\ &= \mathbf{b}^T \mathbf{b} - 2\mathbf{x}^T \mathbf{A}^T \mathbf{b} + \mathbf{x}^T \mathbf{A}^T \mathbf{A} \mathbf{x}\end{aligned}$$

$$\mathbf{0} = \nabla \phi(\mathbf{x}) = 2\mathbf{A}^T \mathbf{A} \mathbf{x} - 2\mathbf{A}^T \mathbf{b},$$

$$\mathbf{A}^T \mathbf{A} \mathbf{x} = \mathbf{A}^T \mathbf{b}$$

“Normal equations method”

We wish to minimize the squared Euclidean norm of the residual vector $\mathbf{r} = \mathbf{b} - \mathbf{A}\mathbf{x}$.

$$\mathbf{A}^T \mathbf{A} \mathbf{x} = \mathbf{A}^T \mathbf{b}$$

Can now be solved using methods we looked at last week

$$\mathbf{A}^T \mathbf{A} \quad (\text{is an } n \times n \text{ matrix})$$

Going back to this example:

$$\mathbf{Ax} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \\ -1 & 1 & 0 \\ -1 & 0 & 1 \\ 0 & -1 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} \cong \begin{bmatrix} 1237 \\ 1941 \\ 2417 \\ 711 \\ 1177 \\ 475 \end{bmatrix} = \mathbf{b}$$

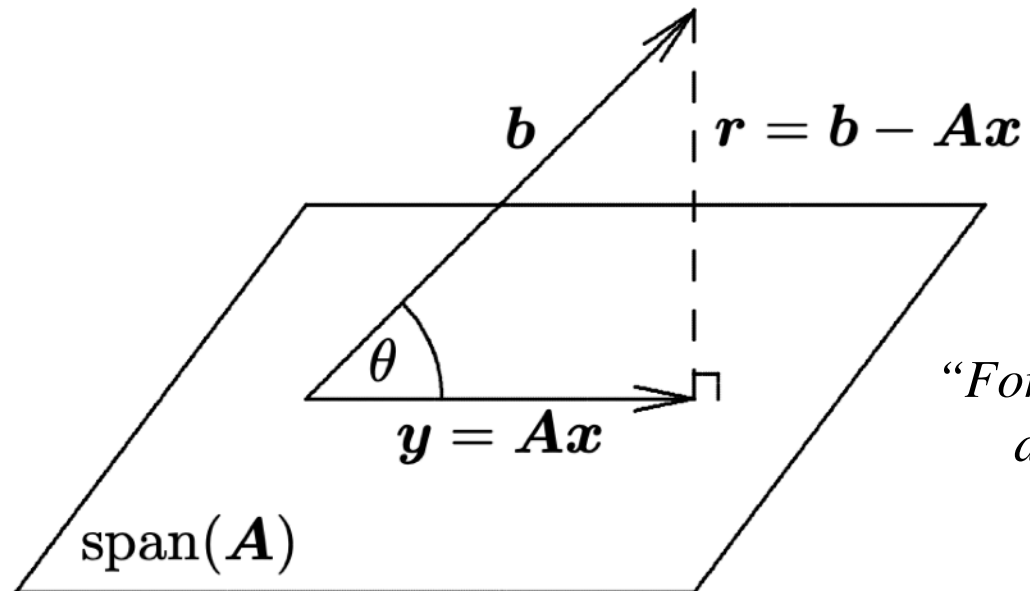
Going back to this example:

$$\mathbf{A}\mathbf{x} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \\ -1 & 1 & 0 \\ -1 & 0 & 1 \\ 0 & -1 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} \cong \begin{bmatrix} 1237 \\ 1941 \\ 2417 \\ 711 \\ 1177 \\ 475 \end{bmatrix} = \mathbf{b}$$

$$\mathbf{A}^T \mathbf{A}\mathbf{x} = \begin{bmatrix} 3 & -1 & -1 \\ -1 & 3 & -1 \\ -1 & -1 & 3 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} -651 \\ 2177 \\ 4069 \end{bmatrix} = \mathbf{A}^T \mathbf{b}$$

$$\mathbf{x} = [1236, 1943, 2416]^T$$

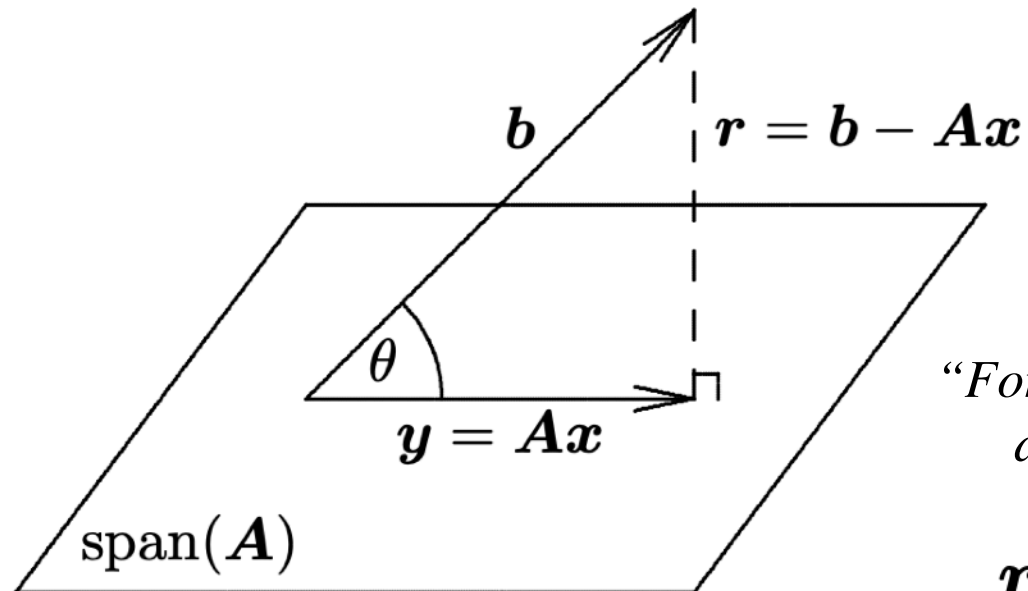
Geometric Considerations:



$$\mathbf{A}\mathbf{x} \cong \mathbf{b}$$

“For a least squares problem with $m > n$, the m -vector $\mathbf{b}$ generally does not lie in $\text{span}(\mathbf{A})$, a subspace of dimension at most n ”

Geometric Considerations:



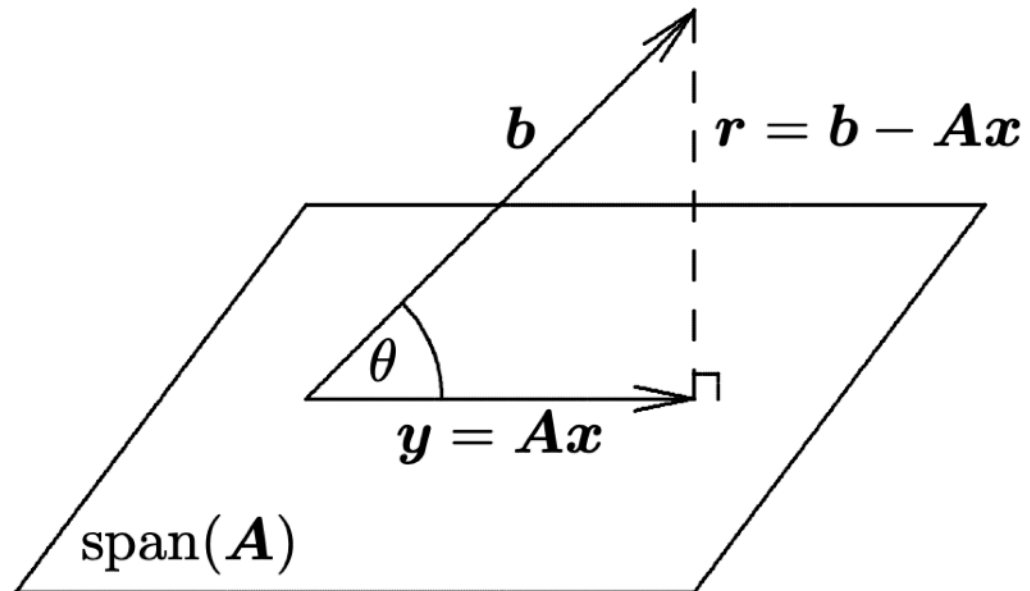
$$\mathbf{Ax} \cong \mathbf{b}$$

“For a least squares problem with $m > n$, the m -vector $\mathbf{b}$ generally does not lie in $\text{span}(\mathbf{A})$, a subspace of dimension at most n ”

$\mathbf{r} = \mathbf{b} - \mathbf{Ax}$ is orthogonal to $\text{span}(\mathbf{A})$

*The **span** of a set of vectors is the set of all linear combinations of the vectors*

Geometric Considerations:

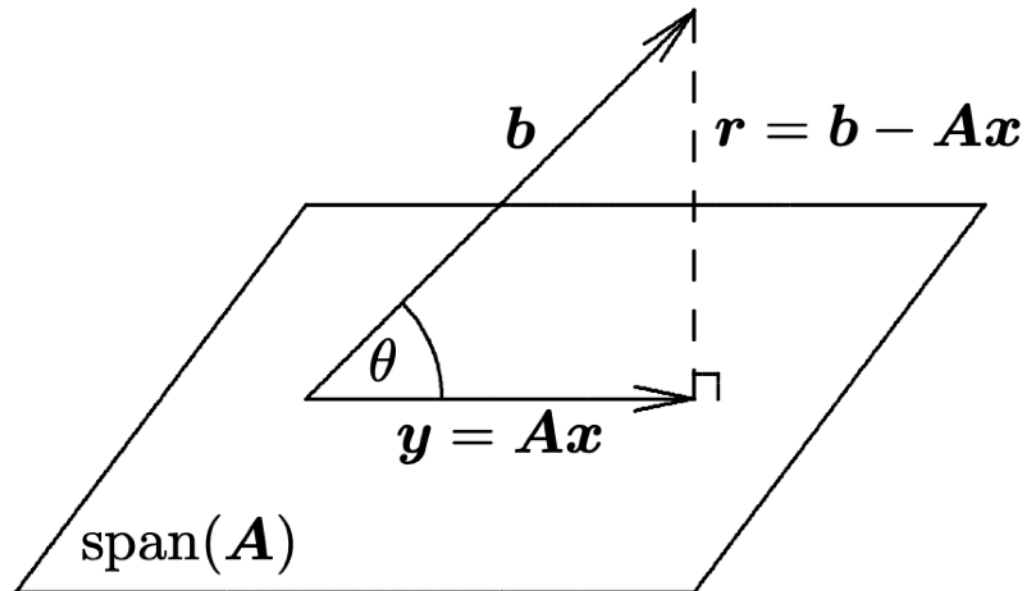


For the least squares solution the residual vector $\mathbf{r}$ must be orthogonal to each column of $\mathbf{A}$, and hence we must have:

First remember:

$$\mathbf{v}_1^T \mathbf{v}_2 = \|\mathbf{v}_1\|_2 \cdot \|\mathbf{v}_2\|_2 \cdot \cos(\theta)$$

Geometric Considerations:



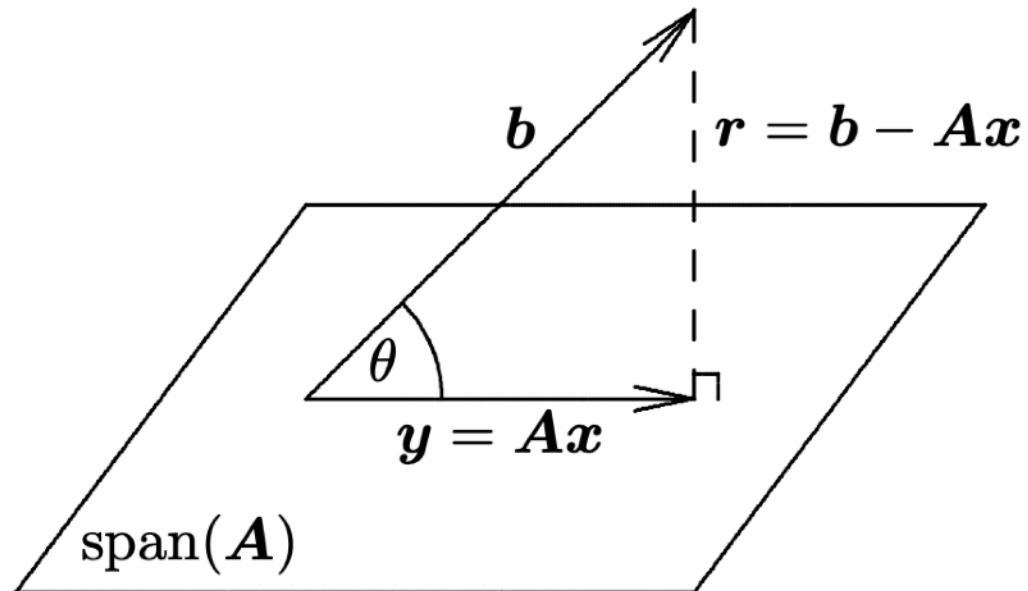
For the least squares solution the residual vector $\mathbf{r}$ must be orthogonal to each column of $\mathbf{A}$, and hence we must have:

$$\mathbf{0} = \mathbf{A}^T \mathbf{r} = \mathbf{A}^T (\mathbf{b} - \mathbf{A}\mathbf{x})$$

$$\mathbf{A}^T \mathbf{A}\mathbf{x} = \mathbf{A}^T \mathbf{b}$$

...which is the same system of normal equations we derived earlier using calculus.

Geometric Considerations:



Back to the example:

$$\mathbf{r} = \mathbf{b} - \mathbf{A}\mathbf{x} = \mathbf{b} - \mathbf{y} = \begin{bmatrix} 1237 \\ 1941 \\ 2417 \\ 711 \\ 1177 \\ 475 \end{bmatrix} - \begin{bmatrix} 1236 \\ 1943 \\ 2416 \\ 707 \\ 1180 \\ 473 \end{bmatrix} = \begin{bmatrix} 1 \\ -2 \\ 1 \\ 4 \\ -3 \\ 2 \end{bmatrix}$$

$$\mathbf{A}^T \mathbf{r} = \mathbf{0} \quad \text{Check!}$$

$$\mathbf{A}\mathbf{x} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \\ -1 & 1 & 0 \\ -1 & 0 & 1 \\ 0 & -1 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} \cong \begin{bmatrix} 1237 \\ 1941 \\ 2417 \\ 711 \\ 1177 \\ 475 \end{bmatrix} = \mathbf{b}$$

Transformations:

We will now consider several methods for transforming an overdetermined linear least squares problem into a square (ultimately triangular) linear system

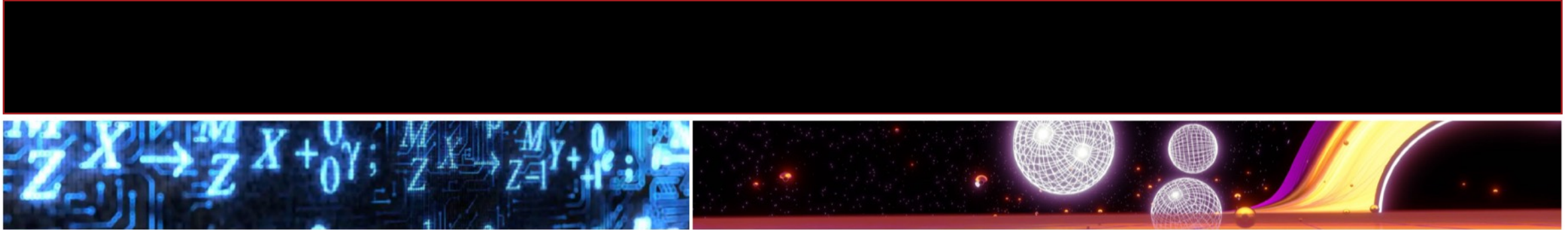
Problem with the previous method: “**normal equations method**”

$$\mathbf{A}^T \mathbf{A} \mathbf{x} = \mathbf{A}^T \mathbf{b}$$

$$\mathbf{A} = \begin{bmatrix} 1 & 1 \\ \epsilon & 0 \\ 0 & \epsilon \end{bmatrix}$$

$$\mathbf{A}^T \mathbf{A} = \begin{bmatrix} 1 + \epsilon^2 & 1 \\ 1 & 1 + \epsilon^2 \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}$$

(Which is singular...)



Johan Samsing
Faculty of Science
University of Copenhagen

Lecture W2 - L2

Linear Least Squares Problems

$$\mathbf{Ax} = \mathbf{b}$$

1. A square linear system is exactly determined: with the same number of equations and unknowns, a unique solution always exists, provided the matrix is nonsingular.

$$\mathbf{Ax} = \begin{bmatrix} 2 & 3 \\ 5 & 4 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \end{bmatrix} = \mathbf{b}$$

Linear Least Squares Problems

$$\mathbf{Ax} = \mathbf{b}$$

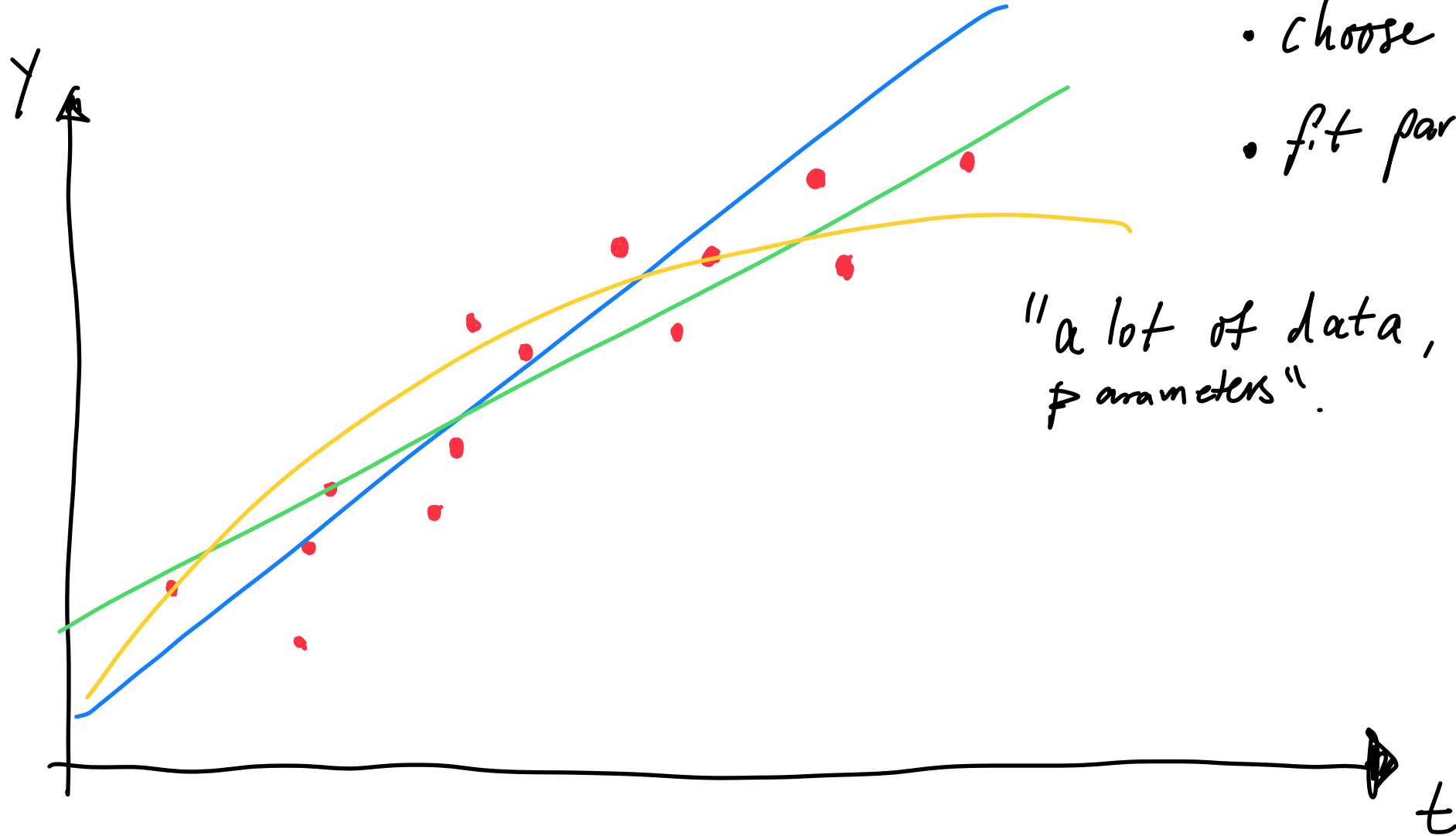
1. A square linear system is exactly determined: with the same number of equations and unknowns, a unique solution always exists, provided the matrix is nonsingular.
2. In the current setting, however, we presume that the given data are noisy or contain irrelevant detail, and hence there is no particular virtue in fitting them exactly.

$$\mathbf{Ax} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \\ -1 & 1 & 0 \\ -1 & 0 & 1 \\ 0 & -1 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} \approx \begin{bmatrix} 1237 \\ 1941 \\ 2417 \\ 711 \\ 1177 \\ 475 \end{bmatrix} = \mathbf{b}$$

Linear Least Squares Problems

Lets consider an example with noisy data

Linear Least Squares Problems

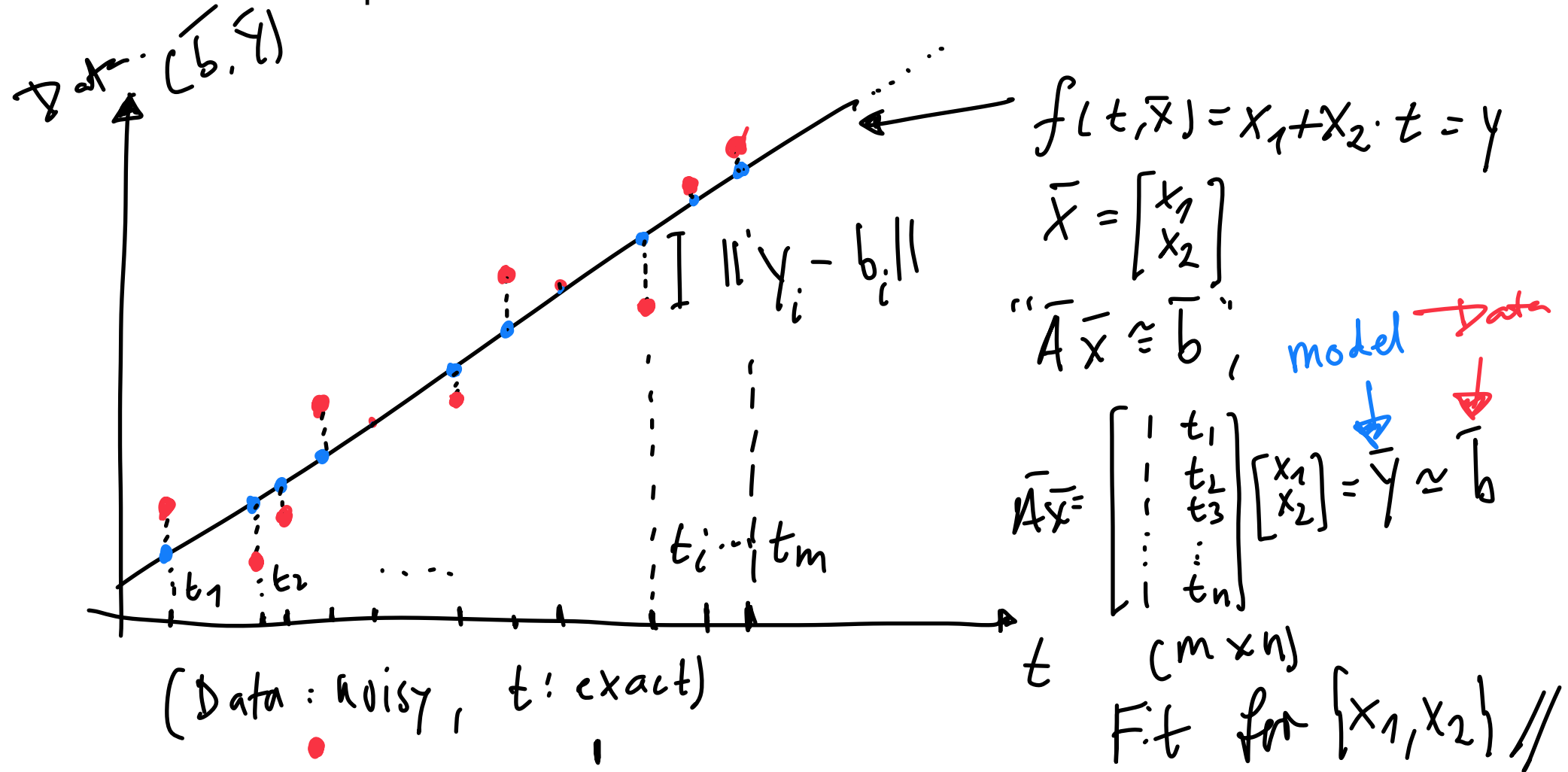


- choose function
- fit parameters

"a lot of data, few parameters".

Linear Least Squares Problems

We wish to minimize the squared Euclidean norm of the residual vector $\mathbf{r} = \mathbf{b} - \mathbf{A}\mathbf{x}$.



Linear Least Squares Problems

We wish to minimize the squared Euclidean norm of the residual vector $\mathbf{r} = \mathbf{b} - \mathbf{A}\mathbf{x}$.

In mathematical terms we can write this as:

Define:
$$\phi(\mathbf{x}) = \|\mathbf{r}\|_2^2 = \mathbf{r}^T \mathbf{r} = (\mathbf{b} - \mathbf{A}\mathbf{x})^T (\mathbf{b} - \mathbf{A}\mathbf{x})$$

Linear Least Squares Problems

We wish to minimize the squared Euclidean norm of the residual vector $\mathbf{r} = \mathbf{b} - \mathbf{A}\mathbf{x}$.

In mathematical terms we can write this as:

Define:
$$\phi(\mathbf{x}) = \|\mathbf{r}\|_2^2 = \mathbf{r}^T \mathbf{r} = (\mathbf{b} - \mathbf{A}\mathbf{x})^T (\mathbf{b} - \mathbf{A}\mathbf{x})$$

Find minimum:
$$\mathbf{0} = \nabla \phi(\mathbf{x}) = 2\mathbf{A}^T \mathbf{A}\mathbf{x} - 2\mathbf{A}^T \mathbf{b},$$

Linear Least Squares Problems

We wish to minimize the squared Euclidean norm of the residual vector $\mathbf{r} = \mathbf{b} - \mathbf{A}\mathbf{x}$.

In mathematical terms we can write this as:

Define:
$$\phi(\mathbf{x}) = \|\mathbf{r}\|_2^2 = \mathbf{r}^T \mathbf{r} = (\mathbf{b} - \mathbf{A}\mathbf{x})^T (\mathbf{b} - \mathbf{A}\mathbf{x})$$

Find minimum:
$$\mathbf{0} = \nabla \phi(\mathbf{x}) = 2\mathbf{A}^T \mathbf{A}\mathbf{x} - 2\mathbf{A}^T \mathbf{b},$$

Solution:
$$\mathbf{A}^T \mathbf{A}\mathbf{x} = \mathbf{A}^T \mathbf{b}$$

Linear Least Squares Problems

Initial Problem:

$$Ax = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \\ -1 & 1 & 0 \\ -1 & 0 & 1 \\ 0 & -1 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} \cong \begin{bmatrix} 1237 \\ 1941 \\ 2417 \\ 711 \\ 1177 \\ 475 \end{bmatrix} = b$$



Linear Least Squares Problems

Initial Problem:

minimize the squared Euclidean norm

$$Ax = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \\ -1 & 1 & 0 \\ -1 & 0 & 1 \\ 0 & -1 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} \cong \begin{bmatrix} 1237 \\ 1941 \\ 2417 \\ 711 \\ 1177 \\ 475 \end{bmatrix} = b$$

$$A^T Ax = A^T b$$



Linear Least Squares Problems

Initial Problem:

minimize the squared Euclidean norm

Ready to solve:

$$\mathbf{A}\mathbf{x} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \\ -1 & 1 & 0 \\ -1 & 0 & 1 \\ 0 & -1 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} \cong \begin{bmatrix} 1237 \\ 1941 \\ 2417 \\ 711 \\ 1177 \\ 475 \end{bmatrix} = \mathbf{b}$$

$$\mathbf{A}^T \mathbf{A} \mathbf{x} = \mathbf{A}^T \mathbf{b}$$

$$\mathbf{A}^T \mathbf{A} \mathbf{x} = \begin{bmatrix} 3 & -1 & -1 \\ -1 & 3 & -1 \\ -1 & -1 & 3 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} -651 \\ 2177 \\ 4069 \end{bmatrix} = \mathbf{A}^T \mathbf{b}$$



Linear Least Squares Problems

Strategy:

Rectangular $\longrightarrow$ square $\longrightarrow$ triangular

A

Linear Least Squares Problems

Strategy:

Rectangular $\longrightarrow$ square $\longrightarrow$ triangular

$$A \qquad A^T A$$

Linear Least Squares Problems

Strategy:

Rectangular $\longrightarrow$ square $\longrightarrow$ triangular

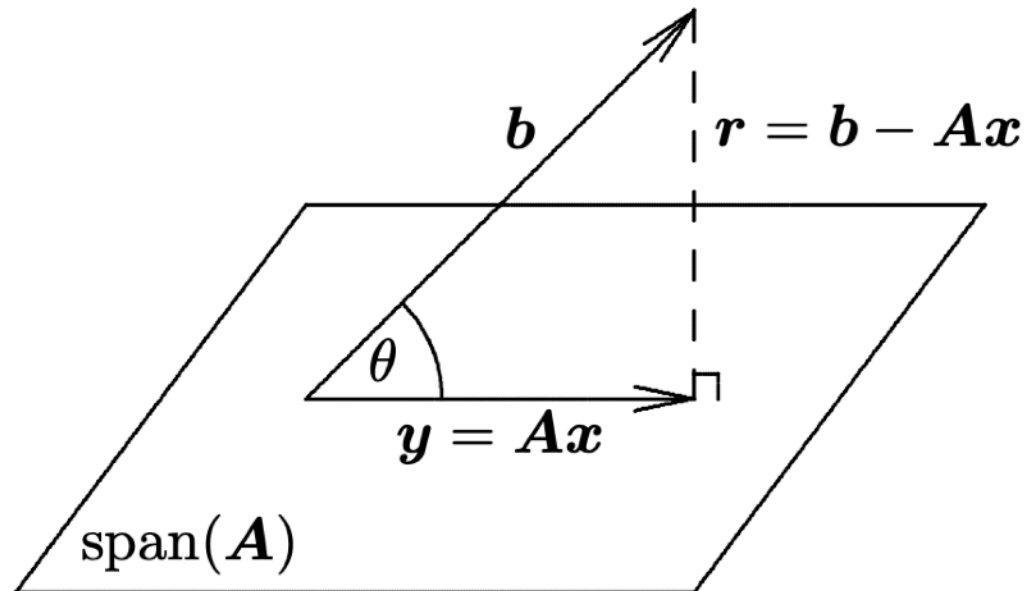
For example:

$$\mathbf{A} \qquad \mathbf{A}^T \mathbf{A} \qquad \begin{bmatrix} 1 & 0 \\ -a_2/a_1 & 1 \end{bmatrix} \begin{bmatrix} a_1 \\ a_2 \end{bmatrix} = \begin{bmatrix} a_1 \\ 0 \end{bmatrix}$$

Linear Least Squares Problems

Geometric Picture

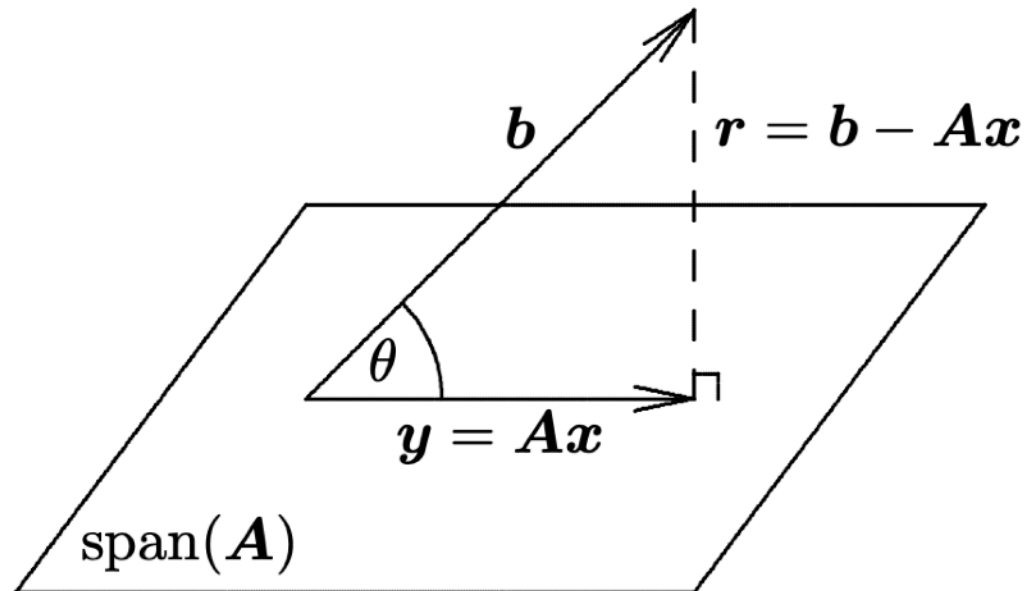
Linear Least Squares Problems



For the least squares solution the residual vector $\mathbf{r}$ must be orthogonal to each column of $\mathbf{A}$, and hence we must have:

$$\mathbf{0} = \mathbf{A}^T \mathbf{r} = \mathbf{A}^T (\mathbf{b} - \mathbf{A} \mathbf{x})$$

Linear Least Squares Problems



For the least squares solution the residual vector $\mathbf{r}$ must be orthogonal to each column of $\mathbf{A}$, and hence we must have:

$$\mathbf{0} = \mathbf{A}^T \mathbf{r} = \mathbf{A}^T (\mathbf{b} - \mathbf{A}\mathbf{x})$$

$$\mathbf{A}^T \mathbf{A}\mathbf{x} = \mathbf{A}^T \mathbf{b}$$

Linear Least Squares Problems

In other words

The best fit vector $\mathbf{y}$ is the *orthogonal projection* of $\mathbf{b}$ onto space that is spanned by $\mathbf{A}$.

Linear Least Squares Problems

Problems with:

$$\mathbf{A}^T \mathbf{A} \mathbf{x} = \mathbf{A}^T \mathbf{b}$$

Linear Least Squares Problems

Problems with:

$$\mathbf{A}^T \mathbf{A} \mathbf{x} = \mathbf{A}^T \mathbf{b}$$

$$\mathbf{A} = \begin{bmatrix} 1 & 1 \\ \epsilon & 0 \\ 0 & \epsilon \end{bmatrix}$$

Linear Least Squares Problems

Problems with:

$$\mathbf{A}^T \mathbf{A} \mathbf{x} = \mathbf{A}^T \mathbf{b}$$

$$\mathbf{A} = \begin{bmatrix} 1 & 1 \\ \epsilon & 0 \\ 0 & \epsilon \end{bmatrix} \longrightarrow \mathbf{A}^T \mathbf{A} = \begin{bmatrix} 1 + \epsilon^2 & 1 \\ 1 & 1 + \epsilon^2 \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}$$

3.1. If a vertical beam has a downward force applied at its lower end, the amount by which it stretches will be proportional to the magnitude of the force. Thus, the total length y of the beam is given by the equation

$$y = x_1 + x_2 t,$$

where x_1 is its original length, t is the force applied, and x_2 is the proportionality constant. Suppose that the following measurements are taken:

t	10	15	20
y	11.60	11.85	12.25

(a) Set up the overdetermined 3×2 system of linear equations corresponding to the data collected.

(b) Is this system consistent? If not, compute each possible pair of values for (x_1, x_2) obtained by selecting any two of the equations from the system. Is there any reason to prefer any one of these results?

(c) Set up the system of normal equations and solve it to obtain the least squares solution to the overdetermined system. Compare your result with those obtained in part b.

3.3. Set up the linear least squares system $\mathbf{Ax} \cong \mathbf{b}$ for fitting the model function $f(t, \mathbf{x}) = x_1 t + x_2 e^t$ to the three data points (1,2), (2,3), (3,5).

Exercise 3.1, 3.3, 3.5:

3.5. Let $\mathbf{x}$ be the solution to the linear least squares problem $\mathbf{Ax} \cong \mathbf{b}$, where

$$\mathbf{A} = \begin{bmatrix} 1 & 0 \\ 1 & 1 \\ 1 & 2 \\ 1 & 3 \end{bmatrix}.$$

Let $\mathbf{r} = \mathbf{b} - \mathbf{Ax}$ be the corresponding residual vector. Which of the following three vectors is a possible value for $\mathbf{r}$? Why?

(a) $\begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \end{bmatrix}$	(b) $\begin{bmatrix} -1 \\ -1 \\ 1 \\ 1 \end{bmatrix}$	(c) $\begin{bmatrix} -1 \\ 1 \\ 1 \\ -1 \end{bmatrix}$
--	--	--

Orthogonal Transformations

We seek a more numerically robust type of transformation that will yield a simpler problem whose solution is the same as that of the original least squares problem but is more easily computed!

Orthogonal Transformations

We now introduce a square real matrix $\mathbf{Q}$ with the property:

$$\mathbf{Q}^T \mathbf{Q} = \mathbf{I}$$

Orthogonal Transformations

We now introduce a square real matrix $\mathbf{Q}$ with the property:

$$\mathbf{Q}^T \mathbf{Q} = \mathbf{I}$$

An orthogonal transformation $\mathbf{Q}$ preserves the Euclidean norm of any vector $\mathbf{v}$:

$$\|\mathbf{Q}\mathbf{v}\|_2^2 = (\mathbf{Q}\mathbf{v})^T \mathbf{Q}\mathbf{v} = \mathbf{v}^T \mathbf{Q}^T \mathbf{Q}\mathbf{v} = \mathbf{v}^T \mathbf{v} = \|\mathbf{v}\|_2^2$$

Orthogonal Transformations

We now introduce a square real matrix $\mathbf{Q}$ with the property:

$$\mathbf{Q}^T \mathbf{Q} = \mathbf{I}$$

An orthogonal transformation $\mathbf{Q}$ preserves the Euclidean norm of any vector $\mathbf{v}$:

$$\|\mathbf{Q}\mathbf{v}\|_2^2 = (\mathbf{Q}\mathbf{v})^T \mathbf{Q}\mathbf{v} = \mathbf{v}^T \mathbf{Q}^T \mathbf{Q}\mathbf{v} = \mathbf{v}^T \mathbf{v} = \|\mathbf{v}\|_2^2$$

Their norm-preserving property means that they do not amplify error

Orthogonal Transformations

As we did with square linear systems, let us consider least squares problems having an upper triangular matrix:

$$\begin{bmatrix} \mathbf{R} \\ \mathbf{O} \end{bmatrix} \mathbf{x} \approx \begin{bmatrix} \mathbf{c}_1 \\ \mathbf{c}_2 \end{bmatrix}$$

where $\mathbf{R}$ is an $n \times n$ upper triangular matrix

$$\|\mathbf{r}\|_2^2 = \|\mathbf{c}_1 - \mathbf{R}\mathbf{x}\|_2^2 + \|\mathbf{c}_2\|_2^2$$

Orthogonal Transformations

$$\|\boldsymbol{r}\|_2^2 = \|\boldsymbol{c}_1 - \boldsymbol{R}\boldsymbol{x}\|_2^2 + \|\boldsymbol{c}_2\|_2^2$$

Orthogonal Transformations

$$\|\mathbf{r}\|_2^2 = \|\mathbf{c}_1 - \mathbf{R}\mathbf{x}\|_2^2 + \|\mathbf{c}_2\|_2^2$$

Solution:

$$\mathbf{R}\mathbf{x} = \mathbf{c}_1$$

Orthogonal Transformations

$$\|\mathbf{r}\|_2^2 = \|\mathbf{c}_1 - \mathbf{R}\mathbf{x}\|_2^2 + \|\mathbf{c}_2\|_2^2$$

Solution:
(Back-substitution)

$$\mathbf{R}\mathbf{x} = \mathbf{c}_1$$

Minimum sum of squares:

$$\|\mathbf{r}\|_2^2 = \|\mathbf{c}_2\|_2^2$$

QR Factorisation

How can we create such a system?

QR Factorisation

*Orthogonal transformation to **triangular form** is accomplished by the QR factorization:*

$$\mathbf{A} = \mathbf{Q} \begin{bmatrix} \mathbf{R} \\ \mathbf{O} \end{bmatrix}$$

$\mathbf{Q}$ is a $m \times m$ orthogonal matrix and $\mathbf{R}$ is an $n \times n$ upper triangular matrix.

QR Factorisation

Is this substitution a solution?

$$\mathbf{A} = \mathbf{Q} \begin{bmatrix} \mathbf{R} \\ \mathbf{O} \end{bmatrix}$$

QR Factorisation

Is this substitution a solution?

$$\mathbf{A} = \mathbf{Q} \begin{bmatrix} \mathbf{R} \\ \mathbf{O} \end{bmatrix}$$

$$\|\mathbf{r}\|_2^2 = \|\mathbf{b} - \mathbf{A}\mathbf{x}\|_2^2 = \|\mathbf{b} - \mathbf{Q} \begin{bmatrix} \mathbf{R} \\ \mathbf{O} \end{bmatrix} \mathbf{x}\|_2^2 = \|\mathbf{Q}^T \mathbf{b} - \begin{bmatrix} \mathbf{R} \\ \mathbf{O} \end{bmatrix} \mathbf{x}\|_2^2 = \|\mathbf{c}_1 - \mathbf{R}\mathbf{x}\|_2^2 + \|\mathbf{c}_2\|_2^2$$



$$\mathbf{Q}^T \mathbf{b} = \begin{bmatrix} \mathbf{c}_1 \\ \mathbf{c}_2 \end{bmatrix}$$

QR Factorisation

Is this substitution a solution?

$$\mathbf{A} = \mathbf{Q} \begin{bmatrix} \mathbf{R} \\ \mathbf{O} \end{bmatrix}$$

$$\|\mathbf{r}\|_2^2 = \|\mathbf{b} - \mathbf{A}\mathbf{x}\|_2^2 = \|\mathbf{b} - \mathbf{Q} \begin{bmatrix} \mathbf{R} \\ \mathbf{O} \end{bmatrix} \mathbf{x}\|_2^2 = \|\mathbf{Q}^T \mathbf{b} - \begin{bmatrix} \mathbf{R} \\ \mathbf{O} \end{bmatrix} \mathbf{x}\|_2^2 = \|\mathbf{c}_1 - \mathbf{R}\mathbf{x}\|_2^2 + \|\mathbf{c}_2\|_2^2$$



How can we turn our system into this form?

$$\mathbf{Q}^T \mathbf{b} = \begin{bmatrix} \mathbf{c}_1 \\ \mathbf{c}_2 \end{bmatrix}$$

Householder Transformation

One way is through the Householder Transformation:

$$\mathbf{H}_n \cdots \mathbf{H}_1 \mathbf{A} = \begin{bmatrix} \mathbf{R} \\ \mathbf{O} \end{bmatrix}$$

Again: The idea is to create an upper triangular matrix form - in case $\mathbf{R}$

Householder Transformation

This transformation is based on the following form:

$$\mathbf{H} = \mathbf{I} - 2 \frac{\mathbf{v}\mathbf{v}^T}{\mathbf{v}^T \mathbf{v}}$$

Householder Transformation

This transformation is based on the following form:

$$\mathbf{H} = \mathbf{I} - 2 \frac{\mathbf{v}\mathbf{v}^T}{\mathbf{v}^T \mathbf{v}}$$

Let us now find a form for the vector $\mathbf{v}$

Householder Transformation

Given a vector $\mathbf{a}$, we want:



$$H\mathbf{a} = \begin{bmatrix} \alpha \\ 0 \\ \vdots \\ 0 \end{bmatrix} = \alpha \begin{bmatrix} 1 \\ 0 \\ \vdots \\ 0 \end{bmatrix} = \alpha \mathbf{e}_1$$

Householder Transformation

Given a vector $\mathbf{a}$, we want:

$$\mathbf{H}\mathbf{a} = \begin{bmatrix} \alpha \\ 0 \\ \vdots \\ 0 \end{bmatrix} = \alpha \begin{bmatrix} 1 \\ 0 \\ \vdots \\ 0 \end{bmatrix} = \alpha \mathbf{e}_1$$



$$\alpha \mathbf{e}_1 = \mathbf{H}\mathbf{a} = \left(\mathbf{I} - 2 \frac{\mathbf{v}\mathbf{v}^T}{\mathbf{v}^T\mathbf{v}} \right) \mathbf{a} = \mathbf{a} - 2\mathbf{v} \frac{\mathbf{v}^T\mathbf{a}}{\mathbf{v}^T\mathbf{v}}$$

Householder Transformation

Given a vector $\mathbf{a}$, we want:

$$\mathbf{H}\mathbf{a} = \begin{bmatrix} \alpha \\ 0 \\ \vdots \\ 0 \end{bmatrix} = \alpha \begin{bmatrix} 1 \\ 0 \\ \vdots \\ 0 \end{bmatrix} = \alpha \mathbf{e}_1$$



$$\alpha \mathbf{e}_1 = \mathbf{H}\mathbf{a} = \left(\mathbf{I} - 2 \frac{\mathbf{v}\mathbf{v}^T}{\mathbf{v}^T\mathbf{v}} \right) \mathbf{a} = \mathbf{a} - 2\mathbf{v} \frac{\mathbf{v}^T\mathbf{a}}{\mathbf{v}^T\mathbf{v}}$$



$$\mathbf{v} = (\mathbf{a} - \alpha \mathbf{e}_1) \frac{\mathbf{v}^T\mathbf{v}}{2\mathbf{v}^T\mathbf{a}}$$

Householder Transformation

$$\mathbf{v} = (\mathbf{a} - \alpha \mathbf{e}_1) \frac{\mathbf{v}^T \mathbf{v}}{2\mathbf{v}^T \mathbf{a}}$$



$$\mathbf{v} = \mathbf{a} - \alpha \mathbf{e}_1$$

Householder Transformation

$$\mathbf{v} = (\mathbf{a} - \alpha \mathbf{e}_1) \frac{\mathbf{v}^T \mathbf{v}}{2\mathbf{v}^T \mathbf{a}}$$



$$\mathbf{v} = \mathbf{a} - \alpha \mathbf{e}_1$$

$$\alpha = \pm \|\mathbf{a}\|_2$$

$$\alpha = -\text{sign}(a_1) \|\mathbf{a}\|_2$$

Householder Transformation

Consider:

$$\mathbf{a} = \begin{bmatrix} 2 \\ 1 \\ 2 \end{bmatrix}$$

Calculate:

Use:

$$\mathbf{v} = \mathbf{a} - \alpha \mathbf{e}_1$$

$$\alpha = -\text{sign}(a_1) \|\mathbf{a}\|_2$$

Householder Transformation

Consider:

$$\mathbf{a} = \begin{bmatrix} 2 \\ 1 \\ 2 \end{bmatrix}$$

Calculate:

$$\mathbf{v} = \mathbf{a} - \alpha \mathbf{e}_1 = \begin{bmatrix} 2 \\ 1 \\ 2 \end{bmatrix} - \alpha \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 2 \\ 1 \\ 2 \end{bmatrix} - \begin{bmatrix} \alpha \\ 0 \\ 0 \end{bmatrix}$$

$$\alpha = \pm \|\mathbf{a}\|_2 = \pm 3$$

Use:

$$\mathbf{v} = \mathbf{a} - \alpha \mathbf{e}_1$$

$$\alpha = -\text{sign}(a_1) \|\mathbf{a}\|_2$$

Householder Transformation

Consider:

$$\mathbf{a} = \begin{bmatrix} 2 \\ 1 \\ 2 \end{bmatrix}$$

Use:

$$\mathbf{v} = \mathbf{a} - \alpha \mathbf{e}_1$$

$$\alpha = -\text{sign}(a_1) \|\mathbf{a}\|_2$$

Calculate:

$$\mathbf{v} = \mathbf{a} - \alpha \mathbf{e}_1 = \begin{bmatrix} 2 \\ 1 \\ 2 \end{bmatrix} - \alpha \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 2 \\ 1 \\ 2 \end{bmatrix} - \begin{bmatrix} \alpha \\ 0 \\ 0 \end{bmatrix}$$

$$\alpha = \pm \|\mathbf{a}\|_2 = \pm 3$$

$$\mathbf{v} = \begin{bmatrix} 2 \\ 1 \\ 2 \end{bmatrix} - \begin{bmatrix} -3 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 5 \\ 1 \\ 2 \end{bmatrix}$$

Householder Transformation

Consider:

$$\mathbf{a} = \begin{bmatrix} 2 \\ 1 \\ 2 \end{bmatrix}$$

Use:

$$\mathbf{v} = \mathbf{a} - \alpha \mathbf{e}_1$$

$$\alpha = -\text{sign}(a_1) \|\mathbf{a}\|_2$$

Calculate:

$$\mathbf{v} = \mathbf{a} - \alpha \mathbf{e}_1 = \begin{bmatrix} 2 \\ 1 \\ 2 \end{bmatrix} - \alpha \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 2 \\ 1 \\ 2 \end{bmatrix} - \begin{bmatrix} \alpha \\ 0 \\ 0 \end{bmatrix}$$

$$\alpha = \pm \|\mathbf{a}\|_2 = \pm 3$$

$$\mathbf{v} = \begin{bmatrix} 2 \\ 1 \\ 2 \end{bmatrix} - \begin{bmatrix} -3 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 5 \\ 1 \\ 2 \end{bmatrix}$$

(Check)

$$\mathbf{H}\mathbf{a} = \mathbf{a} - 2 \frac{\mathbf{v}^T \mathbf{a}}{\mathbf{v}^T \mathbf{v}} \mathbf{v} = \begin{bmatrix} 2 \\ 1 \\ 2 \end{bmatrix} - 2 \frac{15}{30} \begin{bmatrix} 5 \\ 1 \\ 2 \end{bmatrix} = \begin{bmatrix} -3 \\ 0 \\ 0 \end{bmatrix}$$

Householder Transformation

We now have a transformation that “annihilates” all but the first component of the vector. As for the LU decomposition, we can now imagine to do this repetitive until the upper triangular matrix $\mathbf{R}$ has been constructed.

We now put forward the following algorithm:

$$\mathbf{a} = \begin{bmatrix} \mathbf{a}_1 \\ \mathbf{a}_2 \end{bmatrix} \text{ (} m\text{-vector)}$$

where $\mathbf{a}_1$ is a $(k - 1)$ -vector, $1 \leq k < m$

$$\mathbf{v} = \begin{bmatrix} \mathbf{0} \\ \mathbf{a}_2 \end{bmatrix} - \alpha \mathbf{e}_k$$

where $\alpha = -\text{sign}(a_k) \|\mathbf{a}_2\|_2$

“The Householder transformation here annihilates the last $m-k$ components of $\mathbf{a}$ ”

Householder Transformation

Important property:

$$\mathbf{H}\mathbf{u} = \left(\mathbf{I} - 2 \frac{\mathbf{v}\mathbf{v}^T}{\mathbf{v}^T\mathbf{v}} \right) \mathbf{u} = \mathbf{u} - \left(2 \frac{\mathbf{v}^T\mathbf{u}}{\mathbf{v}^T\mathbf{v}} \right) \mathbf{v}$$

You don't have to create the full $\mathbf{H}$
- only store $\mathbf{v}$

Householder Transformation

This leads us to the following solution:

$$\mathbf{H}_n \cdots \mathbf{H}_1 \mathbf{A} = \begin{bmatrix} \mathbf{R} \\ \mathbf{O} \end{bmatrix}$$

Householder Transformation

This leads us to the following solution:

$$\mathbf{H}_n \cdots \mathbf{H}_1 \mathbf{A} = \begin{bmatrix} \mathbf{R} \\ \mathbf{O} \end{bmatrix}$$

$$\begin{bmatrix} \mathbf{R} \\ \mathbf{O} \end{bmatrix} \mathbf{x} \cong \mathbf{Q}^T \mathbf{b} = \begin{bmatrix} \mathbf{c}_1 \\ \mathbf{c}_2 \end{bmatrix}$$

Householder Transformation

This leads us to the following solution:

$$\mathbf{H}_n \cdots \mathbf{H}_1 \mathbf{A} = \begin{bmatrix} \mathbf{R} \\ \mathbf{O} \end{bmatrix}$$

This also shows one solution to the QR factorisation:

$$\begin{array}{ccc} \mathbf{Q}^T = \mathbf{H}_n \cdots \mathbf{H}_1 & \longrightarrow & \mathbf{A} = \mathbf{Q} \begin{bmatrix} \mathbf{R} \\ \mathbf{O} \end{bmatrix} \\ \mathbf{Q} = \mathbf{H}_1 \cdots \mathbf{H}_n & & \end{array}$$

Householder Transformation

$$\mathbf{A}\mathbf{x} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \\ -1 & 1 & 0 \\ -1 & 0 & 1 \\ 0 & -1 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} \cong \begin{bmatrix} 1237 \\ 1941 \\ 2417 \\ 711 \\ 1177 \\ 475 \end{bmatrix} = \mathbf{b}$$

Last time:

$$\mathbf{A}^T \mathbf{A}\mathbf{x} = \begin{bmatrix} 3 & -1 & -1 \\ -1 & 3 & -1 \\ -1 & -1 & 3 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} -651 \\ 2177 \\ 4069 \end{bmatrix} = \mathbf{A}^T \mathbf{b}$$

$$\mathbf{x} = [1236, 1943, 2416]^T$$

Householder Transformation

$$\mathbf{A}\mathbf{x} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \\ -1 & 1 & 0 \\ -1 & 0 & 1 \\ 0 & -1 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} \cong \begin{bmatrix} 1237 \\ 1941 \\ 2417 \\ 711 \\ 1177 \\ 475 \end{bmatrix} = \mathbf{b}$$

$$\mathbf{v}_1 = \mathbf{a}_1 - \alpha \mathbf{e}_1 = \begin{bmatrix} 1 \\ 0 \\ 0 \\ -1 \\ -1 \\ 0 \end{bmatrix} - \begin{bmatrix} -1.7321 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 2.7321 \\ 0 \\ 0 \\ -1 \\ -1 \\ 0 \end{bmatrix}$$

where $\alpha = -\text{sign}(a_k) \|\mathbf{a}_2\|_2$

From this construct:

$$\mathbf{H} = \mathbf{I} - 2 \frac{\mathbf{v}\mathbf{v}^T}{\mathbf{v}^T\mathbf{v}}$$

Householder Transformation

$$Ax = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \\ -1 & 1 & 0 \\ -1 & 0 & 1 \\ 0 & -1 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} \cong \begin{bmatrix} 1237 \\ 1941 \\ 2417 \\ 711 \\ 1177 \\ 475 \end{bmatrix} = b$$

Remember the form:

$$Ax \cong b$$

$$H = I - 2 \frac{vv^T}{v^T v}$$

(Left side)

$$H_1 A = \begin{bmatrix} -1.7321 & 0.5774 & 0.5774 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \\ 0 & 0.7887 & -0.2113 \\ 0 & -0.2113 & 0.7887 \\ 0 & -1 & 1 \end{bmatrix}$$

(Right side)

$$H_1 b = \begin{bmatrix} 376 \\ 1941 \\ 2417 \\ 1026 \\ 1492 \\ 475 \end{bmatrix}$$

Householder Transformation

$$\mathbf{v}_2 = \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0.7887 \\ -0.2113 \\ -1 \end{bmatrix} - \begin{bmatrix} 0 \\ -1.6330 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 2.6330 \\ 0 \\ 0.7887 \\ -0.2113 \\ -1 \end{bmatrix}$$

$$\mathbf{H}_2 \mathbf{H}_1 \mathbf{b} = \begin{bmatrix} 376 \\ -1200 \\ 2417 \\ 85 \\ 1744 \\ 1668 \end{bmatrix}$$

$$\mathbf{H}_2 \mathbf{H}_1 \mathbf{A} = \begin{bmatrix} -1.7321 & 0.5774 & 0.5774 \\ 0 & -1.6330 & 0.8165 \\ 0 & 0 & 1 \\ 0 & 0 & 0.0332 \\ 0 & 0 & 0.7231 \\ 0 & 0 & 0.6899 \end{bmatrix}$$

Now do this 1 more time...

QR Factorisation

Factorize:

$$\mathbf{H}_3\mathbf{H}_2\mathbf{H}_1\mathbf{A} = \begin{bmatrix} -1.7321 & 0.5774 & 0.5774 \\ 0 & -1.6330 & 0.8165 \\ 0 & 0 & -1.4142 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} = \begin{bmatrix} \mathbf{R} \\ \mathbf{O} \end{bmatrix}$$

Solution:

$$\mathbf{H}_3\mathbf{H}_2\mathbf{H}_1\mathbf{b} = \begin{bmatrix} 376 \\ -1200 \\ -3417 \\ 5 \\ 3 \\ 1 \end{bmatrix} = \mathbf{Q}^T\mathbf{b} = \begin{bmatrix} \mathbf{c}_1 \\ \mathbf{c}_2 \end{bmatrix}$$

QR Factorisation

Factorize:

$$H_3 H_2 H_1 A = \begin{bmatrix} -1.7321 & 0.5774 & 0.5774 \\ 0 & -1.6330 & 0.8165 \\ 0 & 0 & -1.4142 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} = \begin{bmatrix} R \\ O \end{bmatrix}$$

$$H_3 H_2 H_1 b = \begin{bmatrix} 376 \\ -1200 \\ -3417 \\ 5 \\ 3 \\ 1 \end{bmatrix} = Q^T b = \begin{bmatrix} c_1 \\ c_2 \end{bmatrix}$$

Solution:

$$Rx = c_1$$

$$\|r\|_2^2 = \|c_2\|_2^2 = 35$$

$$x = [1236, 1943, 2416]^T$$

$$Hu = \left(I - 2 \frac{vv^T}{v^T v} \right) u = u - \left(2 \frac{v^T u}{v^T v} \right) v$$

Algorithm

Algorithm 3.1 Householder QR Factorization

for $k = 1$ to $\min(n, m - 1)$	{ loop over columns }
$\alpha_k = -\text{sign}(a_{kk}) \sqrt{a_{kk}^2 + \cdots + a_{mk}^2}$	{ compute Householder
$\mathbf{v}_k = [0 \quad \cdots \quad 0 \quad a_{kk} \quad \cdots \quad a_{mk}]^T - \alpha_k \mathbf{e}_k$	vector for current col }
$\beta_k = \mathbf{v}_k^T \mathbf{v}_k$	
if $\beta_k = 0$ then	{ skip current column
continue with next k	if it's already zero }
for $j = k$ to n	{ apply transformation
$\gamma_j = \mathbf{v}_k^T \mathbf{a}_j$	to remaining
$\mathbf{a}_j = \mathbf{a}_j - (2\gamma_j/\beta_k) \mathbf{v}_k$	submatrix }
end	
end	

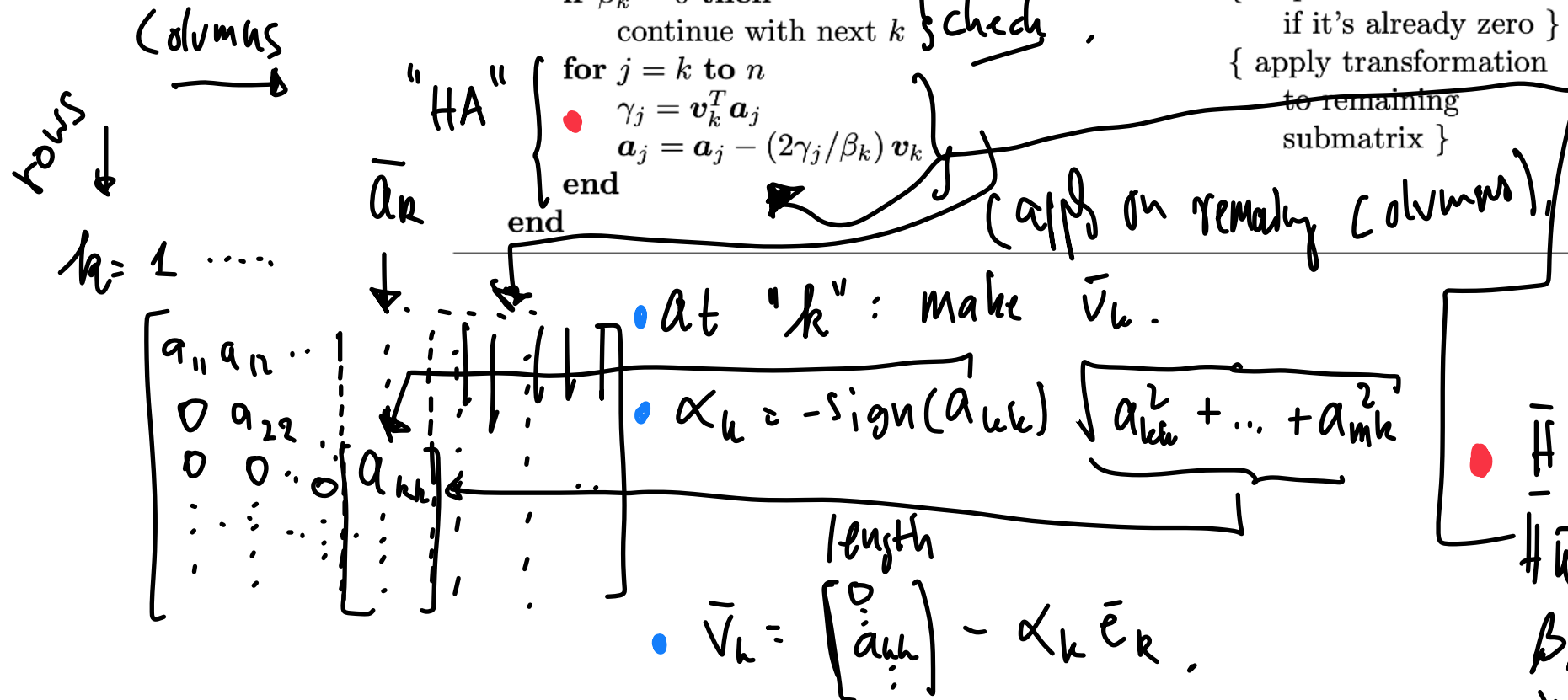
Point is: you don't have to create **H**, but you have to
Perform its transformation to the whole matrix **A**.

$$Hu = \left(I - 2 \frac{vv^T}{v^T v} \right) u = u - \left(2 \frac{v^T u}{v^T v} \right) v$$

Algorithm

Algorithm 3.1 Householder QR Factorization

- for $k = 1$ to $\min(n, m - 1)$ { loop over columns }
 - $\alpha_k = -\text{sign}(a_{kk}) \sqrt{a_{kk}^2 + \dots + a_{mk}^2}$ { compute Householder
 - $v_k = [0 \ \dots \ 0 \ a_{kk} \ \dots \ a_{mk}]^T - \alpha_k e_k$ vector for current col }
 - $\beta_k = v_k^T v_k$
 - if $\beta_k = 0$ then { skip current column
 - continue with next k if it's already zero }
 - for $j = k$ to n { apply transformation
 - $\gamma_j = v_k^T a_j$
 - $a_j = a_j - (2\gamma_j / \beta_k) v_k$ to remaining submatrix }
 - end
- end



Remember:

$$\bar{v} = \begin{bmatrix} 0 \\ a_2 \end{bmatrix} - \alpha \bar{e}_k,$$

$$\bar{a} = \begin{bmatrix} a_1 \\ a_2 \end{bmatrix},$$

$$\bar{a}_1 : (k-1)\text{-vector.}$$

$$\alpha = -\text{sign}(a_{kk}) \|a_{k+1:m,k}\|_2$$

$$\bar{H} = \bar{I} - 2 \frac{\bar{v} \bar{v}^T}{\bar{v}^T \bar{v}}$$

$$\bar{H} \bar{u} = \bar{u} - 2 \frac{(\bar{v}^T \bar{u})}{\bar{v}^T \bar{v}} \bar{v}$$

$$\beta_k = \bar{v}_k^T \bar{v}_k$$

$$\gamma_j = \bar{v}_k^T \bar{a}_j$$

Relevance for the Problem set:

1. $\mathbf{Q}, \mathbf{R} = \text{householder_QR_slow}(\mathbf{A})$, which takes as input a rectangular matrix $\mathbf{M}: m \times n$ and uses the Householder method to compute its QR decomposition. Check that $\mathbf{Q}: m \times m$ is orthogonal, i.e., $\mathbf{Q}^T \mathbf{Q} = \mathbf{Q} \mathbf{Q}^T = \mathbf{I}$, and that the upper triangular matrix $\mathbf{R}: m \times n$ satisfies $\mathbf{M} = \mathbf{Q} \mathbf{R}$.

Relevance for the Problem set:

1. $\mathbf{Q}, \mathbf{R} = \text{householder_QR_slow}(\mathbf{A})$, which takes as input a rectangular matrix $\mathbf{M}: m \times n$ and uses the Householder method to compute its QR decomposition. Check that $\mathbf{Q}: m \times m$ is orthogonal, i.e., $\mathbf{Q}^T \mathbf{Q} = \mathbf{Q} \mathbf{Q}^T = \mathbf{I}$, and that the upper triangular matrix $\mathbf{R}: m \times n$ satisfies $\mathbf{M} = \mathbf{Q} \mathbf{R}$.

Here remember:

(Note that the notations differ from each other!)

$$\mathbf{A} = \mathbf{Q} \begin{bmatrix} \mathbf{R} \\ \mathbf{O} \end{bmatrix} \quad \mathbf{Q} = \mathbf{H}_1 \cdots \mathbf{H}_n \quad \mathbf{H} = \mathbf{I} - 2 \frac{\mathbf{v} \mathbf{v}^T}{\mathbf{v}^T \mathbf{v}}$$

$$\mathbf{Q}^T = \mathbf{H}_n \cdots \mathbf{H}_1$$

Relevance for the Problem set:

2. A more efficient version of (f.1) that doesn't compute the $m \times m$ matrix $\mathbf{Q}$, but just stores the reflection vectors $\mathbf{v}$. You can either give it the interface $\mathbf{V}, \mathbf{R} = \text{householder_fast}(\mathbf{A})$ or (as is done in practice) let the result be a combined $(m+1) \times n$ matrix $\mathbf{VR}$, in which the upper $n \times n$ triangle contains $\mathbf{R}$, and the k th column below the diagonal is $\mathbf{v}_k$ (of length $m - k$).

Review Questions?

3.1. True or false: A linear least squares problem always has a solution.

3.2. True or false: Fitting a straight line to a set of data points is a linear least squares problem, whereas fitting a quadratic polynomial to the data is a nonlinear least squares problem

3.3. True or false: At the solution to a linear least squares problem $\mathbf{Ax} \cong \mathbf{b}$, the residual vector $\mathbf{r} = \mathbf{b} - \mathbf{Ax}$ is orthogonal to $\text{span}(\mathbf{A})$.

3.5. True or false: In solving a linear least squares problem $\mathbf{Ax} \cong \mathbf{b}$, if the vector $\mathbf{b}$ lies in $\text{span}(\mathbf{A})$, then the residual is $\mathbf{0}$.